



École nationale supérieure en systèmes avancés et réseaux

Cycle Ingénieur Esisar

Cours réseaux.

IN363



Introduction

- Qui je suis ? : Jean-Baptiste Caignaert



- Vous êtes en apprentissage et venant de formations différentes
 - > implication 1 : Compétences != en réseaux
 - > implication 2 : Pédagogiquement c'est une problématique

Introduction

Objectif de ce cours :

- > Connaître les couches dites “basses” du réseau
- > être capable de comprendre comment fonctionne le transfert de données

A la fin de ce cours vous saurez notamment (normalement) :

- > comprendre le binaire
- > comprendre la transmission de données
- > comprendre la compression et le checksum d’une trame
- > comprendre une adresse MAC
- > comprendre un chemin de réseau
- > décrypter une trame réseau avec les logiciels idoines

Introduction

Combien d'heures : 8 CM, 6 TD, 4 TP

Notation de ce module :

TP/Projet noté : CC

DS noté : ET1

Note finale : $0,3 * CC + 0,7 * ET1$

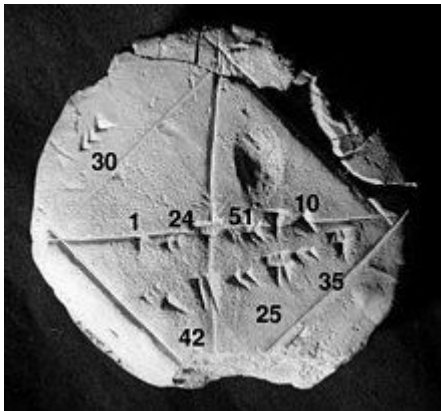
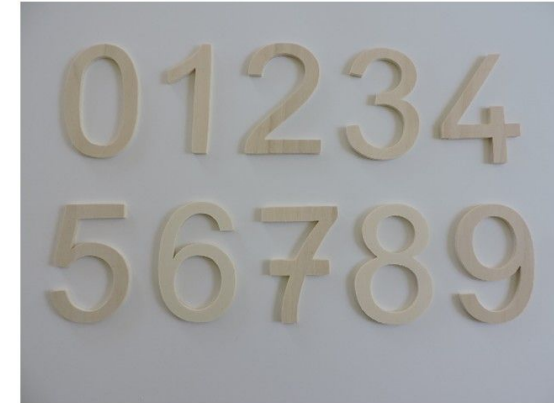
Ma pédagogie est de vous faire manipuler au maximum car c'est un cours traditionnellement très "verbeux" pour les étudiants.

Prélude : calcul en base(s)

Depuis tout petit nous comptons avec la base 10.

Pourquoi 10 ?

Pourquoi pas 11, 12, 2, 3, 60 ?



“Fun” fact : les Babyloniens comptaient en base 60 et voici une pierre qui donne une approximation de $\sqrt{2}$ dans cette base (environ année -1800)

Prélude : calcul en base(s)

Le numérique au sens large utilise la base 2.

Pourquoi ?

On utilise aussi la base 16 (hexadecimal)

Apprenons à compter tous ensemble dans ces 2 bases, cela nous sera indispensable pour la suite.

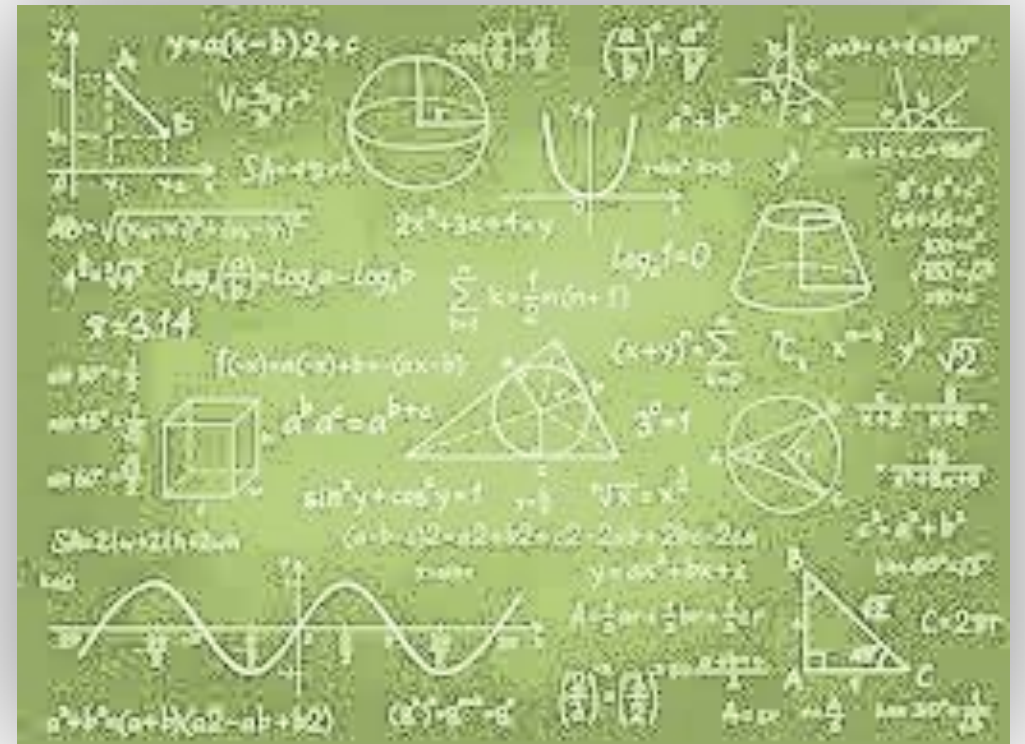
Décimal	Binaire	Héxadécimal
0	0	0
1	1	1
2	10	2
3	11	3
4	100	4
5	101	5
6	110	6
7	111	7
8	1000	8
9	1001	9
10	1010	A
11	1011	B
12	1100	C
13	1101	D
14	1110	E
15	1111	F

Prélude : calcul en base(s)

Traduisez $17_{(10)}$ en base 2 et héra

Traduisez $1A_{(16)}$ en base 10 et 2

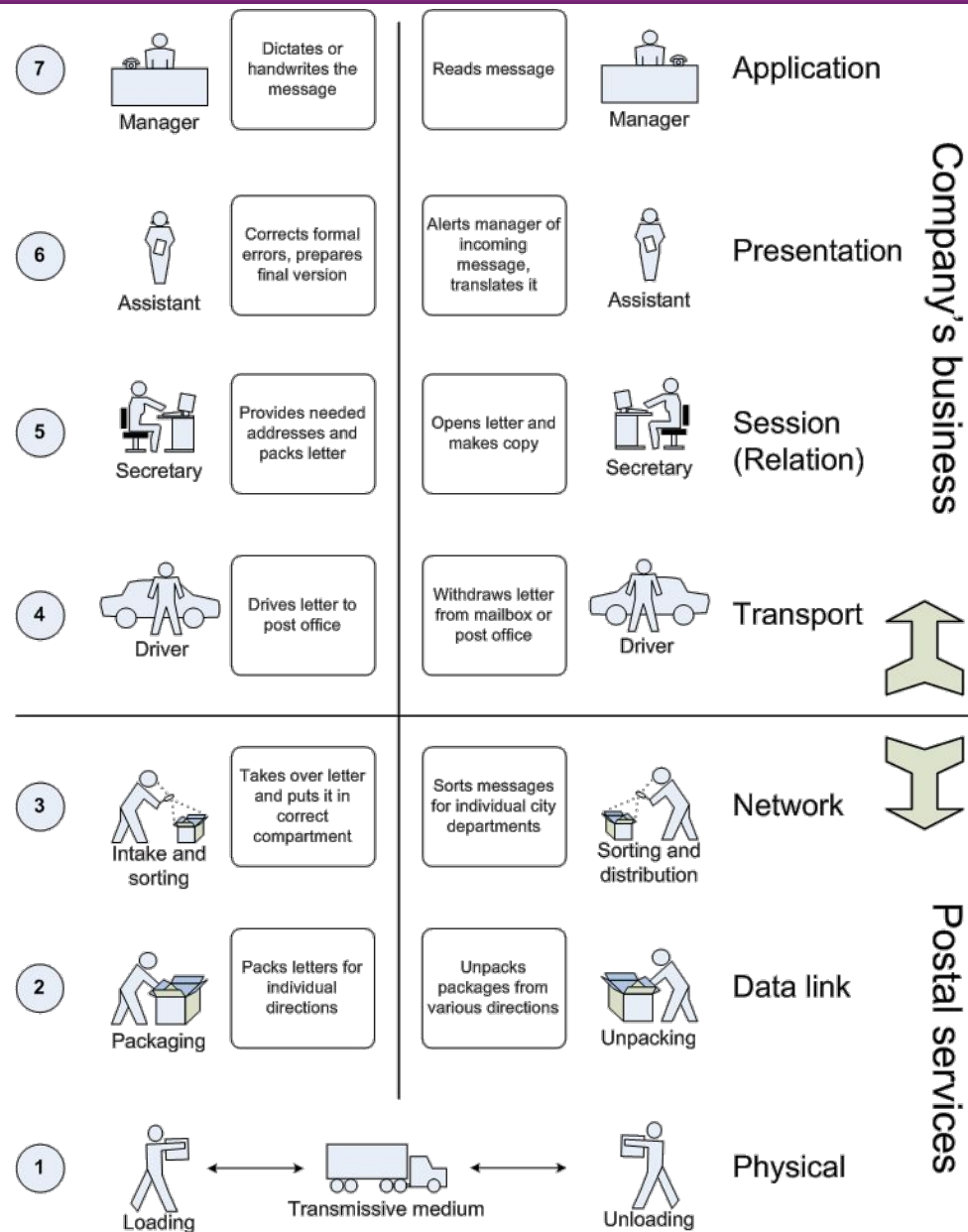
Traduisez $10001101_{(2)}$ en base 16 et 10



Chapitre 1 : Modèle OSI

Modèle validé en 1984 qui a comme objectif de différencier les étapes pour communiquer entre 2 “sites”.

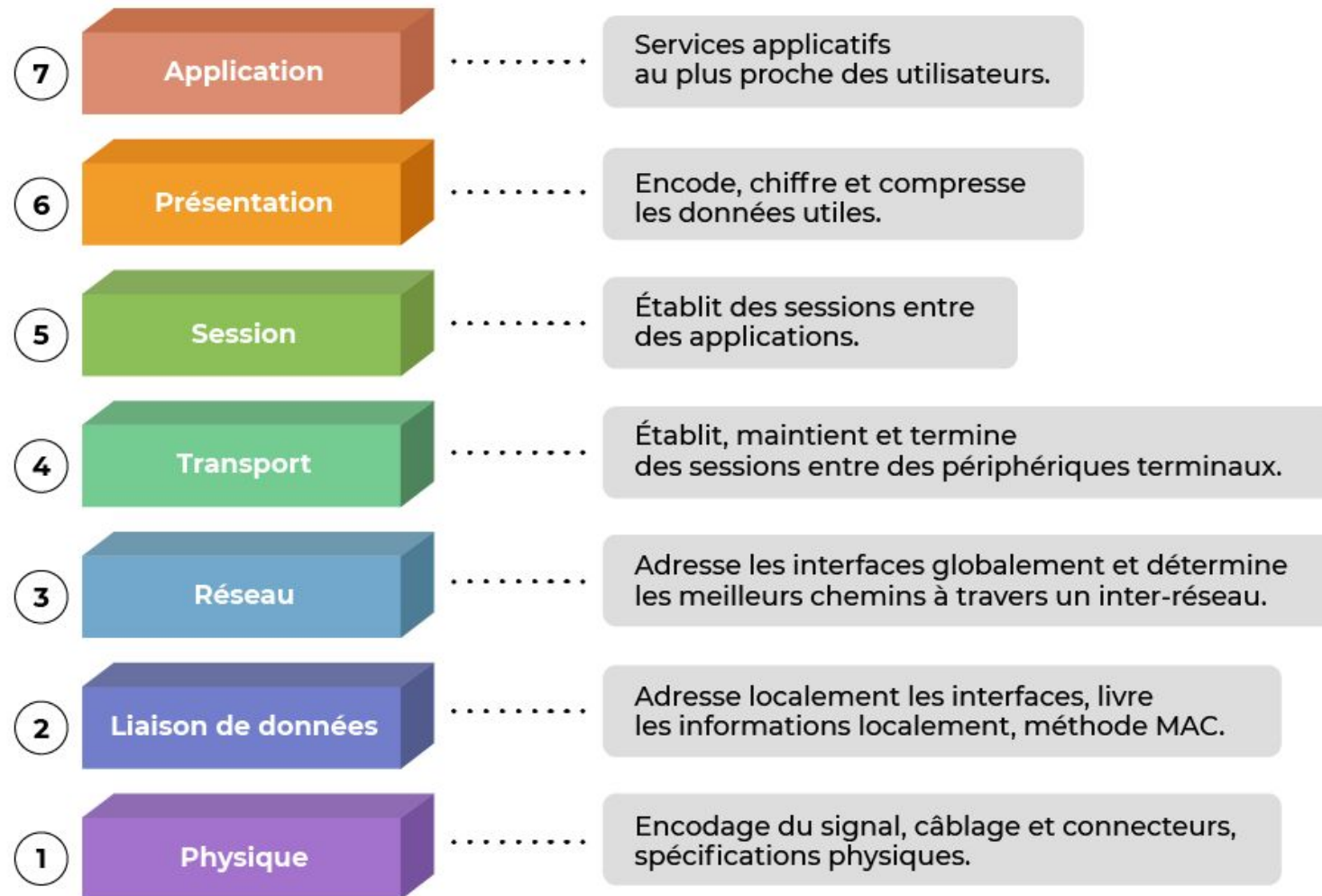
Prenons un exemple réel d’un manager souhaitant envoyer un colis/lettre.



Modèle Osi Métaphoriquement

RM – OSI and letter communication parallel

Modèle OSI

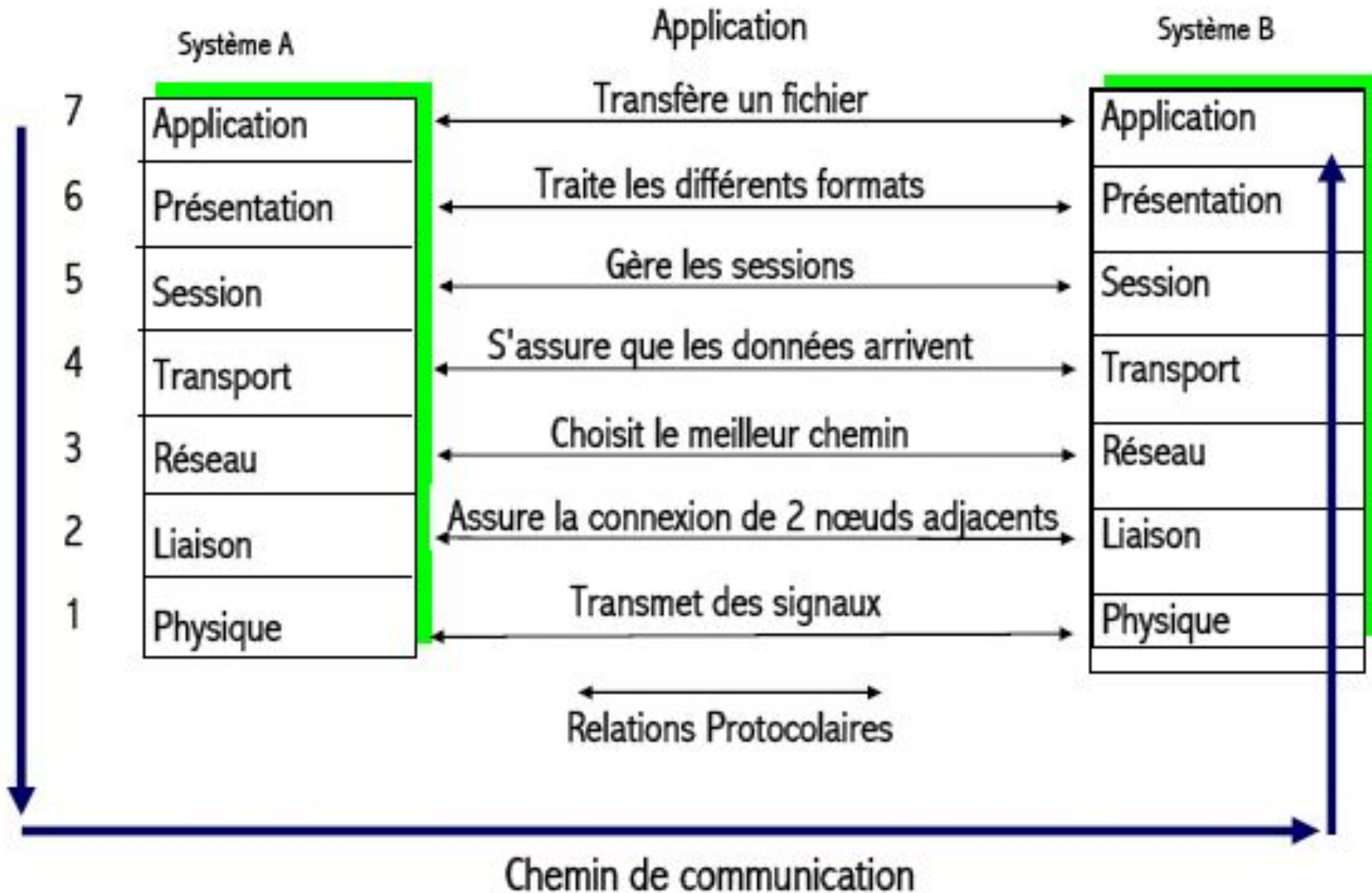


Chapitre 1 : Modèle OSI

Dans ce cours nous allons nous intéresser au 3 couches les plus basses.

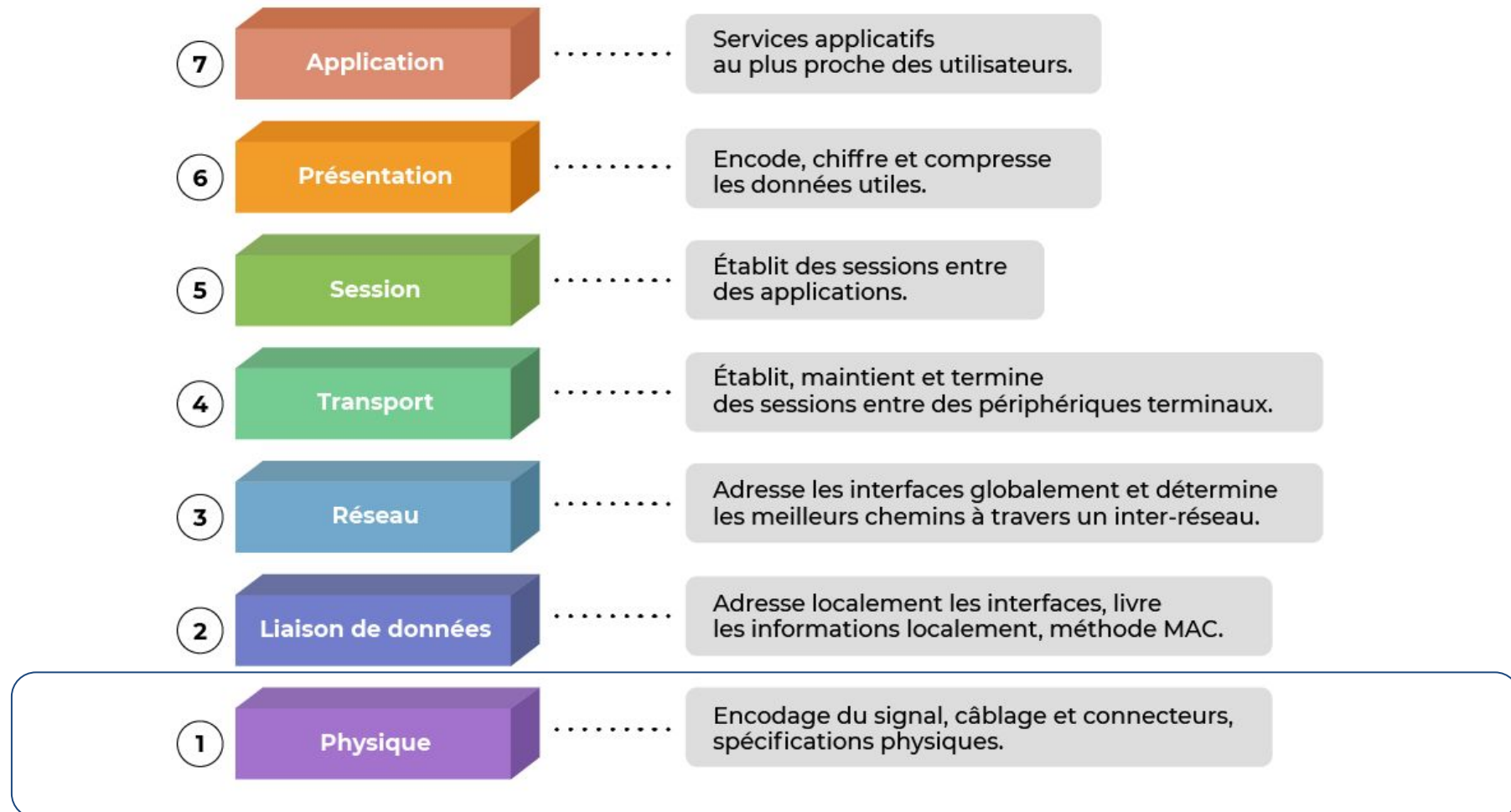
Les fondements de la transmission.

Vous verrez les 4 autres couches, les “applicatives” l’année prochaine.



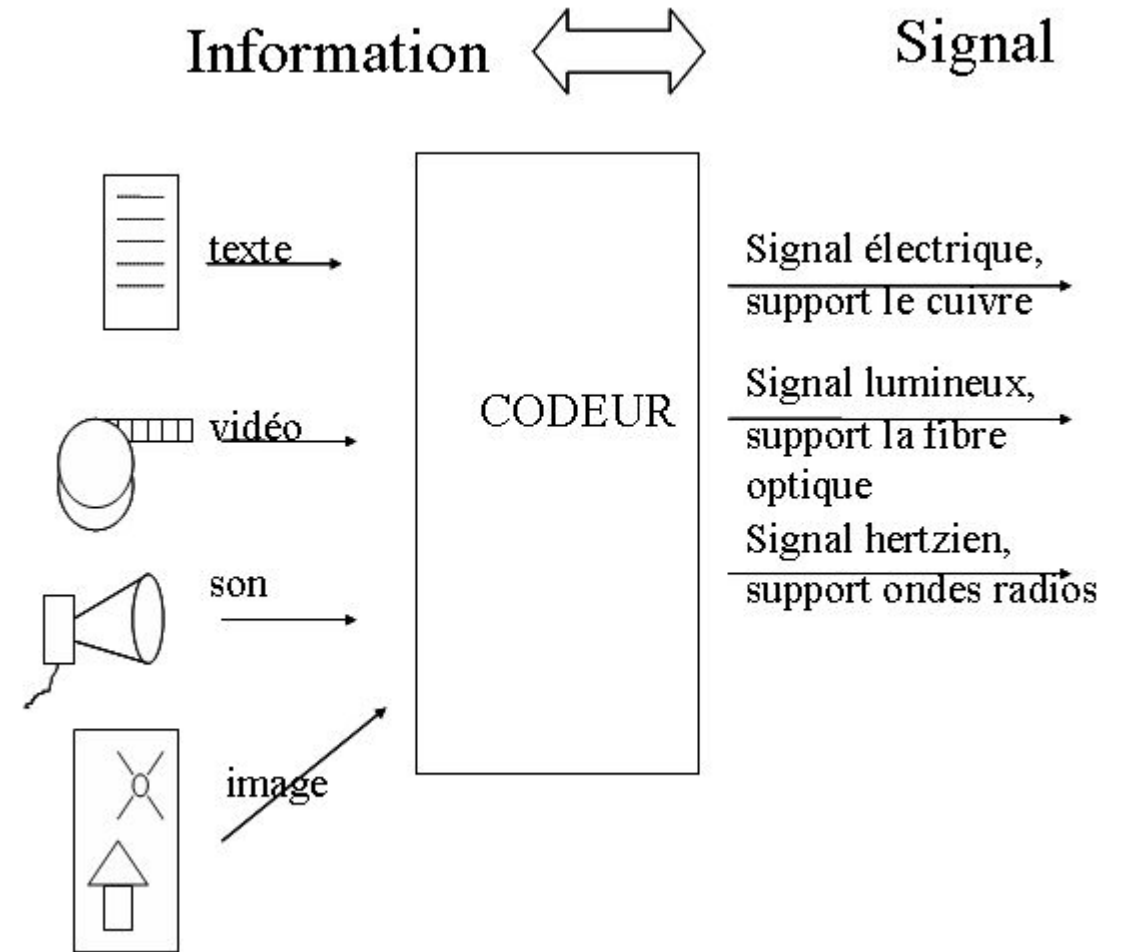
Chapitre 2 : Couche 1

Modèle OSI



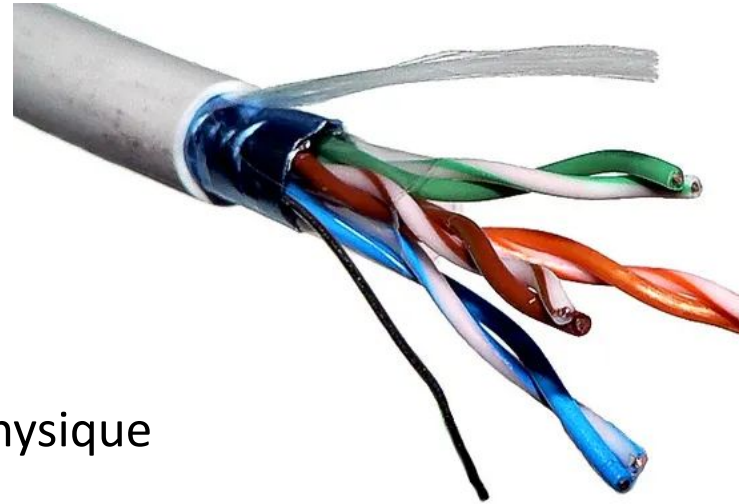
Niveau 1

- Niveau 1 : 3 modes de communication :
 - > câble électrique
 - > cable optique
 - > air (onde)



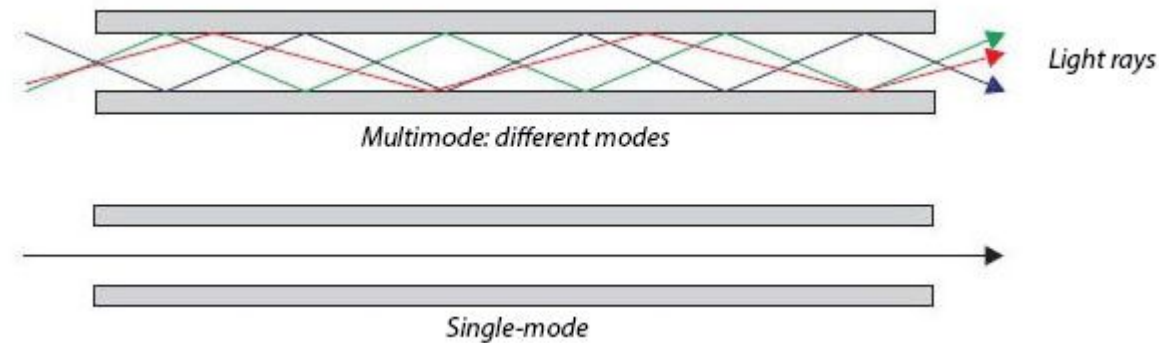
Niveau 1

- Paire torsadé : câble téléphone : cuivre
- Connecteur RJ45 : utilisé comme connectique physique



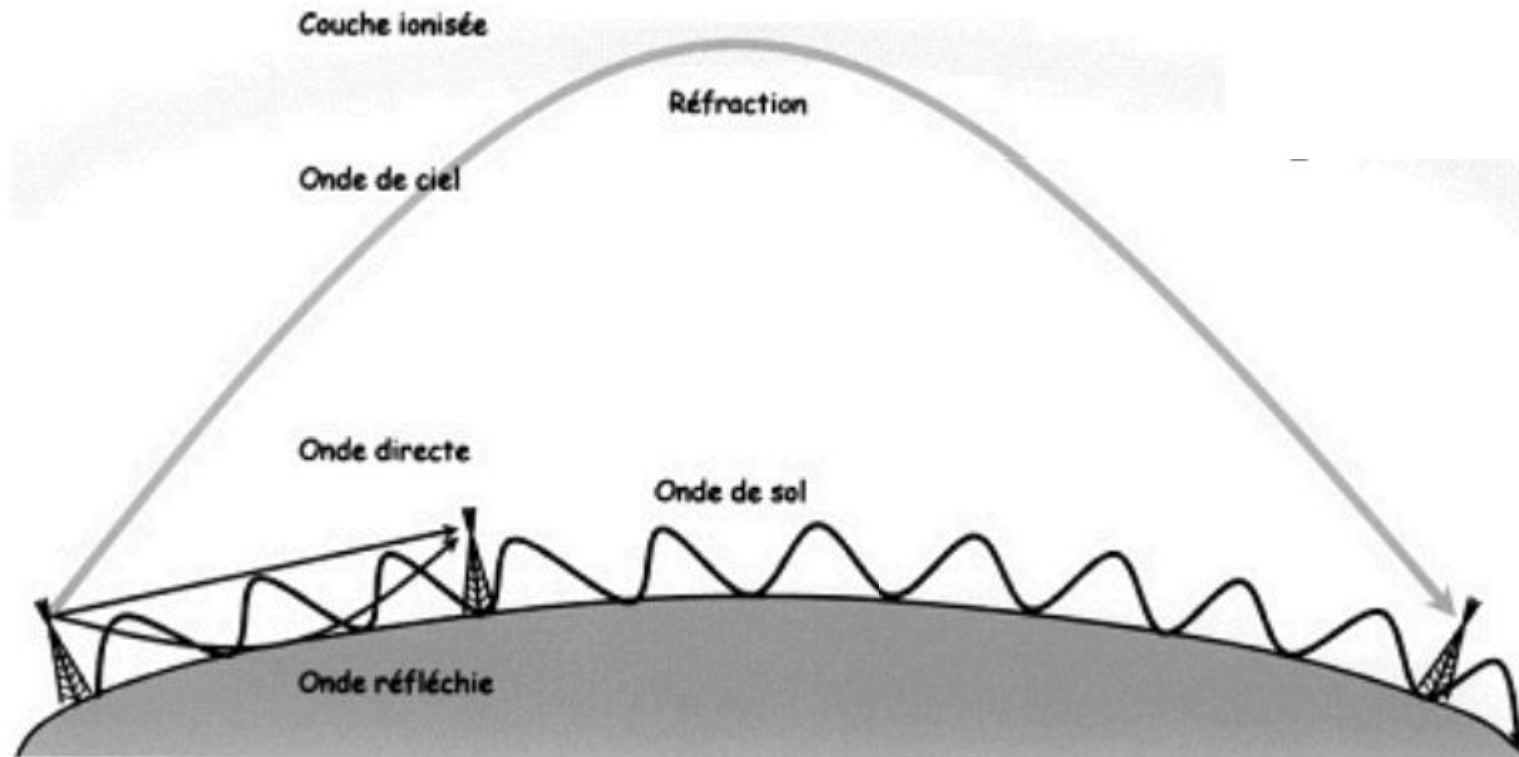
Niveau 1

- Cable optique : la “fibre” : conductrice de lumière, impulsion lumineuse.
Une diode émet un signal lumineux qui est récupéré à l’autre extrémité par une photodiode (qui le transforme en signaux électrique).



Niveau 1

Wifi / 4g ... propagation d'onde dans l'air



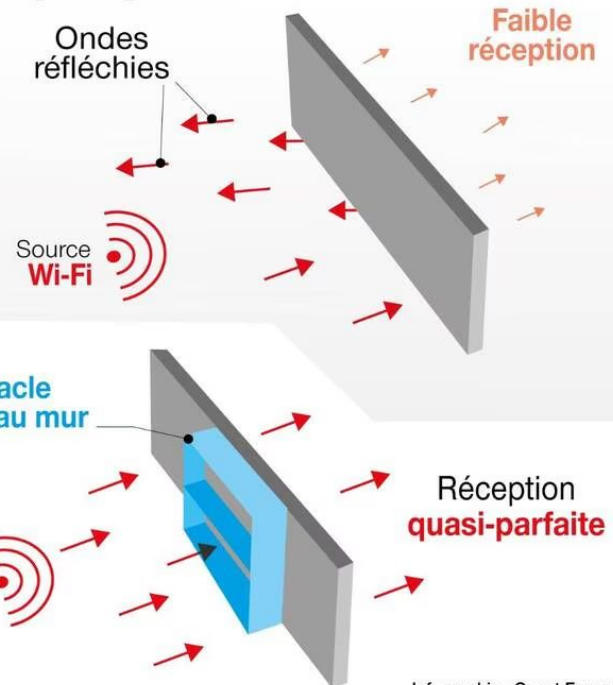
Niveau 1

Fun fact : étude 2022.

Un obstacle peut améliorer le Wi-Fi

Sur un mur en béton entièrement dégagé, une partie des ondes Wi-Fi sont réfléchies, elles « rebondissent » ...

En plaçant un obstacle adapté au même mur de béton avec la même source Wi-Fi, les chercheurs constatent que les ondes passent plus fluidement à travers le mur.



Infographie : Ouest-France.

Transmission
quasi-parfaite

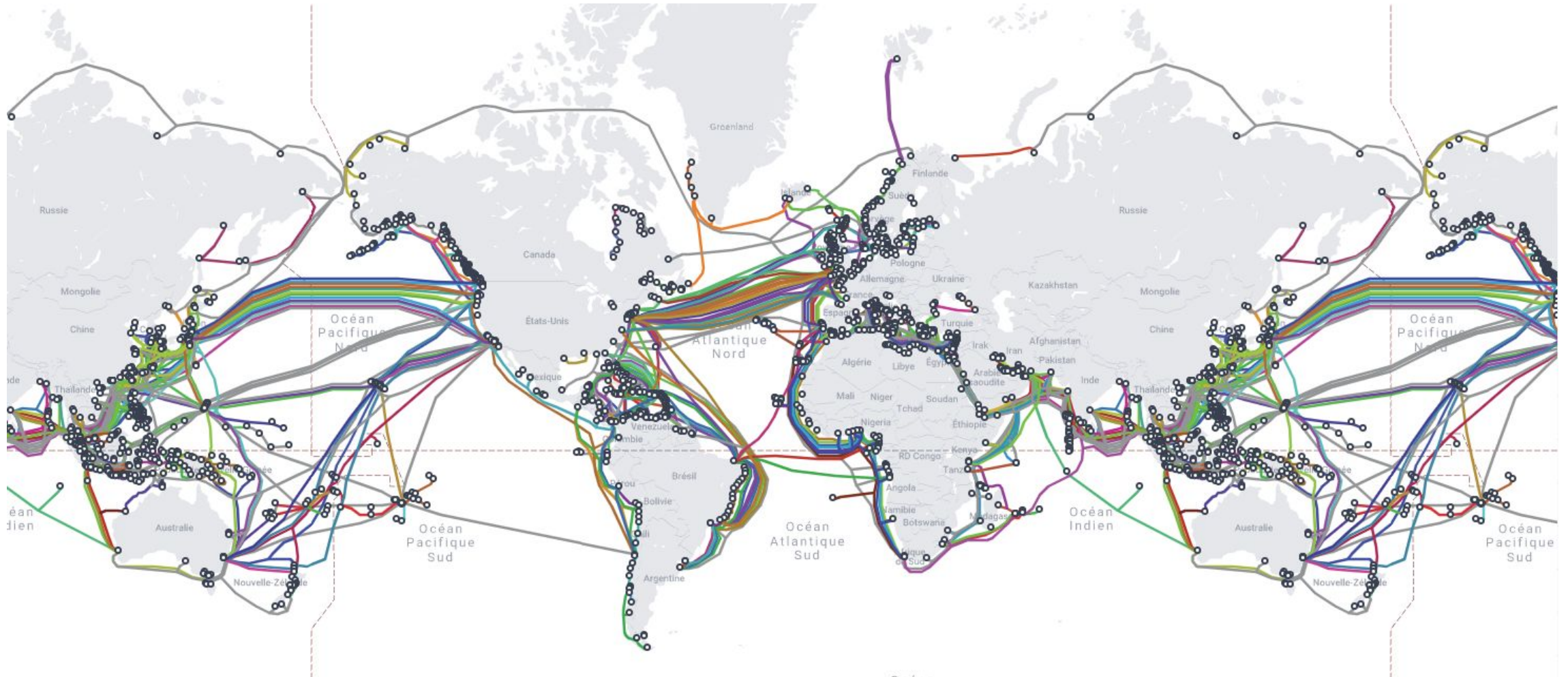
Obstacle additionnel
ajusté au mur

Pas de réflexion

Wi-Fi

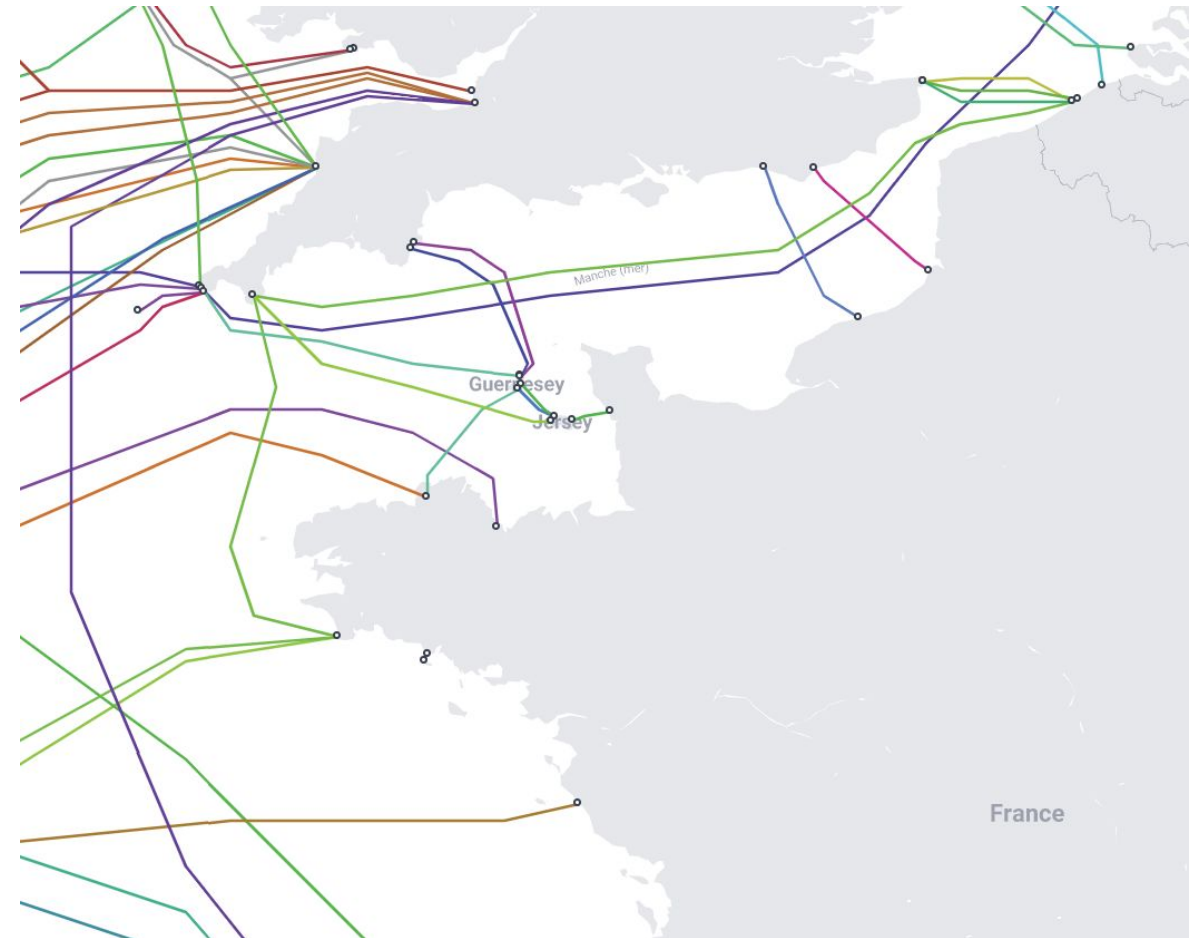
Niveau 1

- Câble sous marin : Internet n'existerait pas sans cela



Niveau 1

- Câble sous marin : Internet n'existerait pas sans cela : 99,8% des transferts intercontinentaux passent par ces câbles.



Niveau 1

- Ressource stratégique, communication, sécurité



Baltique. Deux câbles sous-marins sectionnés, la Suède et l'Allemagne sur la piste d'un sabotage

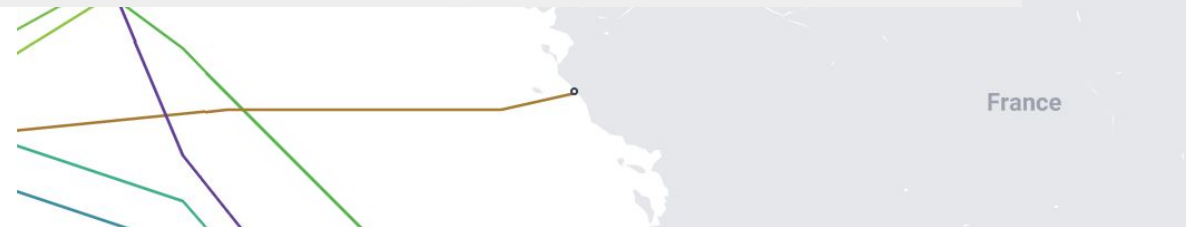
En l'espace de quarante-huit heures, deux câbles de télécommunications ont été coupés en mer Baltique. Le ministre de la Défense allemand a dit partir du principe qu'il s'agissait d'un sabotage.



SOURCE :
Courrier international



Lecture 1 min. 📅 Publié le 19 novembre 2024 à 18h53



Niveau 1

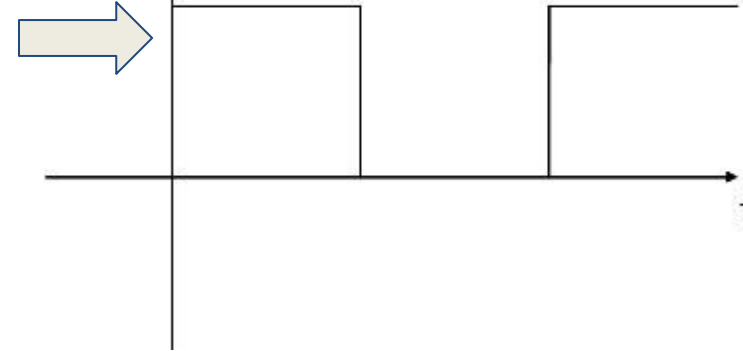
Carte réseau : intégré nativement désormais dans les ordinateurs portables/smartphone ...

Sinon carte réseau s'insère dans la carte mère via un BUS-PCI, la carte réseau peut être wifi ou non, généralement toujours RJ45.

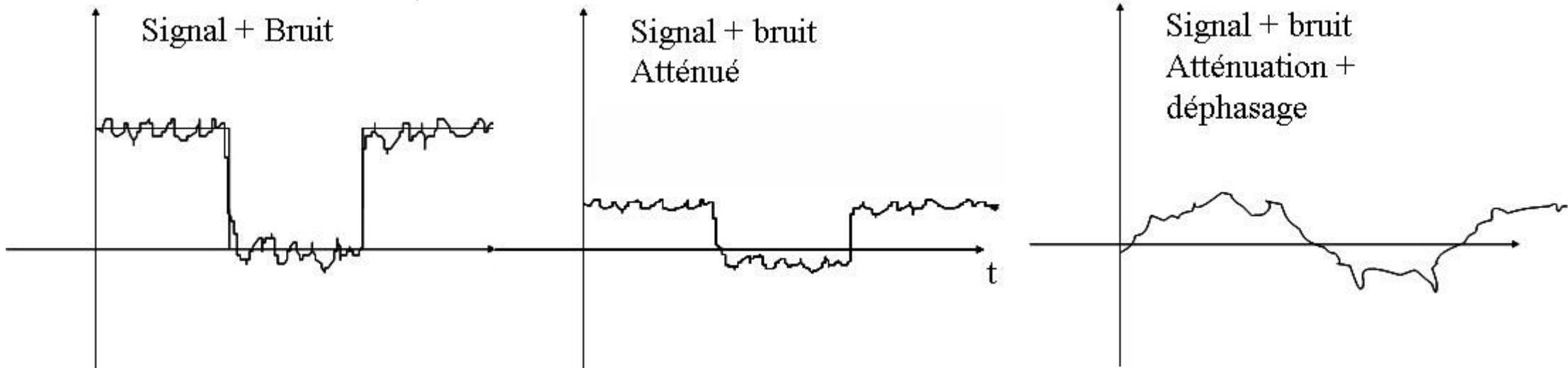


Niveau 1

Théoriquement on envoie des 0 ou des 1 qui seront donc :

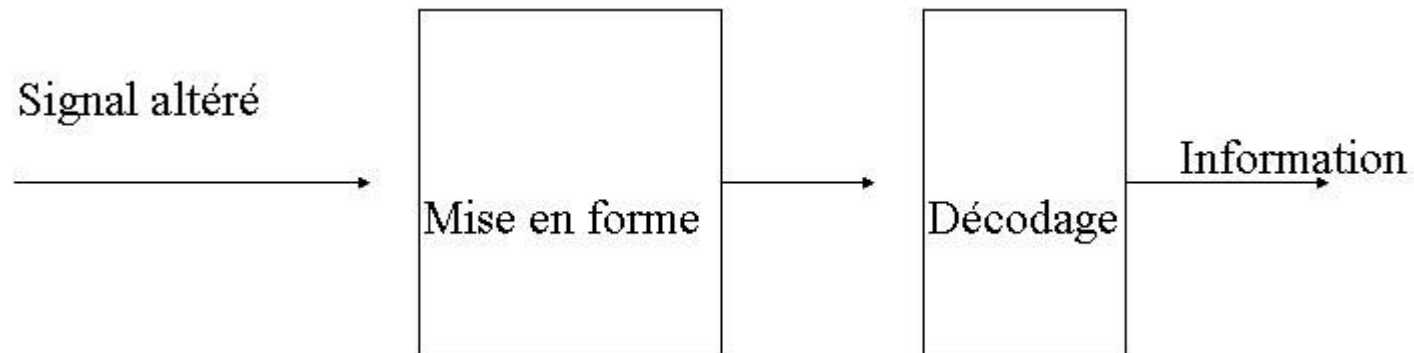


Mais dans la physique "irl" voilà ce qu'on obtient



Niveau 1

On a donc besoin d'un système qui "remet" le signal en information binaire :

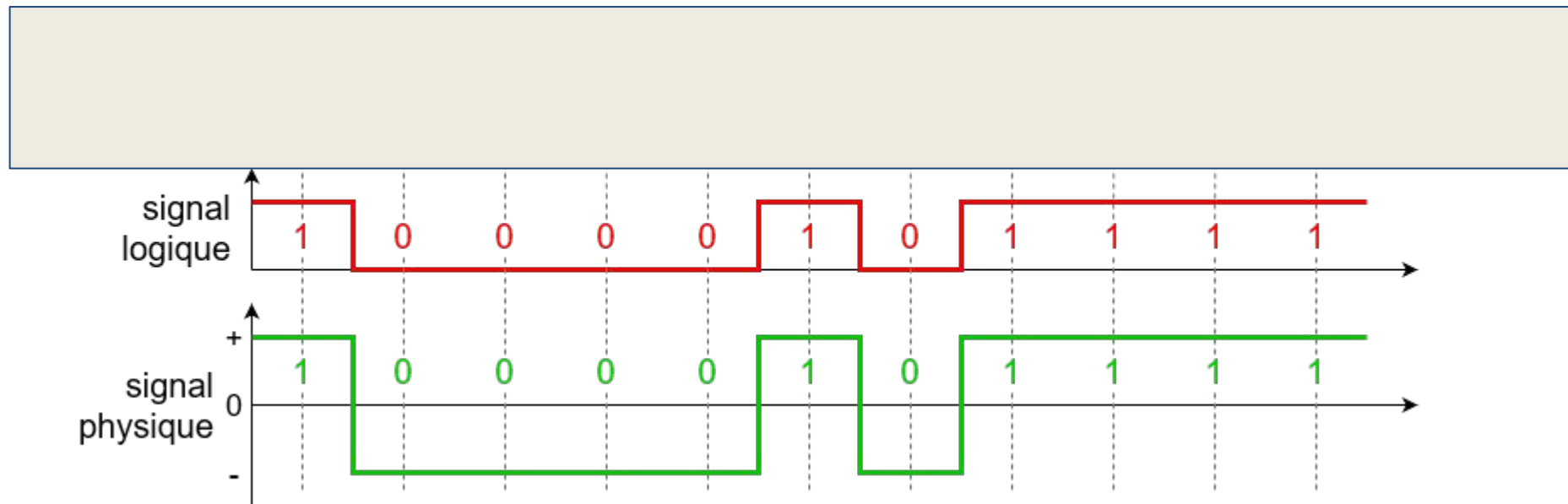


Mais aussi de répartiteur : pour "remettre à jour" le signal

Nous ne verrons pas en détail cette partie dans ce cours, il s'agit de module mathématique pour réaliser cela, nous pouvons donc avoir des "erreurs" d'information.

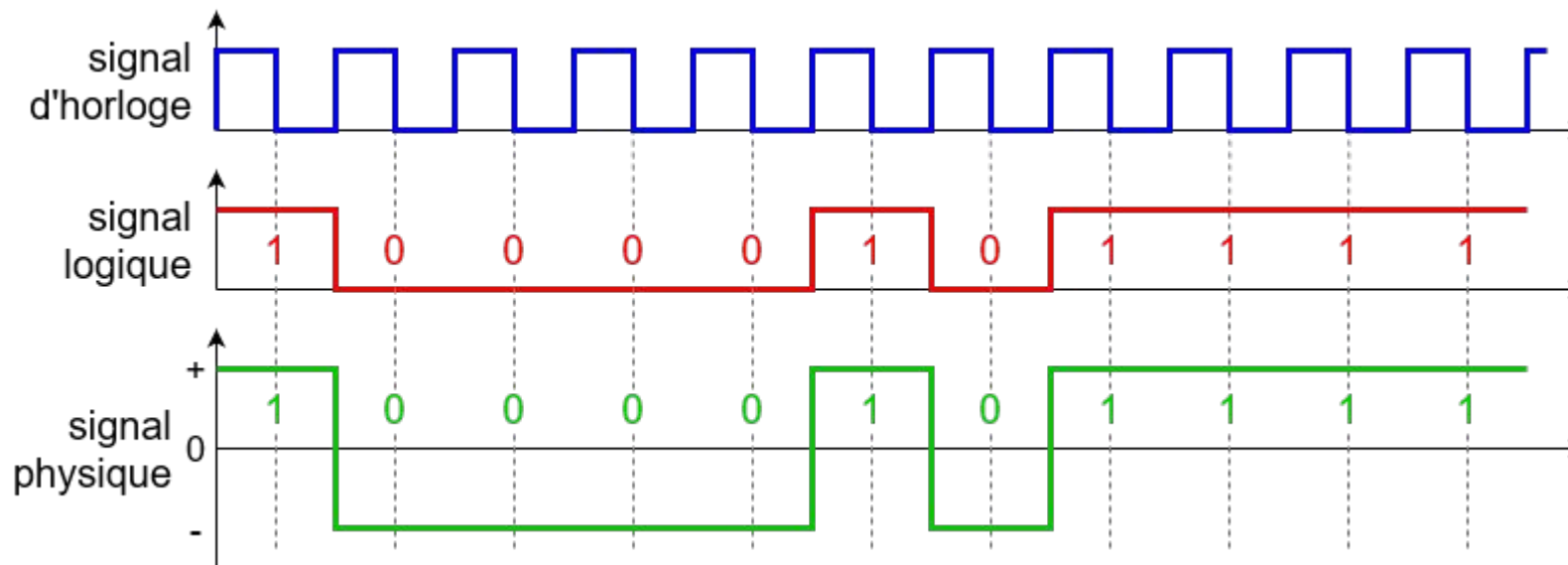
Niveau 1

Comment détecter un envoi de ce type 1000101111 par exemple, quel problématique on peut imaginer ?
 Si on envoie encore + de 0000000 ? Comment distinguer 0000 de 00000 ?



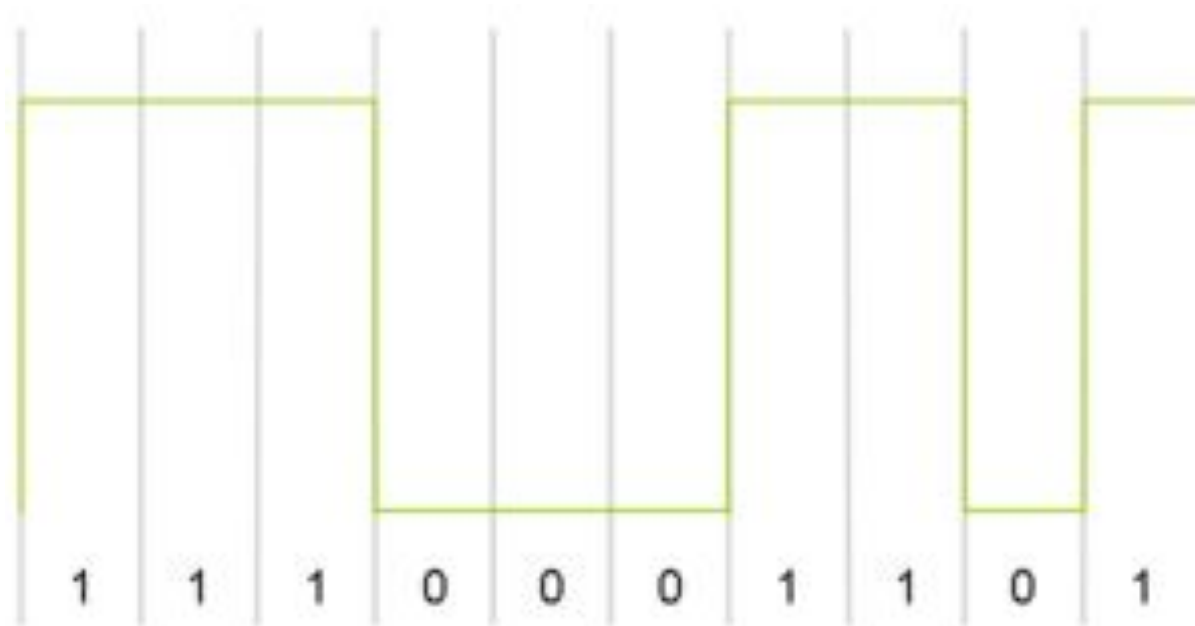
Niveau 1

Idée : Séquencer de manière temporel ! Notion d'Horloge. C'est une bonne idée mais elle doit être synchronisé sur les sites émetteurs et récepteur !



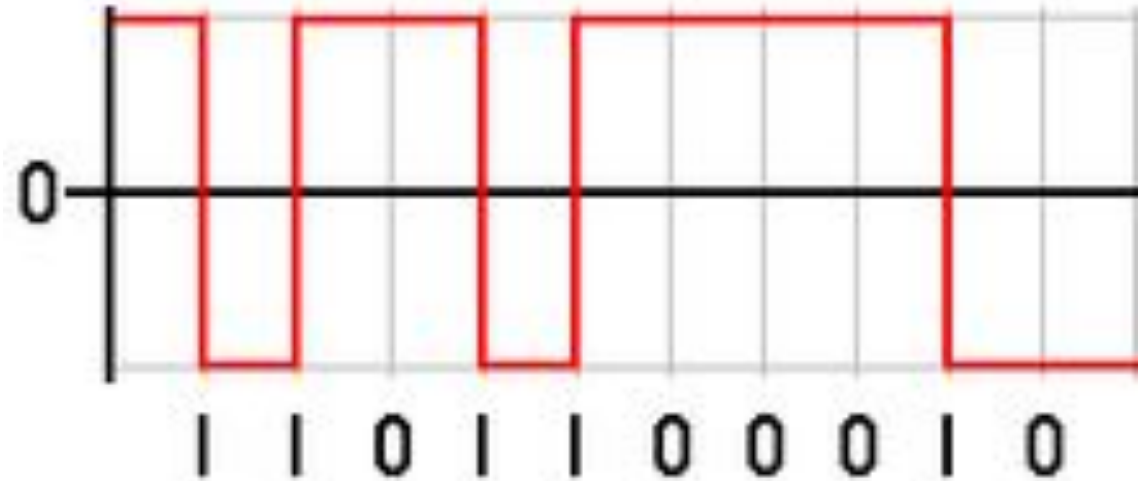
Niveau 1

Premier algorithme : NRZ (Non Return to Zero) : idée basique



Niveau 1

Autre algorithme : NRZI (Non Return to Zero Inverted) : le 1 crée une transition

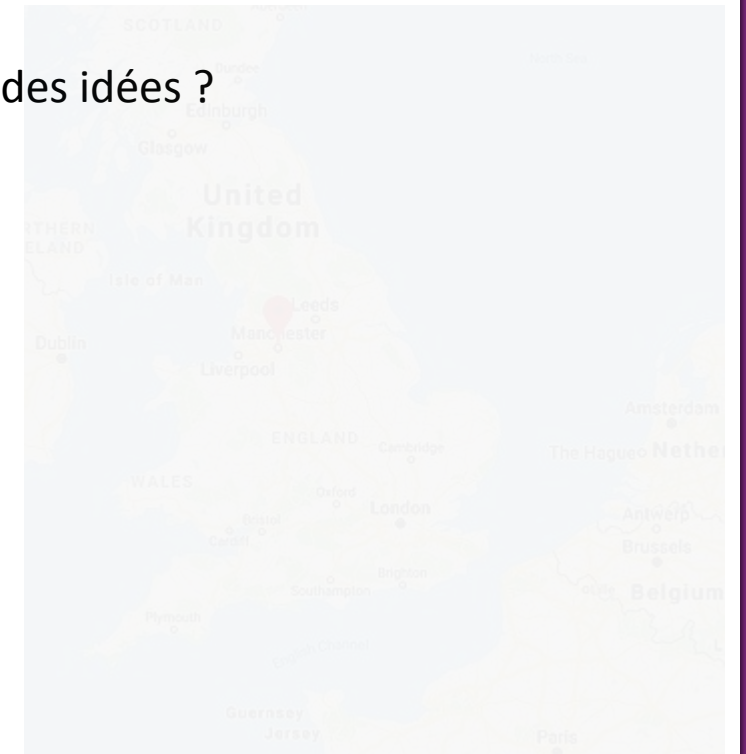


Utilisé en USB : toujours le problème de la suite de 0 -> Idée après 6 zéros consécutifs : rajouter un 1 obligatoire.

-> traçons l'horloge de **1110.0010.0000.00101** (les points sont uniquement là pour lire correctement)

Niveau 1

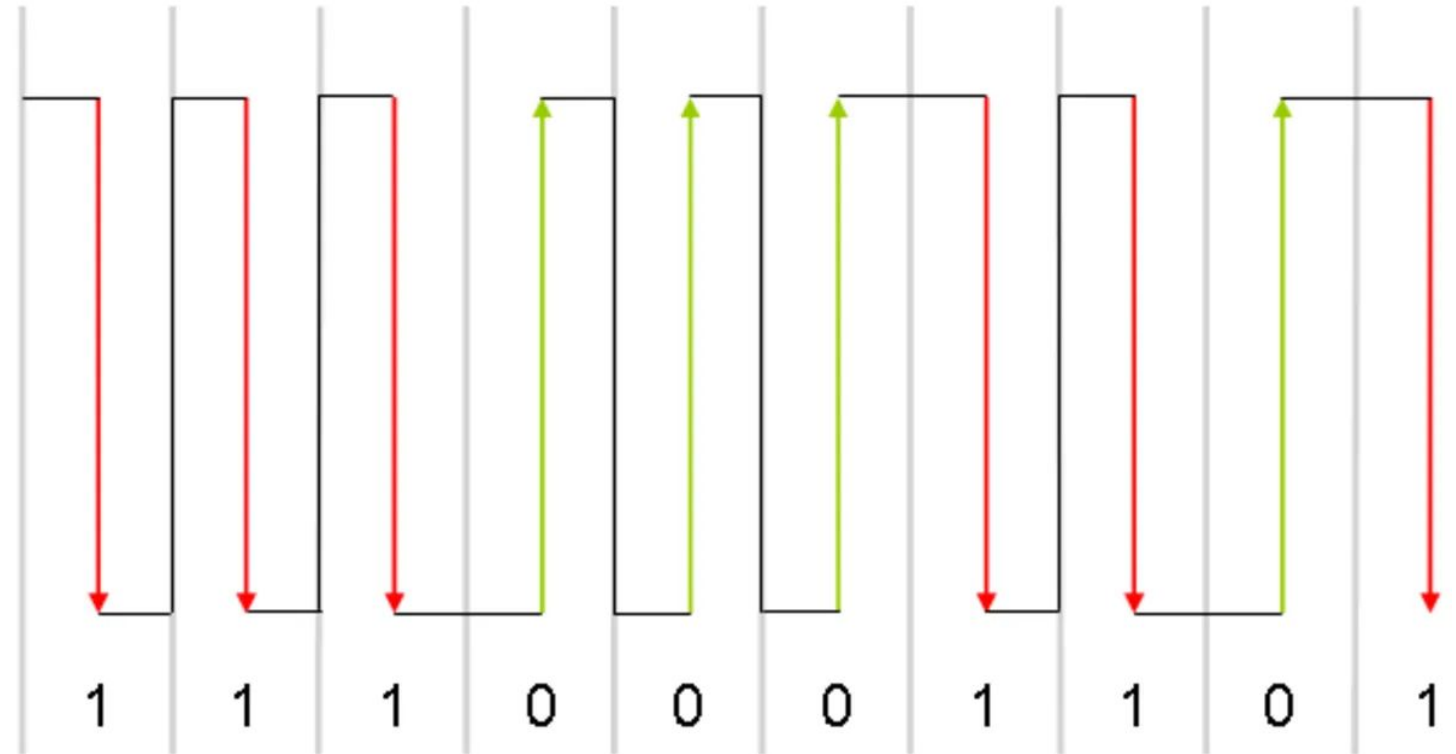
- Idée est de transmettre une “information/modification” pour chaque bit envoyé. Et pas seulement les changements sur le “1”.
- Comment peut on faire ?
 - Exemple **0101.1100.1000** -> pour envoyer ces 12 informations avez vous des idées ?



Niveau 1

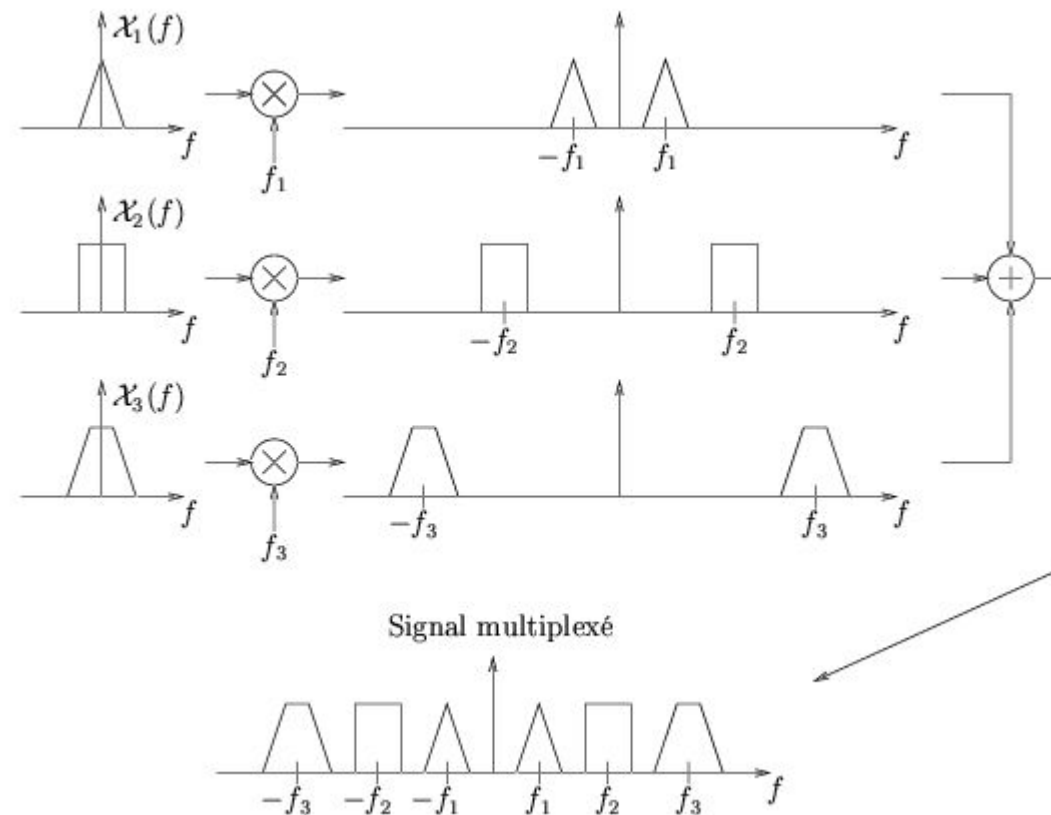
Codage de Manchester.

- Chaque 0 est codé sur une transition vers “le haut”
- Chaque 1 est codé sur une transition vers “le bas”
- Il faut donc le faire à chaque demi-Horloge, c’est le changement qui indique l’information.
- On peut donc indiquer plusieurs 0 ou 1 de suite sans soucis.
- On se rend compte vite si on est plus synchronisé (plusieurs modifications au sein d’une seule horloge indique un soucis).
- Codons le **0101.1100.1000**



Niveau 1

- Sur un axe de transmission on souhaite diffuser plusieurs canaux en même temps :
- Multiplexage de fréquence :



Niveau 1.5 : compression

- C'est bien d'envoyer correctement des 0 et des 1 ...
 - Mais est ce que chaque bit envoyé est vraiment utile ?
 - Ne peut on pas optimiser les 0 et 1 envoyés ?
 - Définition compression avec et sans perte.
-
- Connaissez vous des compressions avec perte et sans perte ?



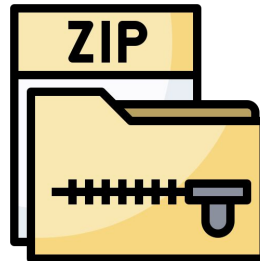
Exemple : ces 2 images doivent elles occuper le même nombre de données envoyées ?

Niveau 1.5 : compression

- Avec perte :



- Sans perte :



Niveau 1.5 : compression

- Réfléchissons : prenons la table Ascii en binaire ... que peut on en dire ?
- Imaginons que je code un livre en français ?
- Puis un en anglais ?

ASCII	BITS	ASCII	BITS
@	1000000	`	1100000
A	1000001	a	1100001
B	1000010	b	1100010
C	1000011	c	1100011
D	1000100	d	1100100
E	1000101	e	1100101
F	1000110	f	1100110
G	1000111	g	1100111
H	1001000	h	1101000
I	1001001	i	1101001
J	1001010	j	1101010
K	1001011	k	1101011
L	1001100	l	1101100
M	1001101	m	1101101
N	1001110	n	1101110
O	1001111	o	1101111
P	1010000	p	1110000
Q	1010001	q	1110001
R	1010010	r	1110010
S	1010011	s	1110011
T	1010100	t	1110100
U	1010101	u	1110101
V	1010110	v	1110110
W	1010111	w	1110111
X	1011000	x	1111000
Y	1011001	y	1111001
Z	1011010	z	1111010
[	1011011	{	1111011
\	1011100		1111100
]	1011101	}	1111101
^	1011110	~	1111110
_	1011111	DEL	1111111

Niveau 1.5 : compression

- En français on veut que le “e” “coûte” moins cher que le Y par exemple.
- On généralise si on veut compresser un texte

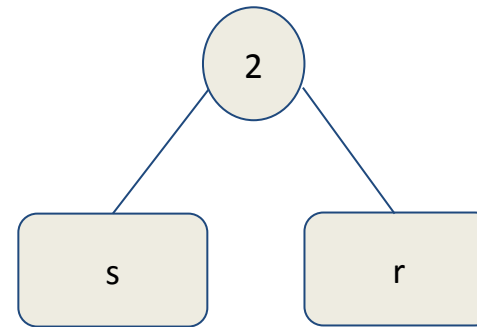
ASCII	BITS	ASCII	BITS
@	1000000	`	1100000
A	1000001	a	1100001
B	1000010	b	1100010
C	1000011	c	1100011
D	1000100	d	1100100
E	1000101	e	1100101
F	1000110	f	1100110
G	1000111	g	1100111
H	1001000	h	1101000
I	1001001	i	1101001
J	1001010	j	1101010
K	1001011	k	1101011
L	1001100	l	1101100
M	1001101	m	1101101
N	1001110	n	1101110
O	1001111	o	1101111
P	1010000	p	1110000
Q	1010001	q	1110001
R	1010010	r	1110010
S	1010011	s	1110011
T	1010100	t	1110100
U	1010101	u	1110101
V	1010110	v	1110110
W	1010111	w	1110111
X	1011000	x	1111000
Y	1011001	y	1111001
Z	1011010	z	1111010
[	1011011	{	1111011
\	1011100		1111100
]	1011101	}	1111101
^	1011110	~	1111110
_	1011111	DEL	1111111

Niveau 1.5 : compression

- Algorithme de Hoffman : idée est de rendre les “caractères” (on pourra le généraliser sur un pixel ou autre) utilisés souvent moins coûteux.
 - Algorithme : on compte les occurrences de chaque éléments : on construit un arbre en partant de ceux les moins courants :
 - On les associe pour créer un nouveau nœud dont le poids équivaut à la somme des poids de ses fils.
 - On recommence jusqu'à n'en avoir plus qu'un seul nœud qui est la racine. Ensuite on place un 0 à chaque branche vers la gauche , et un 1 à chaque branche vers la droite.
-
- Un exemple ?

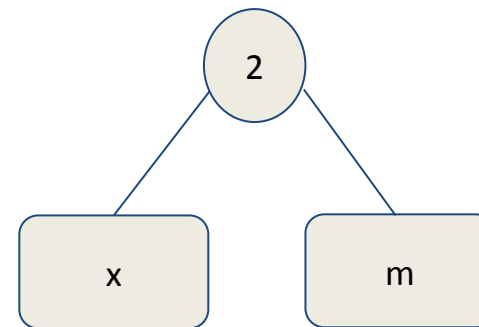
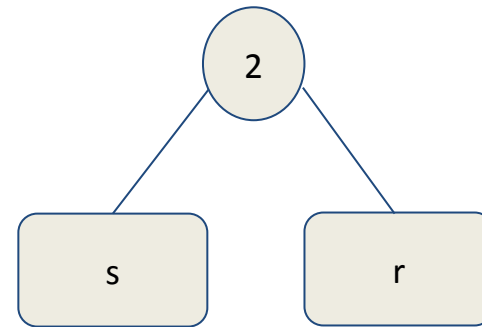
Niveau 1.5 : compression

- “un exemple super” : on compte le nombre d’occurrence :
 - e : 4
 - p : 2
 - “espace” : 2
 - u : 2
 - n : 1
 - x : 1
 - m : 1
 - s : 1
 - r : 1
- On commence par les poids faibles s et r par exemple



Niveau 1.5 : compression

- “un exemple super” : on compte le nombre d’occurrence :
 - e : 4
 - p : 2
 - “espace” : 2
 - “père s et r” : 2
 - u : 2
 - n : 1
 - x : 1
 - m : 1
- On reprend avec les 2 plus faibles x et m

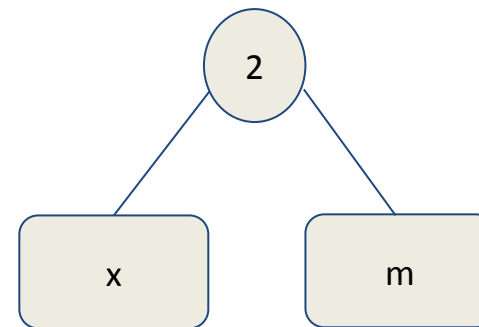
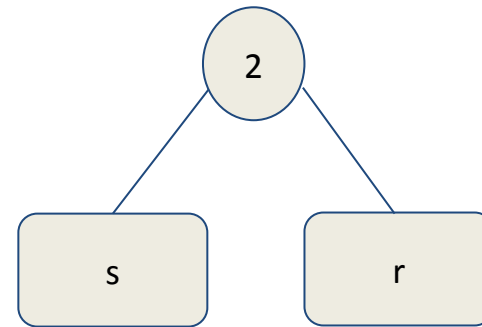


Niveau 1.5 : compression

- “un exemple super” : on compte le nombre d’occurrence :

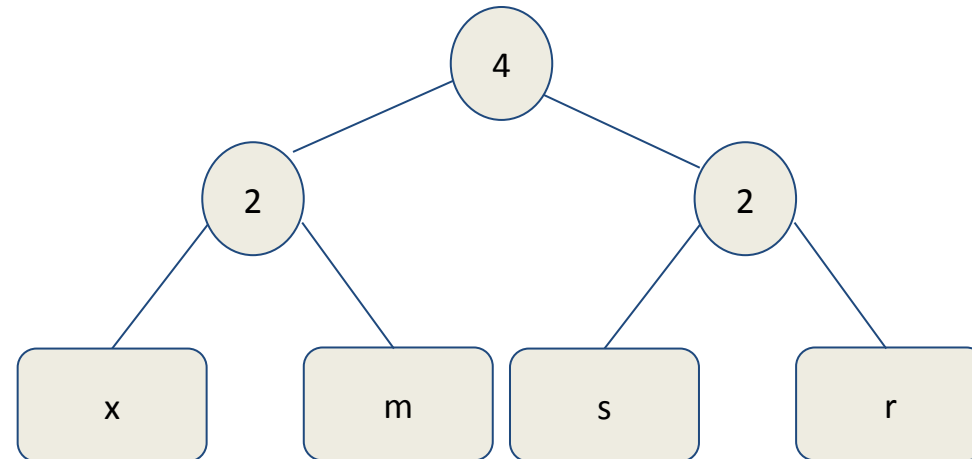
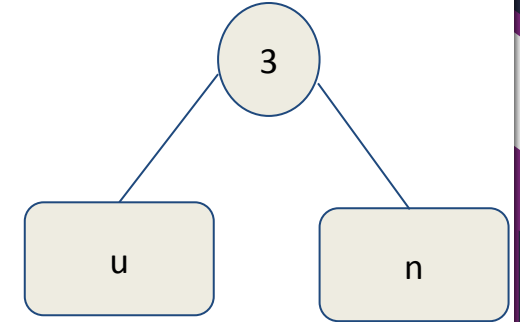
- e : 4
- p : 2
- “espace” : 2
- “père s et r” : 2
- “père x et m” : 2
- u : 2
- n : 1

- On reprend avec les 2 plus faibles x et m



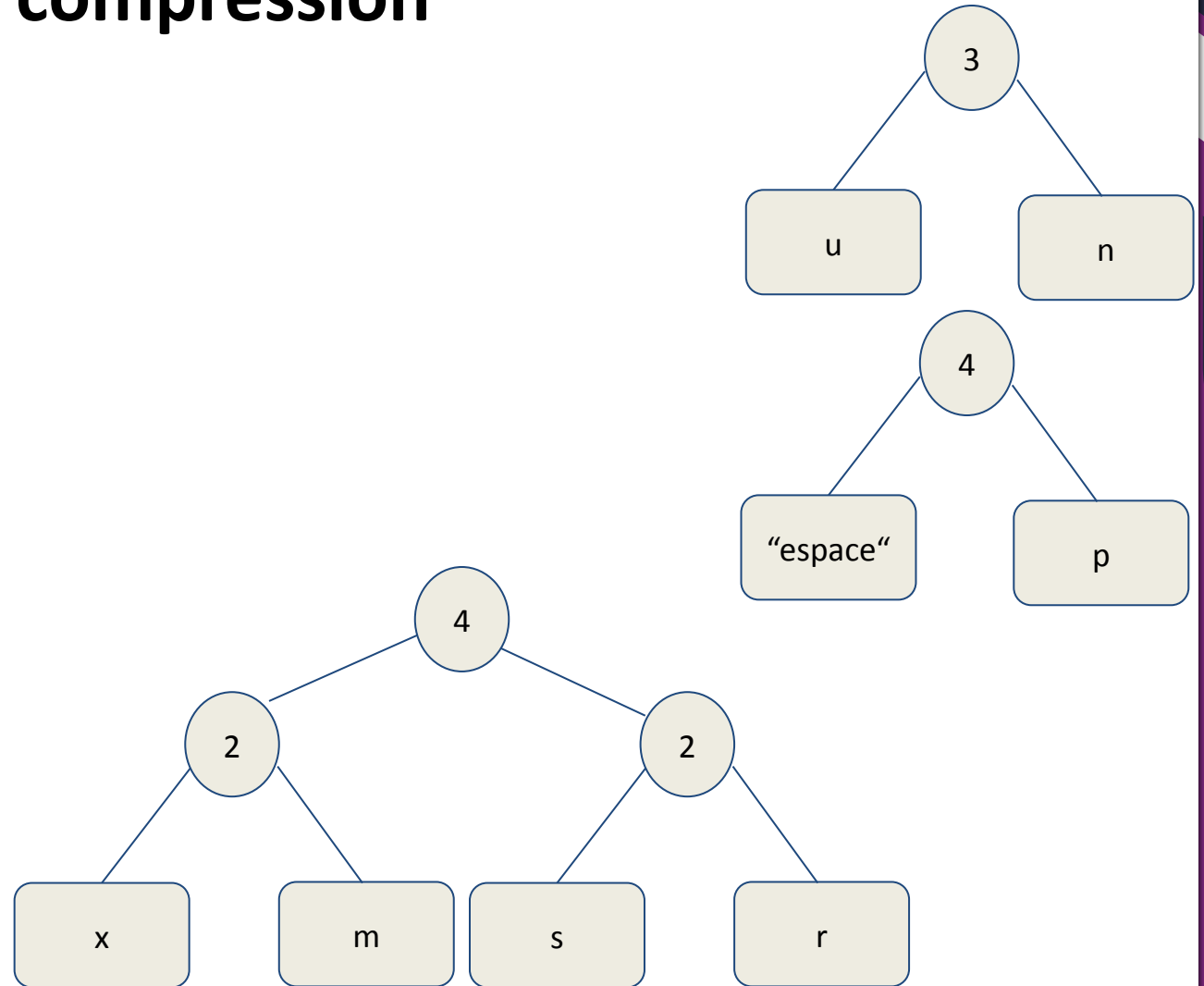
Niveau 1.5 : compression

- “un exemple super” : on compte le nombre d’occurrence :
 - e : 4
 - “Grand père” de “x,m,s,r” : 4
 - “père u et n” : 3
 - p : 2
 - “espace” : 2
- On reprend avec les 2 plus faibles père de x et m et père de s et r



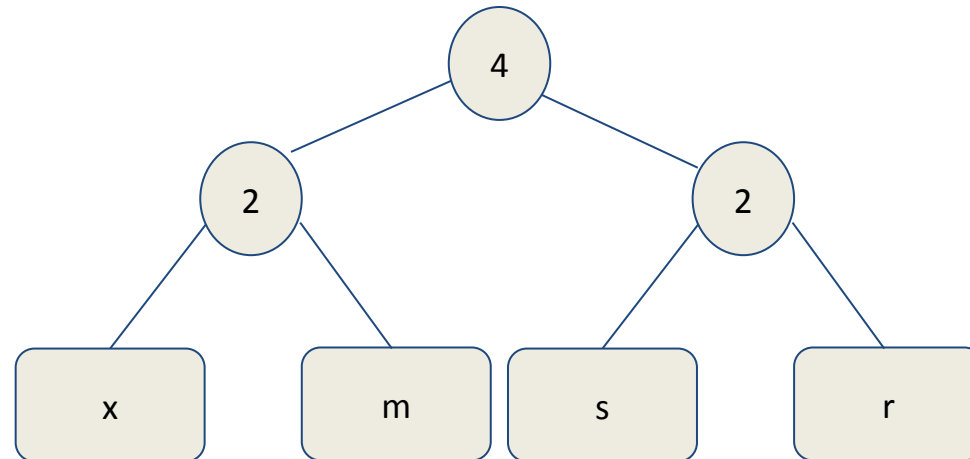
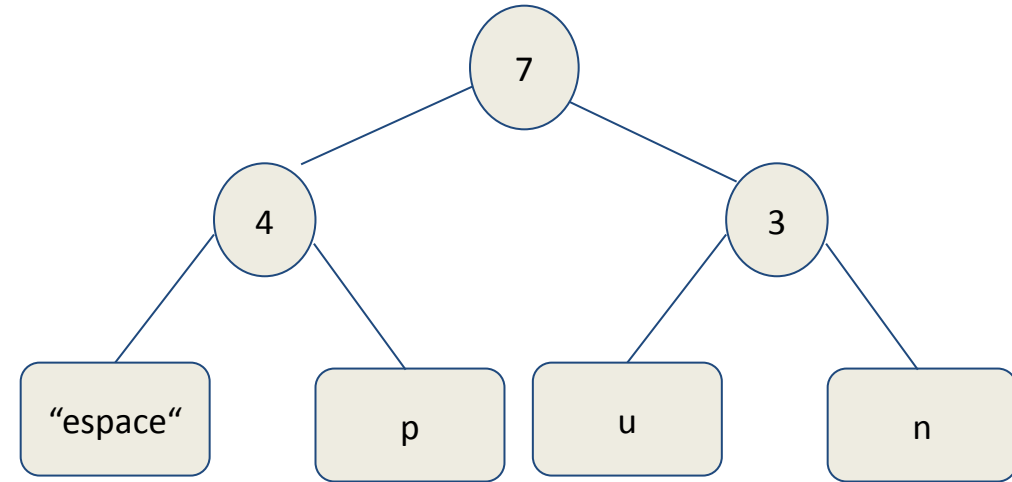
Niveau 1.5 : compression

- “un exemple super” : on compte le nombre d’occurrence :
 - e : 4
 - “Grand père” de “x,m,s,r” : 4
 - “père” “espace” et “p” : 4
 - “père u et n” : 3
- On reprend avec les 2 “espace” et “p”



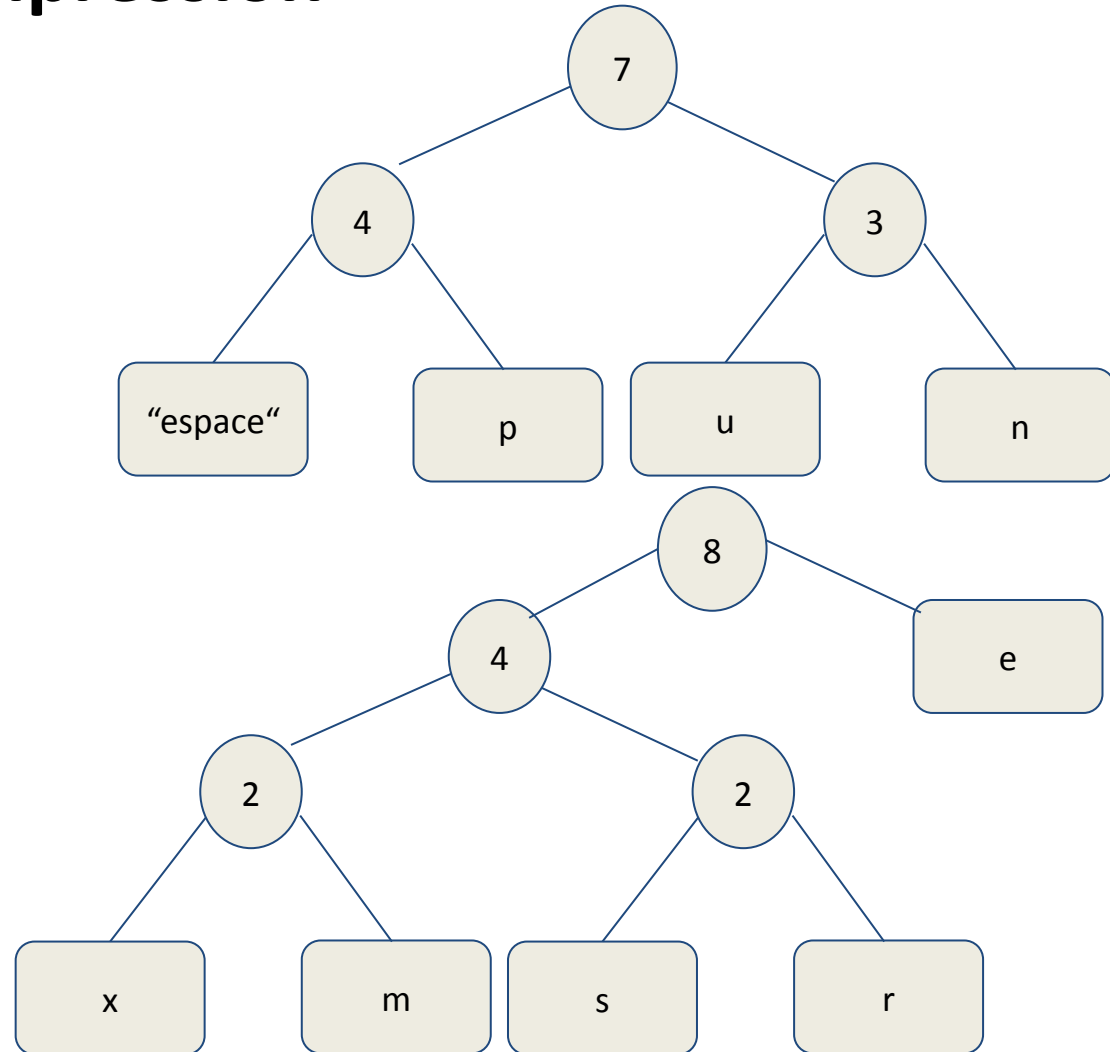
Niveau 1.5 : compression

- “un exemple super” : on compte le nombre d’occurrence :
 - “Grand père de “espace,p,u,n” : 7
 - e : 4
 - “Grand père” de “x,m,s,r” : 4
- On reprend avec les 2 plus faibles père de u et n et père de espace et p



Niveau 1.5 : compression

- “un exemple super” : on compte le nombre d’occurrence :
 - “Grand père de “espace,p,u,n” : 7
 - Le 8 avec tous ses fils
- On reprend avec les 2 plus faibles ceux qui valent 4 et 4 (e)



Niveau 1.5 : compression

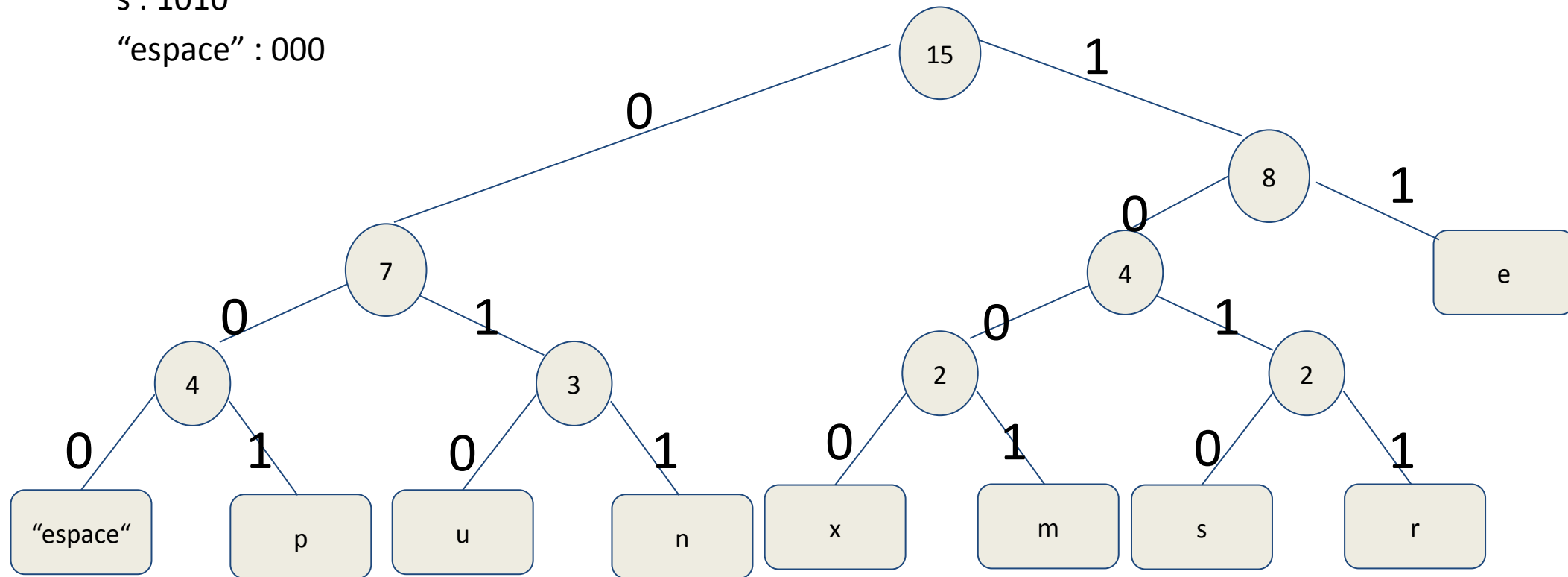
- “un exemple super” : on fini !

e : 11

r : 1011

s : 1010

“espace” : 000



Niveau 1.5 : compression

- Rappel : *Algorithme de Hoffman : idée est de rendre les “caractères” (on pourra le généraliser sur un pixel ou autre) utilisés souvent moins coûteux.*
- *Algorithme : on compte les occurrences de chaque éléments : on construit un arbre en partant de ceux les moins courants :*
 - *on les associe pour créer un nouveau nœud dont le poids équivaut à la somme des poids de ses fils.*
 - *On recommence jusqu'à n'en avoir plus qu'un seul nœud qui est la racine. Ensuite on place un 0 à chaque branche vers la gauche , et un 1 à chaque branche vers la droite.*
- A vous d'essayer : “.a.bb.ccc.dddd.eeeee.fffff.ggggggg.”

Niveau 1.5 : compression

- Rappel : *Algorithme de Hoffman : idée est de rendre les “caractères” (on pourra le généraliser sur un pixel ou autre) utilisés souvent moins coûteux.*
- *Idée pourquoi ne pas le faire DIRECTEMENT sur les 0 et 1 ?*

Tester sur : “**100101010101101111000011101010001110011101010111000110**”

Niveau 1.5 : compression

- La redondance d'information peut être sur plusieurs lettres d'affilés ? -> L'algorithme LZ78

Idée est de pouvoir détecter de la redondance et de compresser les données sans perte. On va créer un dictionnaire au fur et à mesure

L'algorithme parcourt le texte en gardant trace des préfixes rencontrés à chaque étape. Lorsqu'il trouve un préfixe qui n'est pas déjà dans le dictionnaire, il crée un nouveau préfixe, il lui donne le premier code non encore utilisé et le rentre dans le dictionnaire.

Lorsque l'algorithme rentre un nouveau préfixe dans le dictionnaire, il émet sur sa sortie la paire (**préfixe, symbole**), où **symbole** est le dernier symbole rencontré et **préfixe** est le dernier préfixe rencontré avant le symbole.

Niveau 1.5 : compression

- L'algorithme LZ78

Un exemple :

Entrée : ACATAPLASMATIC
Sortie :

Dictionnaire :
 ϵ : 0

Entrée : ACATAPLASMATIC
Sortie : ($\emptyset$, A)

Dictionnaire :
 ϵ : 0
 A : 1

Entrée : ACATAPLASMATIC
Sortie : ($\emptyset$, A)($\emptyset$, C)

Dictionnaire :
 ϵ : 0
 A : 1
 C : 2

Niveau 1.5 : compression

- L'algorithme LZ78

Un exemple :

Entrée : ACATAPLASMATIC
 Sortie : $(\emptyset, A)(\emptyset, C)(1, T)(1, P)$

Dictionnaire :

ϵ	:	$\emptyset$
A	:	1
C	:	2
AT	:	3

Entrée : ACATAPLASMATIC
 Sortie : $(\emptyset, A)(\emptyset, C)(1, T)(1, P)$

Dictionnaire :

ϵ	:	$\emptyset$
A	:	1
C	:	2
AT	:	3
AP	:	4

Entrée : ACATAPLASMATIC
 Sortie : $(\emptyset, A)(\emptyset, C)(1, T)(1, P)(\emptyset, L)$

Dictionnaire :

ϵ	:	$\emptyset$
A	:	1
C	:	2
AT	:	3
AP	:	4
L	:	5

On a un A (position 1) qu'on a déjà dans le dictionnaire !

Niveau 1.5 : compression

- L'algorithme LZ78

Entrée : ACATAPLASMATIC

Sortie : ($\emptyset$,A)($\emptyset$,C)(1,T)(1,P)($\emptyset$,L)(1,S)($\emptyset$,M)

Dictionnaire :

ϵ	:	0
A	:	1
C	:	2
AT	:	3
AP	:	4
L	:	5
AS	:	6
M	:	7

Entrée : ACATAPLASMATIC

Sortie : ($\emptyset$,A)($\emptyset$,C)(1,T)(1,P)($\emptyset$,L)(1,S)($\emptyset$,M)(3,I)

Dictionnaire :

ϵ	:	0
A	:	1
C	:	2
AT	:	3
AP	:	4
L	:	5
AS	:	6
M	:	7
ATI	:	8

On a AT dans le dictionnaire à la position 3, enfin on compresse !

Niveau 1.5 : compression

- L'algorithme LZ78

Entrée : ACATAPLASMATIC

Sortie : ($\emptyset$,A)($\emptyset$,C)(1,T)(1,P)($\emptyset$,L)(1,S)($\emptyset$,M)(3,I)(2, $\emptyset$)

Dictionnaire :

ϵ	:	$\emptyset$
A	:	1
C	:	2
AT	:	3
AP	:	4
L	:	5
AS	:	6
M	:	7
ATI	:	8

Niveau 1.5 : compression

- L'algorithme LZ78 et maintenant la décompression :

Entrée : $(\emptyset, A)(\emptyset, C)(1, T)(1, P)(\emptyset, L)(1, S)(\emptyset, M)(3, I)(2, \emptyset)$

Sortie : ACAT

Dictionnaire :

$\emptyset$: ϵ
 1 : A
 2 : C
 3 : AT

Entrée : $(\emptyset, A)(\emptyset, C)(1, T)(1, P)(\emptyset, L)(1, S)(\emptyset, M)(3, I)(2, \emptyset)$

Sortie : ACATAPLAS

Dictionnaire :

$\emptyset$: ϵ
 1 : A
 2 : C
 3 : AT
 4 : AP
 5 : L
 6 : AS

Niveau 1.5 : compression

- L'algorithme LZ78 et maintenant la décompression :

Entrée : $(\emptyset, A)(\emptyset, C)(1, T)(1, P)(\emptyset, L)(1, S)(\emptyset, M)(3, I)(2, \emptyset)$

Sortie : ACATAPLASMATI

Dictionnaire :

$\emptyset$: ϵ
 1 : A
 2 : C
 3 : AT
 4 : AP
 5 : L
 6 : AS
 7 : M
 8 : ATI

— —

Entrée : $(\emptyset, A)(\emptyset, C)(1, T)(1, P)(\emptyset, L)(1, S)(\emptyset, M)(3, I)(2, \emptyset)$

Sortie : ACATAPLASMATIC

Dictionnaire :

$\emptyset$: ϵ
 1 : A
 2 : C
 3 : AT
 4 : AP
 5 : L
 6 : AS
 7 : M
 8 : ATI

Niveau 1.5 : compression

- L'algorithme LZ78, à vous de faire : “**ABCAUBCDABCABCUBCDABCU.**” Compression et décompression
- Sur un livre de littérature classique, la compression est d'environ 40% avec LZ78
- Compression Zip actuel est un mélange de cet algorithme couplé avec Huffman.

Niveau 1.5 : Vérification

Sur une trame réseau, on peut avoir des interférences qui change un bit de 0 en 1, mais comment le récepteur peut il le savoir ?

Notion de **vérification de l'intégrité**.

Exemple simple : Bit de parité.

On ajoute à 7 bits un huitième (donc un octet) qui indique la “parité” des 7 autres : si nombre de 1 est pair : 0 sinon 1.



Le récepteur peut donc vérifier que les 7 autres ne sont pas altérés

Niveau 1.5 : Vérification

A vous de définir les bits de parité pour :

?000.0111.

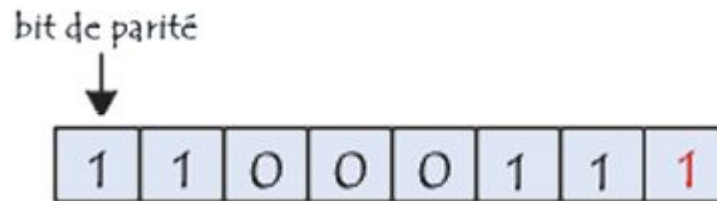
?101.1111.

?110.0010.

Niveau 1.5 : Vérification

En cas d'erreur par exemple on va demander à l'émetteur de relancer la trame en erreur :

Exemple de trame erronée :



Quelle limitation vous voyez à ce système ?

Niveau 1.5 : Vérification

Si il y a 2 erreurs, ce n'est pas détecté !



Plus généralement : si il y a un nombre pair d'erreur -> non détecté !

Niveau 1.5 : Codage Hamming 7,4

On peut faire un code qui permet non seulement de détecter une erreur mais aussi de la corriger !

Par exemple si on envoie 4 bits, on en rajoute 3 qui permettent de détecter une erreur ET de la corriger (donc de savoir où est cette erreur).

On prend l'exemple du plus connu : le codage Hamming 7,4. On envoie 7 bits pour une trame de 4.

Le premier bit supplémentaire compte la parité des bits en positions 1,3,5,... il est stocké en position 1

Le second bit supplémentaire compte la parité des bits en positions 2,3,6,7,10 (...) il est stocké en position 2

Le troisième bit supplémentaire compte la parité des bits en positions 4,5,6,7,12,... il est stocké en position 4

Niveau 1.5 : Codage Hamming 7,4

On va avoir 3 points de vérification :

- p1 vérifie la parité de d1,d2,d4
- p2 vérifie la parité de d1,d3,d4
- p3 vérifie la parité de d2,d3,d4

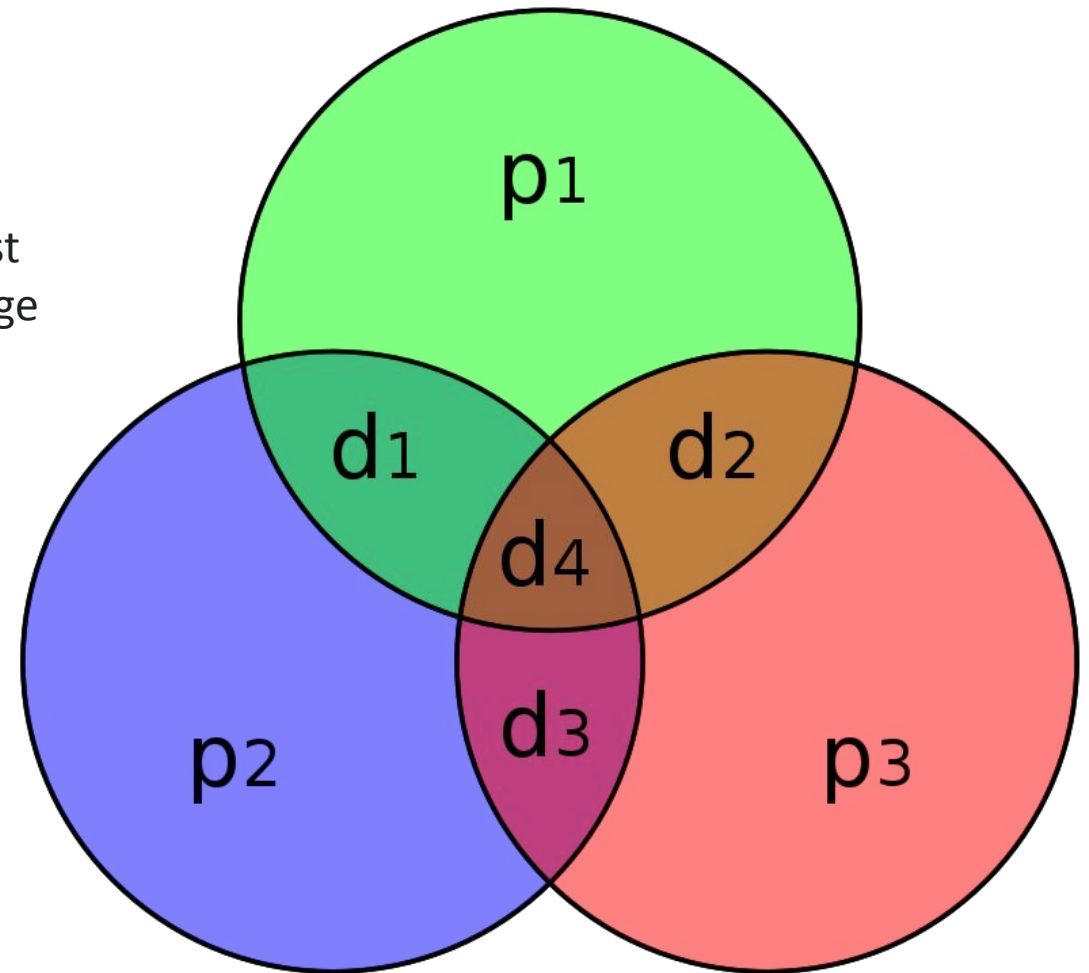
Niveau 1.5 : Codage Hamming 7,4

Les données sont les d_1, d_2, d_3, d_4

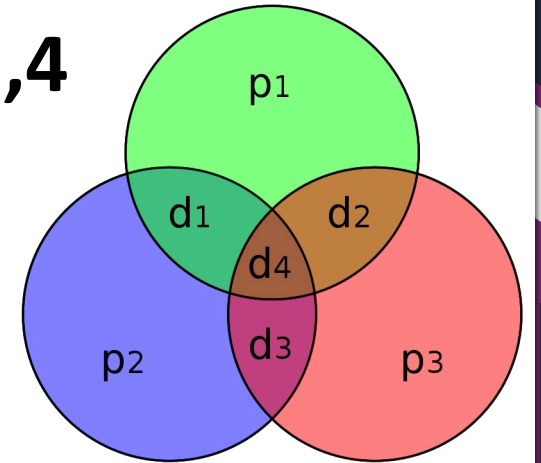
Les poids p_1, p_2, p_3 sont les données de vérification

Si la parité du cercle est conservée alors la cellule associée est verte avec le texte *oui*, dans le cas contraire la cellule est rouge avec le texte *non*.

Bit #	1	2	3	4	5	6	7
bit erroné	p_1	p_2	d_1	p_3	d_2	d_3	d_4
Cercle rouge	Oui	Oui	Oui	Non	Non	Non	Non
Cercle bleu	Oui	Non	Non	Oui	Oui	Non	Non
Cercle vert	Non	Oui	Non	Oui	Non	Oui	Non



Niveau 1.5 : Codage Hamming 7,4



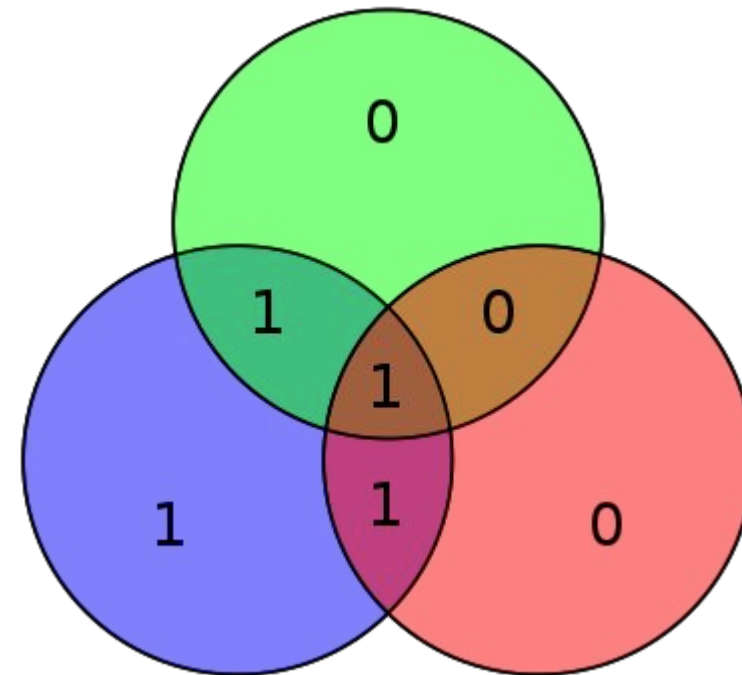
Les données sont les d1,d2,d3,d4

Les poids p1, p2, p3 sont les données de vérification

Exemple : 1011 est envoyé .

0110011 la trame envoyée contient les données en bleu, en position trois, cinq, six et sept.

En rouge les données de contrôle

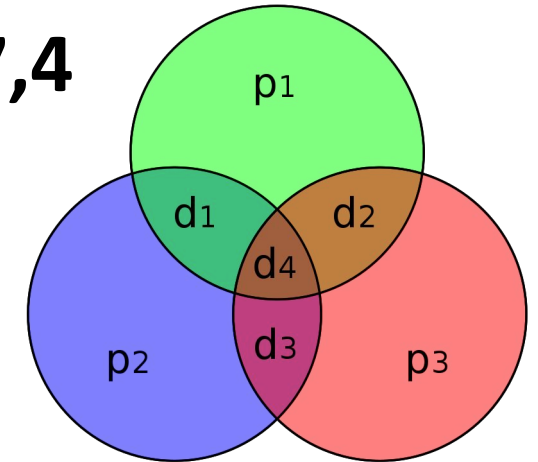
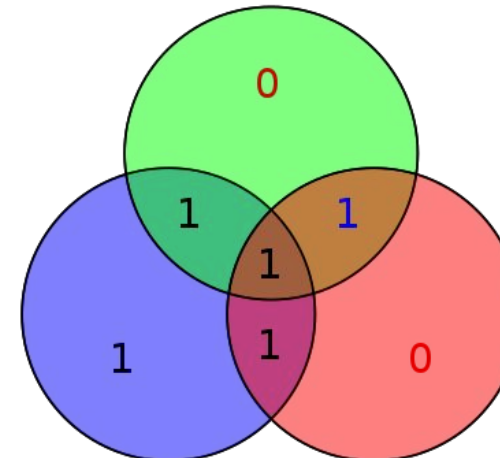
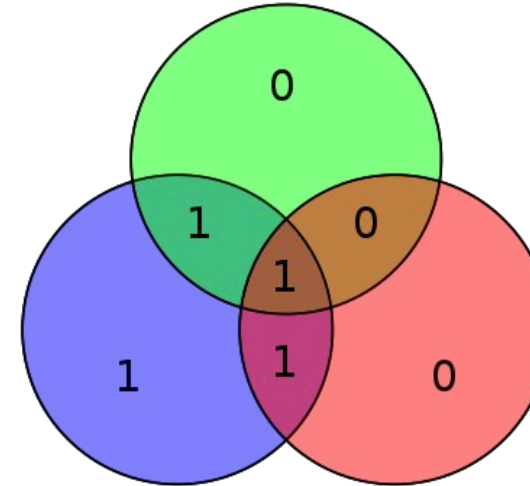


Niveau 1.5 : Codage Hamming 7,4

011.0011 contient les données en bleu, en position trois, cinq, six et sept.

Si on reçoit : **011.0111** on voit que cercle bleu et vert sont en désaccord et c'est donc ce bit numéro 3 à modifier

On voit que les cercles rouge ET vert ne sont pas bon donc c'est le deuxième bit qui n'est pas bon.



Niveau 1.5 : Codage Hamming 7,4

La connaissance de l'image de chaque vecteur de la **base canonique** détermine entièrement l'application d'encodage φ . Les quatre vecteurs de la base correspondent aux messages suivants : $d_1 = 1000$, $d_2 = 0100$, $d_3 = 0010$ et $d_4 = 0001$.

L'image de d_1 , le message qui a des coordonnées nulles en d_2 , d_3 et d_4 , possède deux parités égales à un, celle de p_1 et p_2 et une égale à zéro : p_3 .

En respectant l'ordre de la base de F : $p_1, p_2, d_1, p_3, d_2, d_3$ et d_4 , on obtient l'image du premier vecteur :

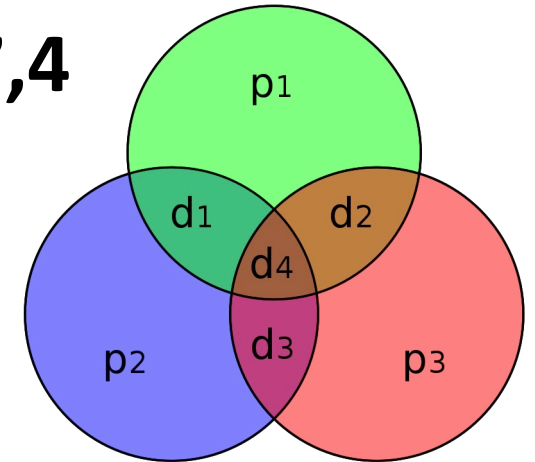
$$\varphi(1000) = 1110000$$

Calculons de la même manière les images des autres vecteurs de la base de E :

$$\varphi(0100) = 1001100, \quad \varphi(0010) = 0101010 \text{ et } \varphi(0001) = 1101001$$

La matrice de φ , appelée **matrice génératrice** est formée des quatre colonnes correspondant aux images des vecteurs de la base canonique par φ , on obtient :

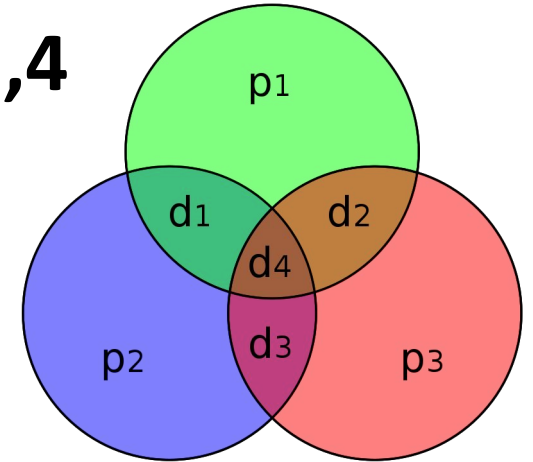
$$\mathbf{G} = \begin{pmatrix} 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$



Niveau 1.5 : Codage Hamming 7,4

Matrice générative : (on admettra dans ce cours)

$$\mathbf{G} = \begin{pmatrix} 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$



La matrice du 7,3 (ordre des nombres de 1 à 7) : $\mathbf{H} = \begin{pmatrix} 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 \end{pmatrix}$

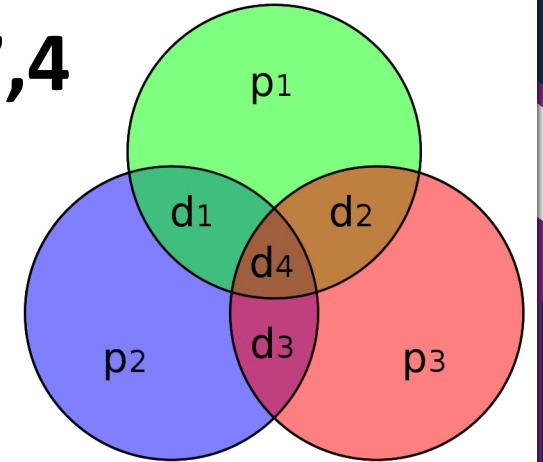
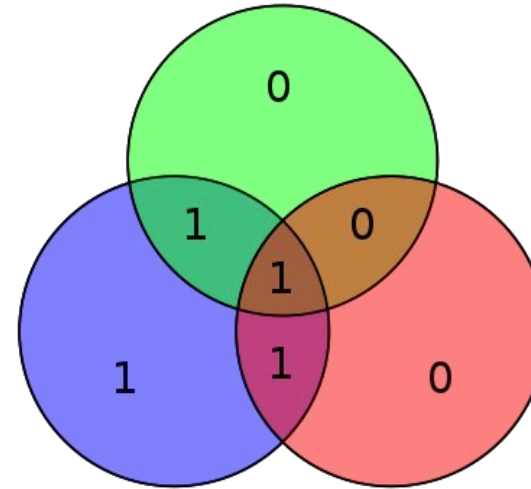
Si on veut coder le message D : **1011**, on enverra **011.0011**

$$\mathbf{D} = \begin{pmatrix} 1 \\ 0 \\ 1 \\ 1 \end{pmatrix} \quad \mathbf{G} \cdot \mathbf{D} = \begin{pmatrix} 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \\ 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \\ 1 \\ 0 \\ 0 \\ 1 \\ 1 \end{pmatrix}$$

Niveau 1.5 : Codage Hamming 7,4

011.0011 contient les données en bleu, en position trois, cinq, six et sept.

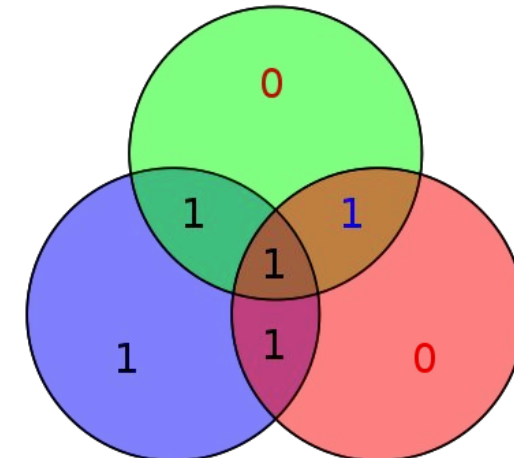
$$\mathbf{H} \cdot \mathbf{C} = \begin{pmatrix} 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} 0 \\ 1 \\ 1 \\ 0 \\ 0 \\ 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}$$



H*envoi doit faire 0

Si on reçoit : **011.0111** on voit que cercle rouge et vert sont en désaccord et c'est donc ce à modifier. (position 5 avec la matrice)

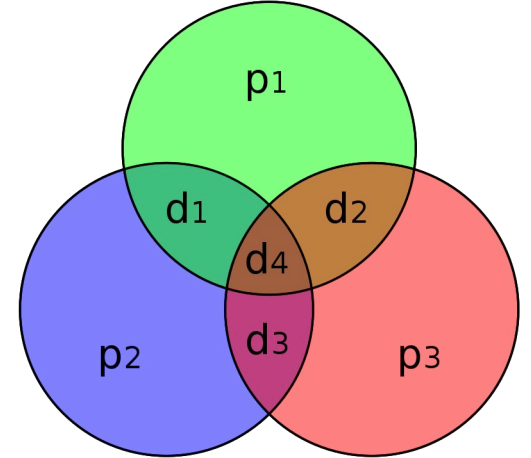
$$\mathbf{H} \cdot \mathbf{X} = \begin{pmatrix} 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} 0 \\ 1 \\ 1 \\ 0 \\ 1 \\ 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix}$$



Niveau 1.5 : Codage Hamming 7,4

$$\mathbf{H} = \begin{pmatrix} 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 \end{pmatrix}$$

$$\mathbf{G} = \begin{pmatrix} 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$



Faites la trame réseau pour envoyer : **0101**.

Ajouter les 3 bits de contrôle.

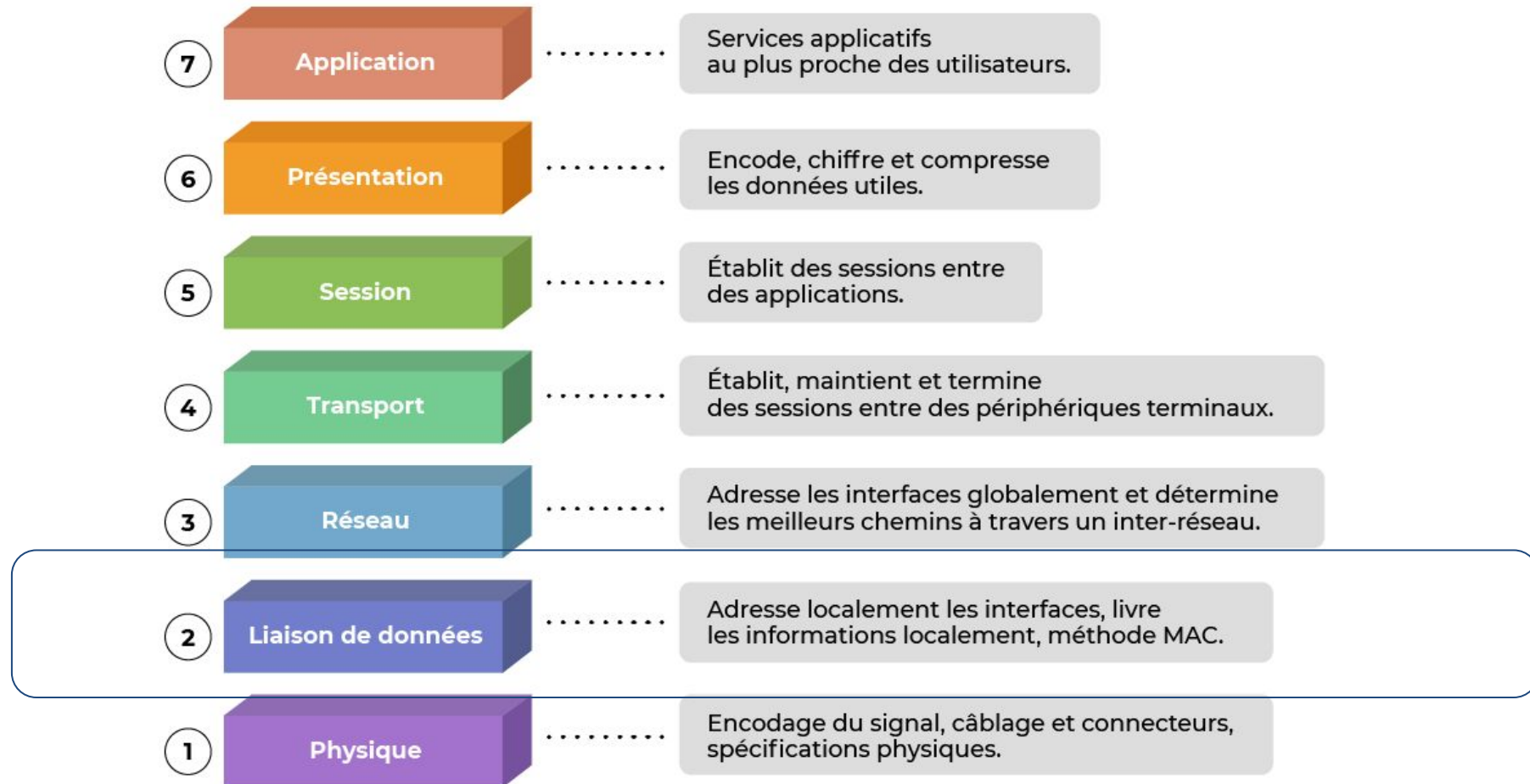
Exemple sur 1011 0110011 contient les données en bleu, en position trois, cinq, six et sept.

Vérifier : la matrice fonctionne.

Vérifier : si on modifie un bit qu'on retrouve bien la position de celui à modifier.

Chapitre 2 : Couche 2

Modèle OSI



Chapitre 2 : Couche 2

Adresse Mac : il s'agit de l'adresse physique de l'appareil ($\neq$ IP !). 6 octets donc théoriquement 2^{48} adresses possibles. Les 3 premiers sont le constructeurs et les 3 suivantes un identifiant.



Chapitre 2 : Couche 2

L'adresse MAC est comme une plaque d'immatriculation pour une voiture, pour l'obtenir (`ip link` sous linux)

```
C:\Users\Jean-Baptiste>ipconfig /all

Configuration IP de Windows

Nom de l'hôte . . . . . : DESKTOP-BNSN6SO
Suffixe DNS principal . . . . . :
Type de noeud . . . . . : Hybride
Routage IP activé . . . . . : Non
Proxy WINS activé . . . . . : Non

Carte Ethernet Ethernet :

Statut du média. . . . . : Média déconnecté
Suffixe DNS propre à la connexion. . . :
Description. . . . . : Realtek PCIe GbE Family Controller
Adresse physique . . . . . : 7C-57-58-14-9E-42
DHCP activé. . . . . : Oui
Configuration automatique activée. . . : Oui

Carte réseau sans fil Connexion au réseau local* 9 :

Statut du média. . . . . : Média déconnecté
Suffixe DNS propre à la connexion. . . :
Description. . . . . : Microsoft Wi-Fi Direct Virtual Adapter
Adresse physique . . . . . : F4-CE-23-F6-EA-A6
DHCP activé. . . . . : Oui
Configuration automatique activée. . . : Oui
```


Chapitre 2 : Couche 2

Sur internet, on peut facilement trouver le résultat du constructeur exemple pour mon poste de travail :

Résultat	
Préfixe d'adresse MAC	Fabricant
F4CE23	Intel Corporate

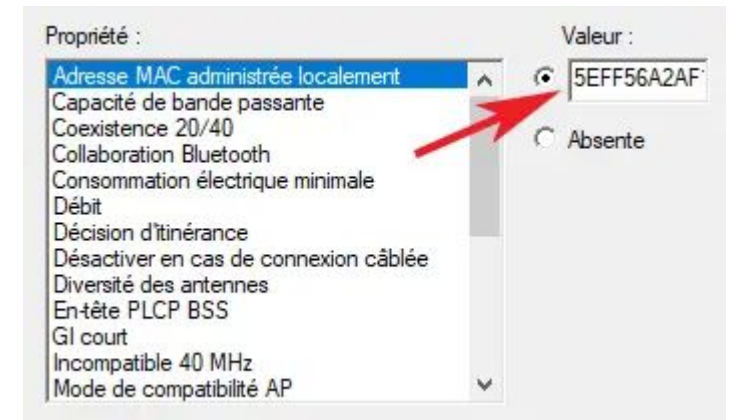
Chapitre 2 : Couche 2

On peut changer de manière logicielle notre adresse MAC.

Cela est pratique pour apparaître comme “nouveau” sur un réseau.

Vous êtes connecté

Vous pouvez vous connecter
pendant 30 minutes,
après avoir accepté la politique
d'utilisation du réseau wifi du GHICL



Chapitre 2 : Couche 2

Attention si vous faites ce changement et que vous mettez “n’importe quoi”, vous pouvez être refusé de connection sur un réseau.

Il faut dans ce cas réinitialiser l’adresse MAC avec celle intrinsèque de votre carte réseau.

“Fun” fact : Edward Snowden explique que la NSA aurait créé un système de traçage des adresses MAC afin de lier celle ci à des identités. Rien n’est prouvé, la NSA ayant toujours démenti cela. Conséquence :

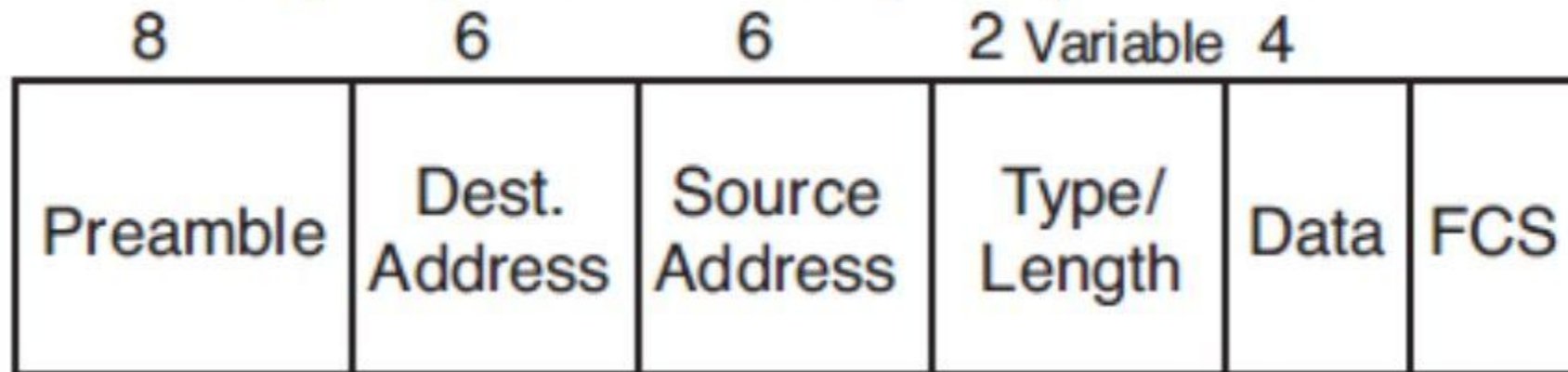


Note: When a device running Android 6.0 (API level 23) initiates a background Wi-Fi or Bluetooth scan, the operation is visible to external devices as originating from a randomized **MAC** address.

Chapitre 2 : Couche 2

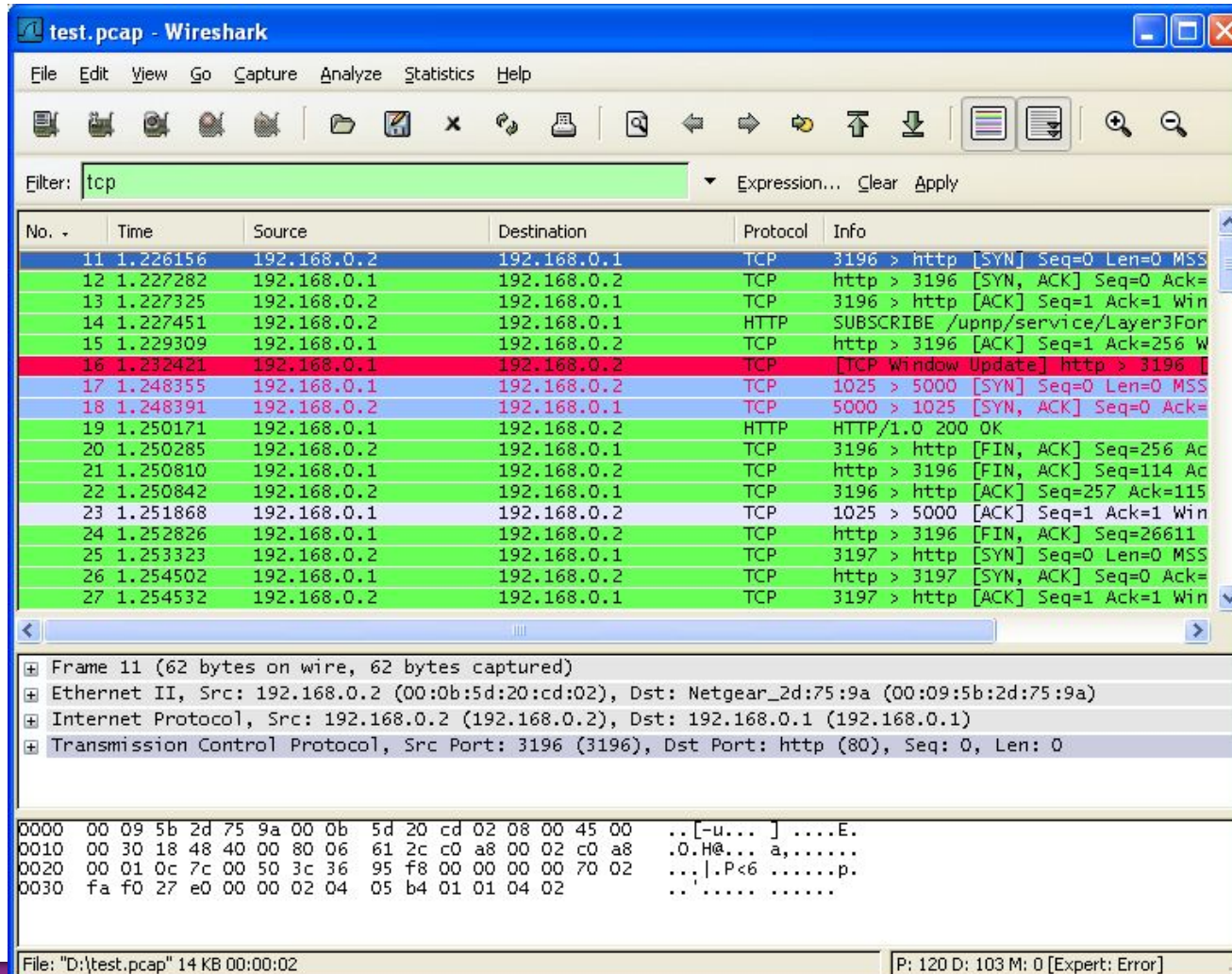
Une trame Ethernet contient au début 2 adresses MAC : celle du destinataire, celle ensuite de la source émettrice . PAD : remplissage ajouté artificiellement pour que la trame atteigne la taille minimale autorisée. (ajouter à DATA selon). Fini par FCS (Frame Check Sequence) qui est la Détection d'erreurs (CRC)

Ethernet (DIX) and Revised (1997) IEEE 802.3



Chapitre 2 : Couche 2

Utilisation Wireshark : Logiciel permettant d'analyser les trames réseaux



test.pcap - Wireshark

Filter: tcp

No.	Time	Source	Destination	Protocol	Info
11	1.226156	192.168.0.2	192.168.0.1	TCP	3196 > http [SYN] Seq=0 Len=0 MSS
12	1.227282	192.168.0.1	192.168.0.2	TCP	http > 3196 [SYN, ACK] Seq=0 Ack=
13	1.227325	192.168.0.2	192.168.0.1	TCP	3196 > http [ACK] Seq=1 Ack=1 Win
14	1.227451	192.168.0.2	192.168.0.1	HTTP	SUBSCRIBE /upnp/service/Layer3For
15	1.229309	192.168.0.1	192.168.0.2	TCP	http > 3196 [ACK] Seq=1 Ack=256 W
16	1.232421	192.168.0.1	192.168.0.2	TCP	[TCP Window Update] http > 3196
17	1.248355	192.168.0.1	192.168.0.2	TCP	1025 > 5000 [SYN] Seq=0 Len=0 MSS
18	1.248391	192.168.0.2	192.168.0.1	TCP	5000 > 1025 [SYN, ACK] Seq=0 Ack=
19	1.250171	192.168.0.1	192.168.0.2	HTTP	HTTP/1.0 200 OK
20	1.250285	192.168.0.2	192.168.0.1	TCP	3196 > http [FIN, ACK] Seq=256 Ac
21	1.250810	192.168.0.1	192.168.0.2	TCP	http > 3196 [FIN, ACK] Seq=114 Ac
22	1.250842	192.168.0.2	192.168.0.1	TCP	3196 > http [ACK] Seq=257 Ack=115
23	1.251868	192.168.0.1	192.168.0.2	TCP	1025 > 5000 [ACK] Seq=1 Ack=1 Win
24	1.252826	192.168.0.1	192.168.0.2	TCP	http > 3196 [FIN, ACK] Seq=26611
25	1.253323	192.168.0.2	192.168.0.1	TCP	3197 > http [SYN] Seq=0 Len=0 MSS
26	1.254502	192.168.0.1	192.168.0.2	TCP	http > 3197 [SYN, ACK] Seq=0 Ack=
27	1.254532	192.168.0.2	192.168.0.1	TCP	3197 > http [ACK] Seq=1 Ack=1 Win

Frame 11 (62 bytes on wire, 62 bytes captured)

- Ethernet II, Src: 192.168.0.2 (00:0b:5d:20:cd:02), Dst: Netgear_2d:75:9a (00:09:5b:2d:75:9a)
- Internet Protocol, Src: 192.168.0.2 (192.168.0.2), Dst: 192.168.0.1 (192.168.0.1)
- Transmission Control Protocol, Src Port: 3196 (3196), Dst Port: http (80), Seq: 0, Len: 0

0000 00 09 5b 2d 75 9a 00 0b 5d 20 cd 02 08 00 45 00 ..[-u...]E.
 0010 00 30 18 48 40 00 80 06 61 2c c0 a8 00 02 c0 a8 .O.He... a,.....
 0020 00 01 0c 7c 00 50 3c 36 95 f8 00 00 00 00 70 02 ...|.P<6p.
 0030 fa f0 27 e0 00 00 02 04 05 b4 01 01 04 02 ..'.....

File: "D:\test.pcap" 14 KB 00:00:02 | P: 120 D: 103 M: 0 [Expert: Error]



Chapitre 2 : Couche 2

0000	f4 ce 23 f6 ea a5 5c a6	2d 3d 69 ff 08 00 45 00	..#... \ . -=i...E
0010	00 34 f2 f2 00 00 79 06	cb 5c 40 e9 a6 bc 82 be	·4...y· ·\@....
0020	19 11 01 bb c6 b5 75 6d	1a 1d 80 9b fb c6 80 10	um
0030	01 09 28 1e 00 00 01 01	05 0a 80 9b fb c5 80 9b	..(.....
0040	fb c6		..

Qui émet cette trame ? Vers qui ?

Chapitre 2 : Couche 2

1049	7.439198	130.190.25.17	64.233.166.188	TCP	55	50869 → 443 [ACK] Seq=1 Ack=1 Win=512 Len=1 [TCP segment of a reassembled PDU]
1050	7.460329	64.233.166.188	130.190.25.17	TCP	66	443 → 50869 [ACK] Seq=1 Ack=2 Win=265 Len=0 SLE=1 SRE=2
1051	8.440519	130.190.25.17	142.251.37.206	UDP	71	51342 → 443 Len=29

Frame 1050: 66 bytes on wire (528 bits), 66 bytes captured (528 bits) on interface \Device\NPF{...}
 Ethernet II, Src: Cisco_3d:69:ff (5c:a6:2d:3d:69:ff), Dst: Intel_f6:ea:a5 (f4:ce:23:f6:ea:a5)
 Internet Protocol Version 4, Src: 64.233.166.188, Dst: 130.190.25.17
 Transmission Control Protocol, Src Port: 443, Dst Port: 50869, Seq: 1, Ack: 2, Len: 0

```

0000  f4 ce 23 f6 ea a5 5c a6 2d 3d 69 ff 08 00 45 00  ..#... \. -=i...E.
0010  00 34 f2 f2 00 00 79 06 cb 5c 40 e9 a6 bc 82 be  .4...y. \@....
0020  19 11 01 bb c6 b5 75 6d 1a 1d 80 9b fb c6 80 10  .....um .....
0030  01 09 28 1e 00 00 01 01 05 0a 80 9b fb c5 80 9b  ..(.....
0040  fb c6  ..
  
```

Qui émet cette trame : 5c a6 2d 3d 69 ff

Qui reçoit : f6 ce 23 f6 ea a5 (donc moi !)

Carte réseau sans fil Connexion au réseau local* 10 :

```

Statut du média. . . . . : Média déconnecté
Suffixe DNS propre à la connexion. . . :
Description. . . . . : Microsoft Wi-Fi Direct Virtual Adapter #2
Adresse physique . . . . . : F6-CE-23-F6-EA-A5 ←
DHCP activé. . . . . : Non
Configuration automatique activée. . . : Oui
  
```

Chapitre 2 : Couche 2

A vous de lancer Wireshark, et de trouver des trames envoyés par votre ordinateur ..

```
Carte réseau sans fil Connexion au réseau local* 10 :
```

```
Statut du média. . . . . : Média déconnecté
Suffixe DNS propre à la connexion. . . :
Description. . . . . : Microsoft Wi-Fi Direct Virtual Adapter #2
Adresse physique . . . . . : F6-CE-23-F6-EA-A5 ←
DHCP activé. . . . . : Non
Configuration automatique activée. . . : Oui
```


Chapitre 2 : Couche 2

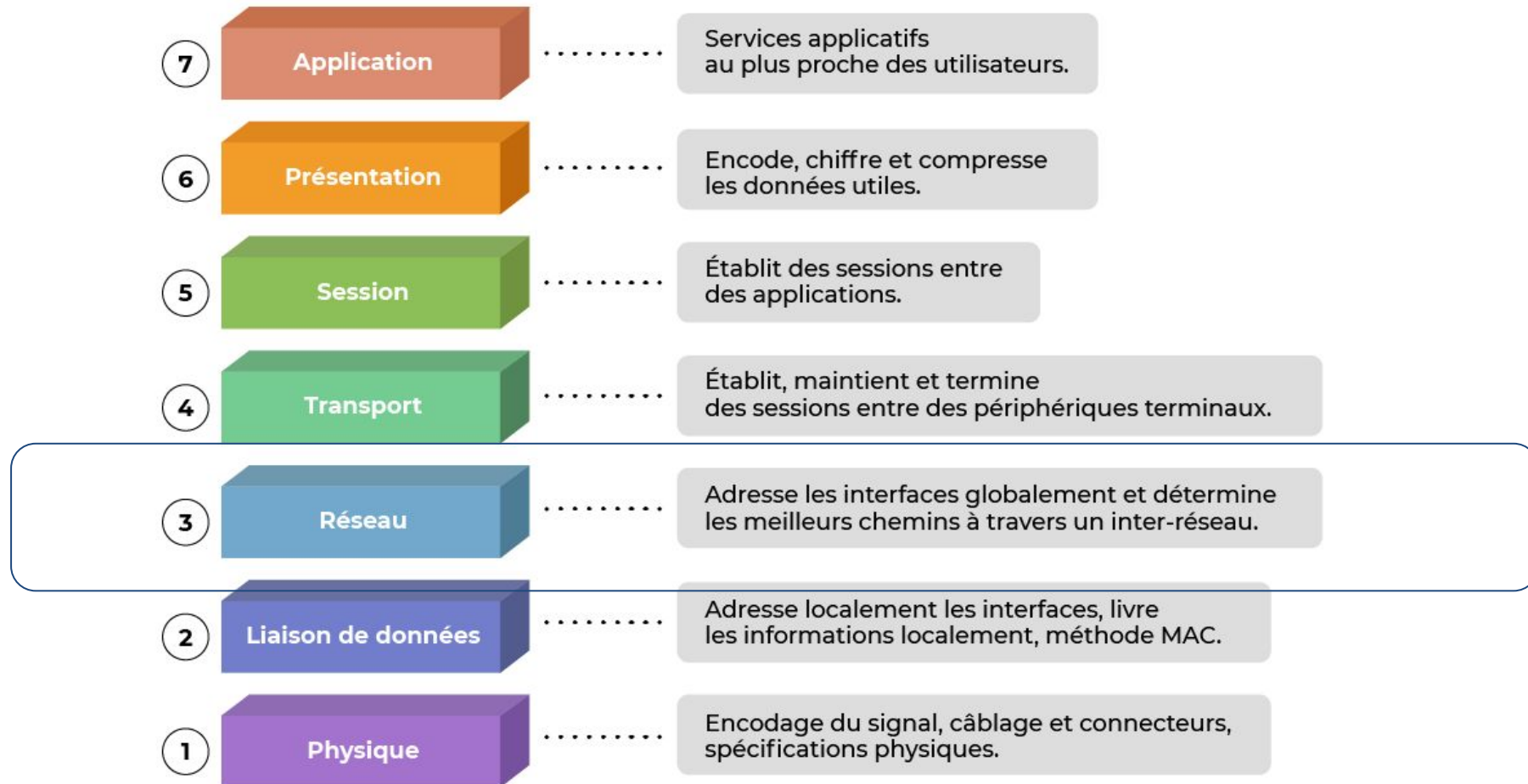
Evidemment les transmissions d'adresse MAC sont primordiales niveau intégrité.

Il y a donc un contrôle qui est fait là dessus : LLS (**L**ogical **L**ink **C**ontrol en anglais).

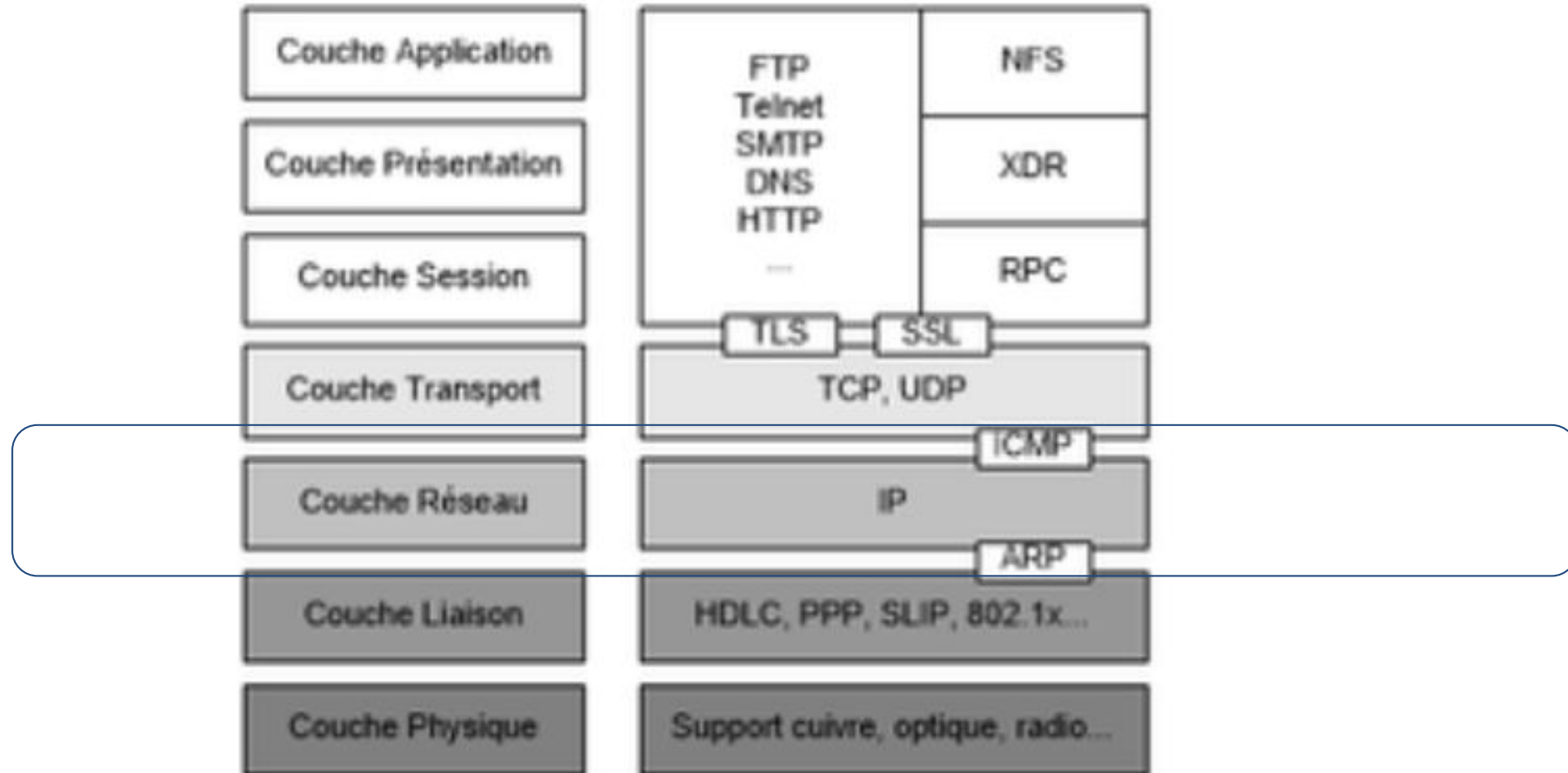
Fonctionnement vous pouvez le voir comme un “pont” entre la couche 2 et 3

Chapitre 3 : Couche 3

Modèle OSI



Chapitre 3 : Couche 3



Chapitre 3 : Couche 3

Les différents types de réseaux

On distingue différents types de réseaux selon leur taille (en terme de nombre de machines), leur vitesse de transfert des données ainsi que leur étendue. On fait généralement trois catégories de réseaux :

- LAN (Local Area Network) : réseau local
- MAN (Metropolitan Area Network) : réseau à l'échelle d'une ville ou d'un campus universitaire
- WAN (Wide Area Network) : il s'agit d'un réseau étendu c'est à dire un réseau informatique (ou de télécommunications) couvrant une grande zone géographique (pays, continent ou la planète entière pour le réseau Internet).

Chapitre 3 : Couche 3

LAN:

Le réseau local LAN (Local Area Network) : C'est un réseau informatique à une échelle géographique relativement restreinte, il est utilisé pour relier entre eux les ordinateurs : par exemple d'une habitation particulière, d'une entreprise, d'une salle informatique, d'un bâtiment. L'infrastructure est privée et est gérée localement. À l'intérieur, ou « sur » le réseau local il y a des ordinateurs fixes ou portables connectés par des câbles ou sans fil (Réseaux locaux sans fil : WLAN).

Ces deux mondes communiquent par l'intermédiaire d'une box ou modem ADSL (selon le FAI). La taille d'un réseau local peut atteindre jusqu'à 100 voire 1000 utilisateurs.

En élargissant le contexte de la définition aux services qu'apportent le réseau local, il est possible de distinguer deux modes de fonctionnement :

- dans un environnement « paire à paire : P2P » (en anglais peer to peer), dans lequel il n'y a pas d'ordinateur central et chaque ordinateur a un rôle similaire.
- dans un environnement « client/serveur », dans lequel un ordinateur central fournit des services réseau aux utilisateurs.

Chapitre 3 : Couche 3

VLAN:

- Un VLAN (Virtual Local Area Network ou Virtual LAN, en français « Réseau Local Virtuel ») est un réseau local regroupant un ensemble de machines de façon logique et non physique.
Ainsi dans un réseau local la communication entre les différentes machines est normalement régie par l'architecture physique. Grâce aux réseaux virtuels (VLANs) il est possible de s'affranchir des limitations de l'architecture physique (contraintes géographiques, contraintes d'adressage, ...) en définissant une segmentation logique (logicielle) basée sur un regroupement de machines grâce à des critères (adresses MAC, numéros de port, protocole, etc.).

Chapitre 3 : Couche 3

MAN :

Le réseau MAN (Metropolitan Area Network) : C'est un réseau métropolitain qui désigne un réseau composé d'ordinateurs habituellement utilisés dans les campus ou dans les villes. Ainsi, un MAN permet à deux nœuds (ordinateurs) distants de communiquer comme si ils faisaient partie d'un même réseau local. Un MAN est formé de commutateurs ou de routeurs interconnectés par des liens hauts débits qui utilisent généralement des fibres optiques. Ces réseaux peuvent être placés sous une autorité publique ou privée comme le réseau intranet d'une entreprise ou d'une ville. Il permet donc pour une société, une ville, de contrôler elle-même son réseau. Ce contrôle comprend la possibilité de gérer, surveiller et effectuer des diagnostics à distance, à la différence de la connexion WAN, pour laquelle elle doit se fier à son fournisseur d'accès pour gérer et maintenir la liaison entre elle et son bureau distant. Ce type de réseau, s'il est municipal par exemple, permet une infrastructure multiservice : il permet de véhiculer la téléphonie, la vidéo surveillance urbaine, la télégestion des feux tricolores, les installations de chauffage, les parkings, l'éclairage de l'Hôtel de Ville, ...

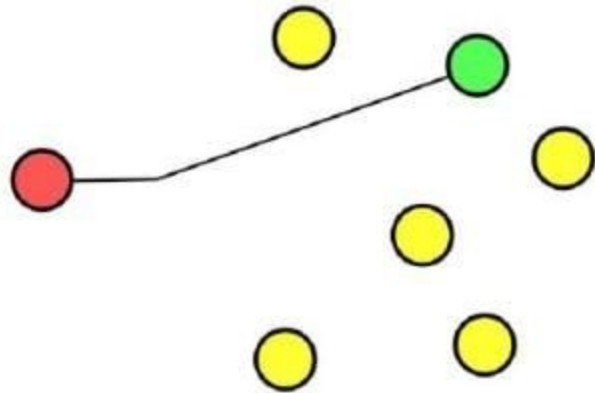
Chapitre 3 : Couche 3

WAN (Wide Area Network) :

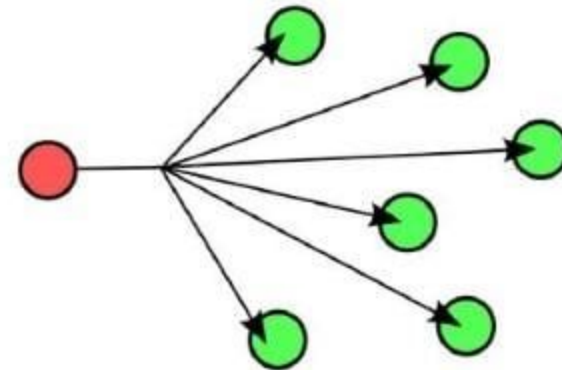
Le réseau Internet (WAN) est un réseau couvrant une grande zone géographique, à l'échelle d'un pays, d'un continent, voire de la planète entière. Il permet l'interconnexion de réseaux locaux et métropolitains vers l'internet mondial. L'infrastructure est en général publique. Le plus grand réseau WAN est le réseau internet : à l'extérieur du réseau dit local, c'est à dire de l'autre côté de la « box » il existe un réseau que l'on nomme communément internet. Les fournisseurs d'accès à internet (ou FAI), moyennant finance, procurent un accès à ce réseau.

Chapitre 3 : Couche 3

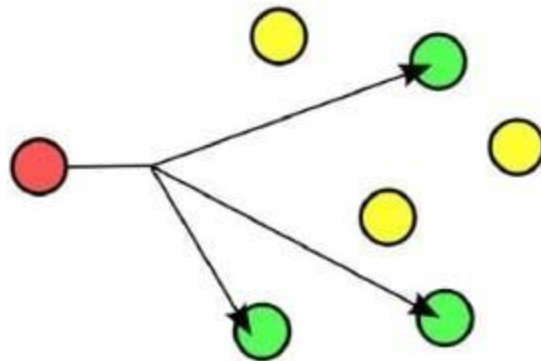
Vocabulaire :



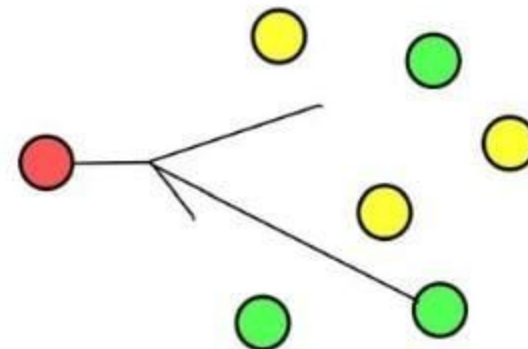
Unicast



Broadcast



Multicast



Anycast

Chapitre 3 : Couche 3

Réseau en étoile

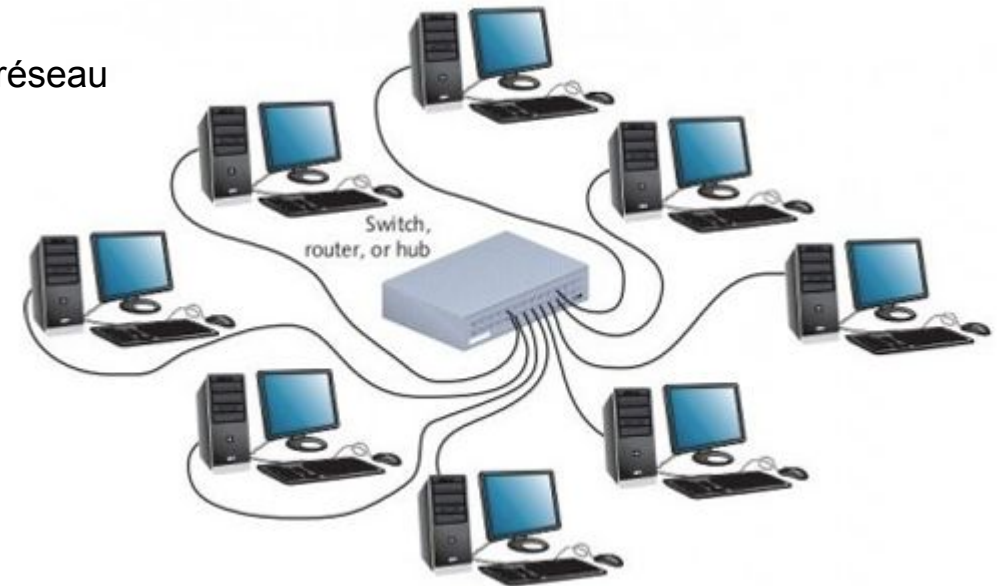
Les équipements du réseau sont reliés à un système matériel central (le nœud). Celui-ci a pour rôle d'assurer la communication entre les différents équipements du réseau. Notamment utilisée par les réseaux Ethernet actuels en RJ45, elle concerne maintenant la majorité des réseaux. Lorsque toutes les stations sont connectées à un commutateur, on parle de topologie en étoile. Les nœuds du réseau sont tous reliés à un nœud central. Dans cette topologie tous les hôtes sont interconnectés grâce à un SWITCH

Les avantages :

- Ajout facile de postes
- Localisation facile des pannes
- Le débranchement d'une connexion ne paralyse pas le reste du réseau
- Simplicité éventuelle des équipements au niveau des nœuds.
- Evolution hiérarchisée du matériel possible.
- On peut facilement déplacer un appareil sur le réseau.

Les inconvénients :

- si le concentrateur est défectueux, tout le réseau est en panne.
- utilisation de multiples routeur ou switch afin de pouvoir communiquer entre différents réseaux ou ordinateur



Chapitre 3 : Couche 3

Réseau en bus

Un réseau en bus est une architecture de communication où la connexion des matériels est assurée par un bus partagé par tous les utilisateurs. Les réseaux de bus permettent de relier simplement de multiples matériels, mais posent des problèmes quand deux machines veulent transmettre des données au même moment sur le bus. Les systèmes qui utilisent une topologie en bus ont normalement un arbitre qui gère l'accès au bus.

Cette topologie en bus a été très répandue car son coût d'installation est faible. Il est très facile de relier plusieurs postes d'une même salle, de relier chez soi deux ou trois ordinateurs. Aujourd'hui cette topologie n'est plus adaptée aux réseaux importants.

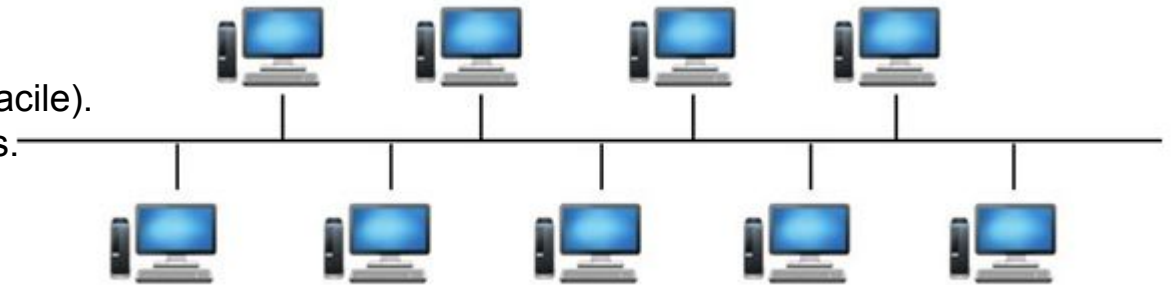
Avantages :

- Facile à mettre en œuvre et à étendre.
- Utilisable pour des réseaux temporaires (installation facile).
- Présente l'un des coûts de mise en réseau le plus bas.

Inconvénients

- Longueur du câble et nombre de stations limités.
- Un câble coupé peut interrompre le réseau.
- Les performances se dégradent avec l'ajout de stations.
 - Faible sécurité des données transitant sur le réseau (toutes les stations connectées au bus peuvent lire toutes les données transmises sur le bus).

On remarquera que la technologie « bus » reste très utilisée dans l'industrie pour raccorder par exemple des capteurs à une unité centrale



Chapitre 3 : Couche 3

Réseau en anneau

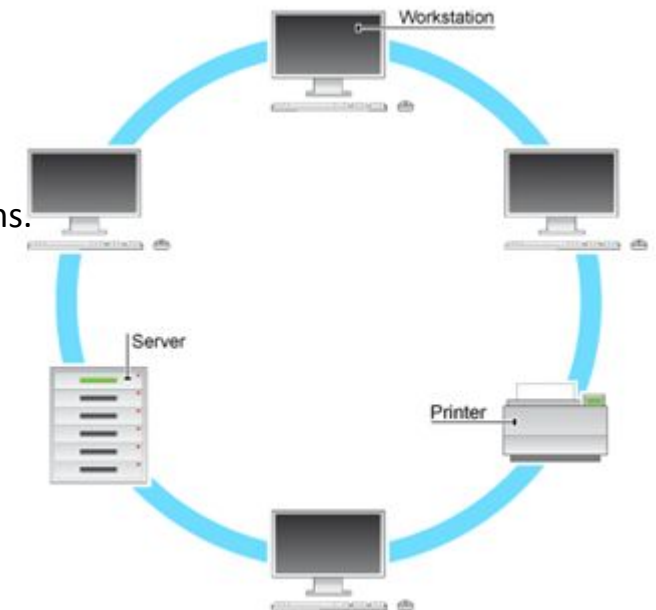
Toutes les machines sont reliées entre elles dans une boucle fermée. Les données circulent dans une direction unique, d'une entité à la suivante. Les ordinateurs communiquent chacun à leur tour. Cela ressemble à un bus mais qui serait refermé sur lui-même : le dernier nœud est relié au premier. Souvent, dans une topologie en anneau, les ordinateurs ne sont pas reliés en boucle, mais sont reliés à un répartiteur (appelé MAU, Multistation Access Unit) qui va gérer la communication entre les ordinateurs qui lui sont reliés en répartissant à chacun d'entre-eux un temps de parole. Elle utilise la méthode d'accès à "jeton" (Token ring). Les données transitent de stations en stations en suivant l'anneau qui chaque fois régénèrent le signal. Le jeton détermine quelle station peut émettre, il est transféré à tour de rôle vers la station suivante. Lorsque la station qui a envoyé les données les récupère, elle les élimine du réseau et passe le jeton au suivant, et ainsi de suite... La topologie en anneau est dite « topologie active » parce que le signal électrique est intercepté et régénéré par chaque machine.

Avantages :

- La quantité de câble nécessaire est réduite
- Le protocole est simple, il évite la gestion des collisions
- Il est assez facile à installer/reconfigurer, ajouter/retirer un matériel -> déplacer seulement 2 connexions.
- Fonctionne mieux qu'une topologie de bus sous une lourde charge de réseau

Inconvénients :

- Le retrait ou la panne d'une entité active paralyse le trafic du réseau.
- Le délai de communication est directement proportionnel au nombre de noeuds du réseau
- Le déplacement, l'ajout et la modification machines connectées peuvent affecter le réseau

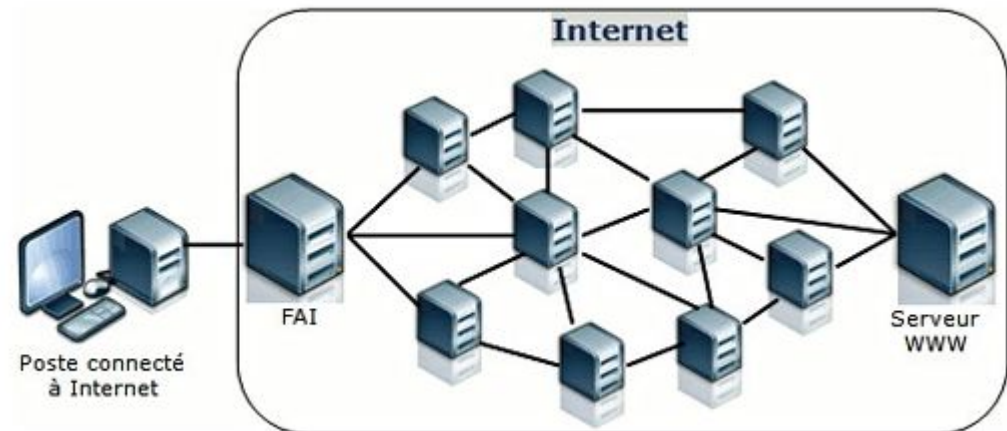
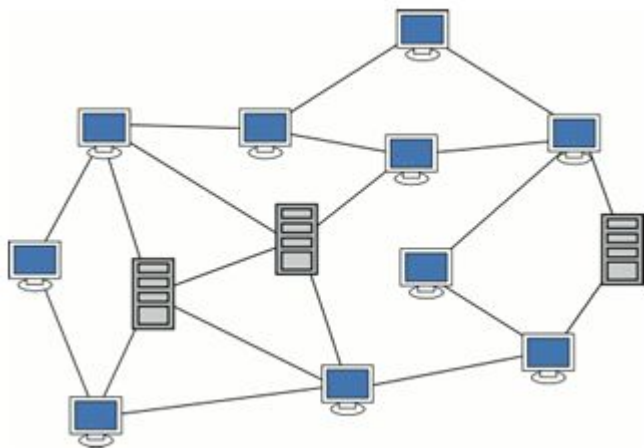


Chapitre 3 : Couche 3

Réseau maillé

Le réseau maillé est une topologie de réseau qualifiant les réseaux (filaire ou non) dont tous les hôtes sont connectés pair à pair sans hiérarchie centrale, formant ainsi une structure en forme de filet. Par conséquent, chaque nœud doit recevoir, envoyer et relayer les données. Cela évite d'avoir des points sensibles, qui en cas de panne, isolent une partie du réseau. Si un hôte est hors service, ses voisins passeront par une autre route.

Les réseaux maillés utilisent plusieurs chemins de transferts entre les différents nœuds. Cette méthode garantit le transfert des données en cas de panne d'un nœud. Le réseau Internet est basé sur une topologie maillée (sur le réseau étendu « WAN », elle garantit la stabilité en cas de panne d'un nœud).



Chapitre 3 : Couche 3

IP principe : assure la livraison de paquet sans connexion et sans garantie.

Il faut un plan d'adressage explicite.

Chaque noeud du réseau doit être identifié par une adresse IP : avec 2 parties : un numéro de réseau logique et une adresse d'hôte sur ce réseau logique.

Métaphore : Numéro de rue pour le numéro IP et Adresse dans cette rue sur le numéro d'hôte.

Gros avantage de l'IP : modularité : mode "urgent", débit "important" ou "haute fiabilité"

Chapitre 3 : Couche 3

IP (v4) est indiqué par 4 octets : par exemple

```
Adresse IPv4. . . . . : 172.20.4.179  
Masque de sous-réseau. . . . . : 255.255.252.0  
Passerelle par défaut. . . . . : 172.20.7.254
```

Le premier octet donne “la classe” de l’adresse

Chapitre 3 : Couche 3

IP (v4) premier octet :

- Classe A : Les adresses IP de classe A ont un premier octet compris entre 1 et 126. Dans cette classe, le premier octet est utilisé pour identifier le réseau, tandis que les trois octets restants sont utilisés pour identifier les hôtes sur ce réseau.
- Classe B : Les adresses IP de classe B ont un premier octet compris entre 128 et 191. Dans cette classe, les deux premiers octets sont utilisés pour identifier le réseau, tandis que les deux derniers octets sont utilisés pour identifier les hôtes sur ce réseau.
- Classe C : Les adresses IP de classe C ont un premier octet compris entre 192 et 223. Dans cette classe, les trois premiers octets sont utilisés pour identifier le réseau, tandis que le dernier octet est utilisé pour identifier les hôtes sur ce réseau.
- Classe D : Les adresses IP de classe D ont un premier octet compris entre 224 et 239. Cette classe est réservée pour les adresses multicast.
- Classe E : Les adresses IP de classe E ont un premier octet compris entre 240 et 255. Cette classe est réservée pour un usage futur et expérimental.

Chapitre 3 : Couche 3

IP notion de masque :

Un masque de sous réseau comme l'adresse IP est une suite de 4 octets, soit 32 bits.

Les bits du masque à 1 représenteront la partie réseau de l'adresse, et les bits à 0 la partie machine.

Le masque servant à faire la séparation en deux parties sur une adresse IP, il est donc indissociable de celle-ci. Une adresse seule ne voudra rien dire puisqu'on ne saura pas quelle est la partie réseau et quelle est la partie machine.

De la même façon, un masque seul n'aura pas de valeur puisqu'on n'aura pas d'adresse sur laquelle l'appliquer.

L'adresse IP et le masque sont donc liés l'un à l'autre, même si l'on peut choisir l'un indépendamment de l'autre.

L'adresse du sous-réseau est obtenue en appliquant l'**opérateur logique ET binaire** entre l'adresse IPv4 et le masque de sous-réseau

Le masque de sous réseau le plus courant, celui que l'on utilise généralement “à la maison” est 255.255.255.0

Chapitre 3 : Couche 3

Masque de sous réseau

un exemple : adresse IP : 192.168.1.2 et le masque de sous réseau 255.255.255.0 :

adresse IP	1 1 0 0 0 0 0 0 . 1 0 1 0 1 0 0 0 . 0 0 0 0 0 0 0 1 . 0 0 0 0 0 0 1 0	(192.168.1.2)
& (opérateur ET)	1 1 1 1 1 1 1 1 . 1 1 1 1 1 1 1 1 . 1 1 1 1 1 1 1 1 . 0 0 0 0 0 0 0 0	(255.255.255.0)
adresse du réseau =	1 1 0 0 0 0 0 0 . 1 0 1 0 1 0 0 0 . 0 0 0 0 0 0 0 1 . 0 0 0 0 0 0 0 0	(192.168.1.0)

Pour trouver l'adresse de notre ordinateur, il faut faire un ET logique entre l'adresse IP et le complément du masque (0.0.0.255 c'est à dire 00000000.00000000.00000000.11111111), on obtient alors 0.0.0.2

Chapitre 3 : Couche 3

Masque IP

En résumé :

si l'adresse IP est 192.45.2.9 et le masque 255.255.255.0 alors

- l'adresse du réseau est : 192.45.2
- l'adresse du matériel (appelé souvent 'hôte') est : 9
- on peut donc avoir sur notre réseau 256 (2^8) adresses différentes (0 à 255), c'est à dire 254 machines (2 adresses sont interdites)

	192	45	2	9
Adresse IP	11000000	00101101	00000010	00001001
	255	255	255	0
Masque	11111111	11111111	11111111	00000000
	192	45	2	0
Sous réseau	11000000	00101101	00000010	00000000
	0	0	0	9
Hôte	00000000	00000000	00000000	00001001

Chapitre 3 : Couche 3

IP notion de masque :

si l'adresse IP est 192.45.35.9 et le masque 255.255.240.0 alors

- l'adresse du réseau est : 192.45.32
- l'adresse du matériel (hôte) est : 3.9

	192	45	35	9
Adresse IP	11000000	00101101	00100011	00001001
	255	255	240	0
Masque	11111111	11111111	11110000	00000000
	192	45	32	0
Sous réseau	11000000	00101101	00100000	00000000
	0	0	3	9
Hôte	00000000	00000000	00000011	00001001

Chapitre 3 : Couche 3

IP notion de masque à vous !

si l'adresse IP est 192.45.48.19 et le masque 255.255.240.0 alors

- l'adresse du réseau est : ?
- l'adresse du matériel (hôte) est : ?

Chapitre 3 : Couche 3

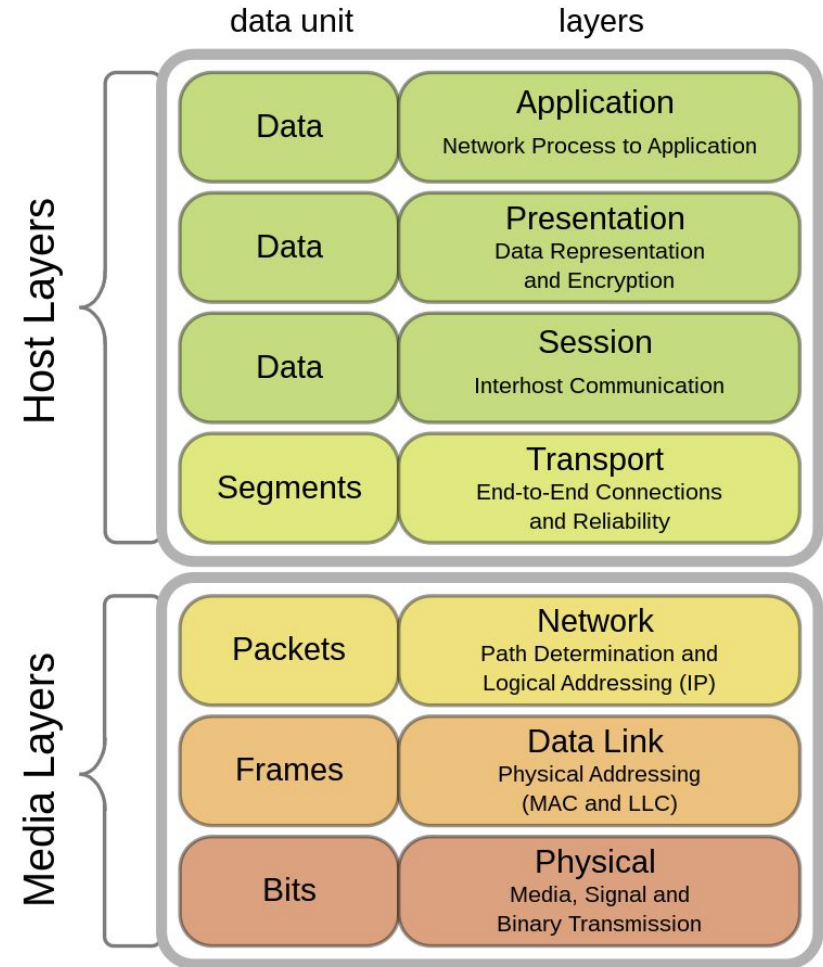
- Le format IPV6 Le nombre d'adresses IP disponibles avec le protocole IPv4, environ 4 milliard, n'est plus suffisant pour répondre à la demande croissante notamment avec l'arrivée des objets connectés.
- C'est pourquoi un nouveau protocole internet a été créé: IPv6 L'adressage se fait sur 16 octets .
- La notation décimale employée pour les adresses IPv4 est abandonnée au profit d'une écriture hexadécimale, où les 8 groupes de 2 octets (soit 16 bits par groupe) sont séparés par un signe deux points.
- Exemple : *2001:0db8:0000:85a3:0000:0000:ac1f:8001*

Il est permis d'omettre de 1 à 3 chiffres zéros non significatifs dans chaque groupe de 4 chiffres hexadécimaux. Ainsi, l'adresse IPv6 ci-dessus est équivalente à :

2001:db8:0:85a3:0:0:ac1f:8001

Couche 4 : la suite !

Le couche 4 est celle du transport (UDP TCP), elle est souvent considérée comme le “lien” entre couche basse (Octets/trame ...) et les couches hautes (applicative ...)



IP/ Adresse Physique

IP et adresse physique sont deux choses différents, néanmoins :

1. Les applications ne connaissent que les IP
2. Les couches les plus basses ce sont les adresses physique qui sont utilisés

Conclusion : Il faut donc pouvoir faire correspondre les deux !

Direct Mapping

Première solution Direct Mapping :

Si adresse physiques < adresse Internet ou utilisateur (admin réseau) peut directement mettre des adresses physique (par exemple en mettant adresse physique = partie host de l'IP)

Il existe dans ce cas une fonction f qui répond à :

$$\text{AdresseIP} = f(\text{AdressePhysique}) \quad \text{AdressePhysique} = f^{-1}(\text{Adresse IP})$$

Si la fonction n'existe pas (ou trop coûteuse) il reste la solution de table de correspondance en dur.

Dynamic Binding

Lorsque la précédente méthode (qui est la plus efficace) n'est pas applicable, par exemple avec les adresses Ethernet, le protocole de bas niveau ARP (***A**ddress **R**esolution **P**rotocol*) peut être utilisé si le réseau possède un mécanisme de diffusion (broadcast) :

- une machine A veut envoyer un paquet à une machine B du même réseau dont elle ne connaît pas l'adresse physique mais seulement son adresse IP : elle envoie en diffusion un paquet particulier d'interrogation contenant l'adresse IP, appelé message ARP d'interrogation,
- toutes les machines reçoivent et analysent ce message,
- seule la machine B, ayant cette adresse IP la reconnaît et doit renvoyer à la machine A un message ARP de réponse contenant son adresse physique
- la machine A reçoit la correspondance entre les deux adresses et la conserve pour la suite. Elle peut désormais envoyer directement les paquets à l'adresse physique de la machine B.

Mémorisation de la correspondance : mémoire cache

Le mécanisme de diffusion est lourd : chaque paquet diffusé est lu et interprété par toutes les machines du réseau alors qu'une seule est intéressée dans ce cas

Il est donc pas souhaitable de diffuser directement le paquet à transmettre à la machine B, car il faudrait par la suite diffuser tous les paquets suivants à destination de cette machine : il faut donc s'en souvenir.

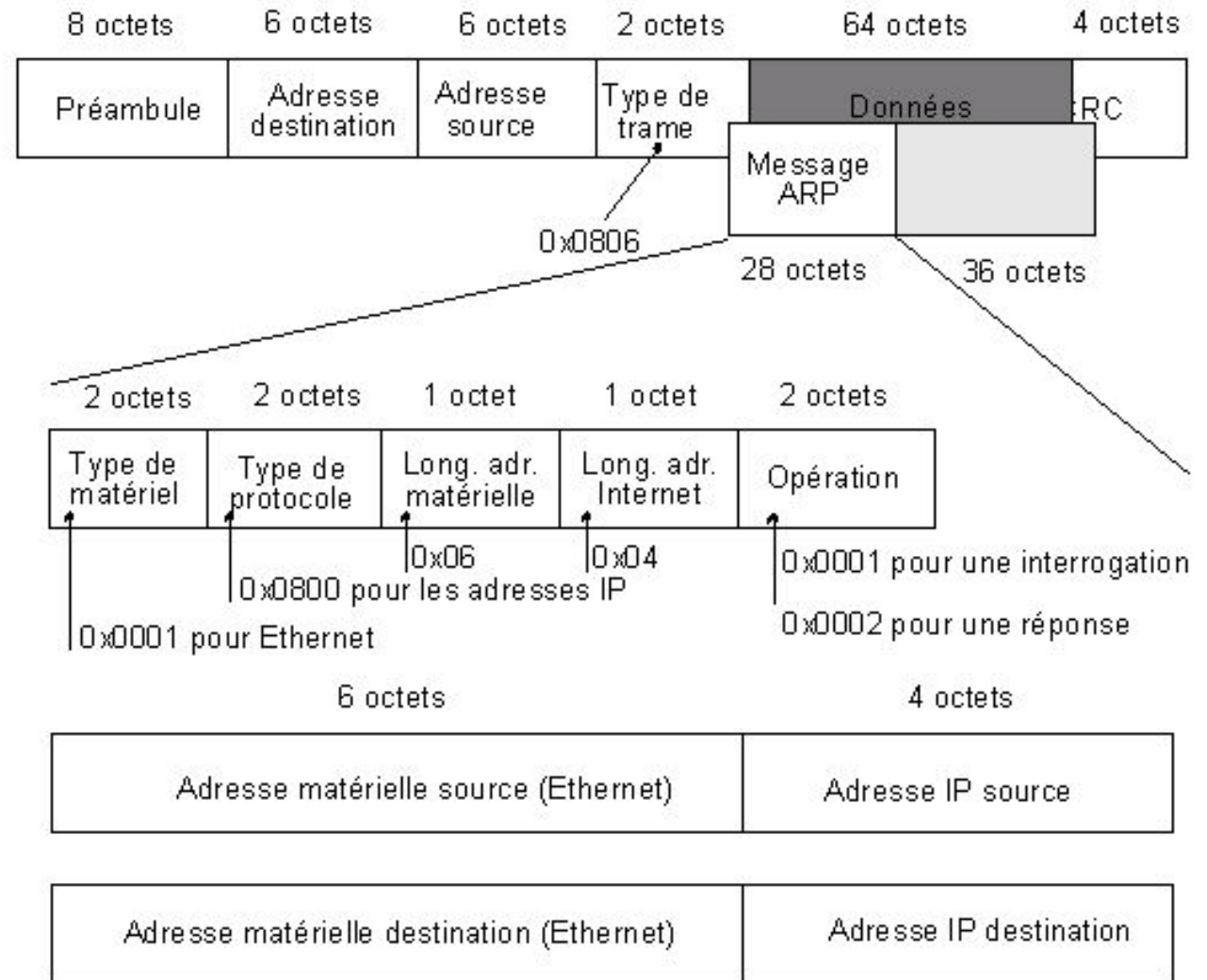
Il est préférable d'utiliser un paquet spécifique d'interrogation, à condition que le protocole ARP conserve dans une mémoire cache les plus récentes adresses Internet utilisées et les adresses physiques correspondantes ce qu'il fait.

Améliorations du protocole ARP

- Lorsque la machine A envoie un paquet à la machine B, il est fort probable que celle-ci doive lui répondre. Le message ARP d'interrogation contient donc aussi l'adresse IP et l'adresse physique de la machine A. La machine B n'a donc pas à faire elle-même l'interrogation par diffusion pour connaître l'adresse physique de la machine A.
- Toutes les machines recevant le message ARP d'interrogation, peuvent elles-mêmes maintenir à jour leurs propres tables de correspondance et ajouter éventuellement celle qui concerne la machine A,
- Si une machine est physiquement modifiée (changement d'une carte de communication par exemple), son adresse physique change mais pas son adresse IP. Les autres machines du réseau, dont le cache ARP a stocké l'ancienne correspondance, ne sont pas à jour. Il est donc souhaitable qu'une diffusion de la nouvelle correspondance soit effectuée par une machine à son redémarrage lorsque son adresse physique a changé.

Trame d'un message ARP

- Exemple trame sur réseau Ethernet



Recherche d'adresse IP au démarrage

Une machine qui démarre, est ce qu'elle a une IP ?

Si elle démarre avec un historique ou pas ...

Avec un système d'exploitation ou pas ...

Comment peut à votre avis peut on faire ?

Recherche d'adresse IP au démarrage

- Problématique

Lorsqu'une machine ou un serveur démarre, elle peut aller chercher dans un fichier de configuration local sa propre adresse IP. La couche réseau du système d'exploitation est alors immédiatement à même de communiquer.

Si machine *diskless*, celle-ci doit charger l'image du système d'exploitation que lui envoie le serveur. Mais elle doit pour cela établir la communication avec le serveur !!. Or elle ne connaît pas son adresse IP qu'elle ne peut pas stocker. Pour sa part, le programme de *boot* est mémorisé en ROM, mais l'adresse IP ne peut y être conservée (il serait impossible de la modifier si la machine était connectée sur un autre réseau).

La machine ne peut donc s'adresser au serveur qu'elle ne connaît pas non plus. Pour résoudre cette dernière difficulté, il lui suffit de diffuser une demande auprès de toutes les machines du réseau. Mais il faut qu'elle puisse être identifiée par le serveur.

RARP (*Reverse Address Resolution Protocol*)

- Solution

Le programme de *boot* contenu dans la ROM interroge l'interface réseau connecté pour récupérer son adresse physique. C'est cette adresse qui est diffusée dans le réseau.

Un message RARP a le même format qu'un message ARP. La machine qui interroge remplit le champ "opération" à la valeur "requête", et les champs "adresse physique source" et "adresse physique destination" à sa propre adresse physique.

Un serveur d'adresses, dit "serveur RARP", reconnaît ce message, remplit le champ "opération" à la valeur "réponse", l'adresse IP de la machine "destination" et le renvoie à l'adresse physique indiquée.

Et en cas d'échec ?

Il est possible que le message RARP ou sa réponse se perdent, en particulier si le serveur RARP est unique est trop occupé.

La machine *diskless* peut alors soit réémettre indéfiniment le message d'interrogation, soit renoncer au bout d'un certain nombre de tentatives infructueuses et signaler l'échec de la connexion.

S'il y a plusieurs serveurs RARP (solution plus fiable), la machine qui interroge a beaucoup plus de chances de recevoir une réponse. Néanmoins, deux problèmes doivent être résolus : éviter la surcharge du réseau par l'émission de nombreuses réponses, et éviter la collision des réponses.

La solution au premier problème consiste à affecter chaque machine *diskless* à un serveur principal. Lorsque la requête est diffusée, tous les serveurs RARP la reçoivent mais seul le serveur principal doit répondre immédiatement.

Les serveurs secondaires se contentent d'enregistrer la demande avec l'heure d'arrivée. Si le serveur principal ne répond pas au bout d'un certain temps (*timeout*), la machine *diskless* réémet la requête. Les serveurs secondaires doivent alors répondre. Pour éviter qu'ils répondent simultanément, provoquant ainsi une collision généralisée, chacun calcule un délai aléatoire avant d'envoyer la réponse.

Inconvénients RARP

Les principaux inconvénients de RARP sont :

- d'être de bas niveau (interaction avec le matériel) et donc d'être très difficilement intégrables dans des applications serveurs de haut niveau,
- de gaspiller beaucoup d'utilisation du réseau avec des trames très courtes, ce qui nécessite d'utiliser des bits dit de "bourrage" si les trames physiques peuvent ou même doivent être beaucoup plus longues (par exemple Ethernet),
- de nécessiter une adresse physique fixe, ce qui est incompatible avec les réseaux qui attribuent des adresses physiques dynamiquement.

Protocole BOOTP

Le protocole BOOTP (***BOOTstrap Protocol***) utilise UDP et IP, et peut donc être utilisé au sein d'une application. Il ne nécessite qu'un seul échange de trames, diffusées dans les deux sens en *broadcast* limité au réseau (adresse = 255.255.255.255), dont la longueur fixe est de 279 octets :

- le code indiquant s'il s'agit d'une requête ou d'une réponse,
- l'adresse physique du client l'adresse IP du client (le demandeur) remplit par lui s'il la connaît,
- l'adresse IP du client remplie en réponse par le serveur si le client ne la connaît pas,
- l'adresse IP ou le nom d'une machine serveur, si le client connaît le serveur auquel il doit s'adresser,
- l'adresse IP d'un routeur,
- le nom du fichier de *boot* à utiliser. Dans la requête, le client indique seulement un nom générique (par exemple "Unix") ou même un champ nul, alors que la trame renvoyée par le serveur contient le véritable nom du fichier de *boot*, que le client peut demander à l'étape suivante (en utilisant le protocole de transfert TFTP : ***Trivial File Transfer Protocol***) et qui contient l'image mémoire du système pour démarrer. Cela offre la possibilité à un serveur d'une part de prendre en compte des configurations hétérogènes, avec plusieurs systèmes d'exploitation et des machines différentes, d'autre part de ne pas gérer lui-même toutes les images mémoire. Le serveur joue alors simplement le rôle de serveur d'images mémoire.

Protocole DHCP (*Dynamic Host Configuration Protocol*)

3ème protocole qui est utilisé majoritairement :

Le principe du protocole DHCP est de disposer sur un serveur d'un certain nombre d'adresses IP qui seront allouées à un client pour un laps de temps fini demandé par le client, imposé par le serveur, en tout cas négocié entre les deux partis. Comme BOOTP, le protocole DHCP reconnaît le client qui l'interroge par un identifiant, en général son adresse physique. Grâce à la souplesse de DHCP, l'attribution de l'adresse peut être effectuée soit "à la main" par l'administrateur, soit définitivement à une valeur fixe au moment de la première requête, soit enfin dynamiquement à chaque requête.

Le message DHCP contient les mêmes informations que le message BOOTP, augmenté d'un champ options. La requête est en général diffusée sur le réseau local, à tous les serveurs d'adresses. Ceux qui reconnaissent ce client par son identifiant lui renvoient (en principe en unicast) les informations dont ils disposent. Le client en choisit alors un et négocie alors la durée d'utilisation maximale de l'attribution de l'adresse.

Le client peut (doit) libérer l'adresse lorsqu'il a terminé sa connexion avant expiration de la durée accordée. Inversement, un mécanisme simple lui permet en cours d'utilisation de demander un prolongement à son serveur (ou éventuellement à un autre si le premier n'est plus accessible).

Voyons maintenant comment IP-Application

IP n'est pas fiable à 100% il faut donc l'encapsuler

- la communication peut être considérée comme un **flux** d'octets : les octets sont reçus dans l'ordre dans lequel ils sont émis, sans aucune structure, ni séparation, ni limite,
- il existe une connexion par un "**circuit virtuel**" entre les deux extrémités de la communication : tout se passe comme si les deux extrémités étaient physiquement connectées. Le protocole TCP se charge donc d'établir la connexion de bout en bout, d'en vérifier la persistance pendant toute la durée de la communication, et de terminer proprement par une phase de déconnexion,
- les transferts sont tamponnés ; à chaque écriture de l'application ne correspond pas forcément une émission de données. Le protocole TCP effectue les émissions physiques de façon optimisée. Il est cependant possible de forcer l'émission au moyen du mécanisme "**flush**",
- la connexion est "full duplex", c'est-à-dire à double sens. Les deux entités communicantes peuvent sans problème émettre en même temps : les informations se croisent. En fait, TCP gère deux tampons, l'un d'admission, l'autre de réception. La réduction du trafic est obtenue par une astuce appelée le **piggybacking** : les acquittements des datagrammes reçus sont, si possible, renvoyés en même temps que des données.

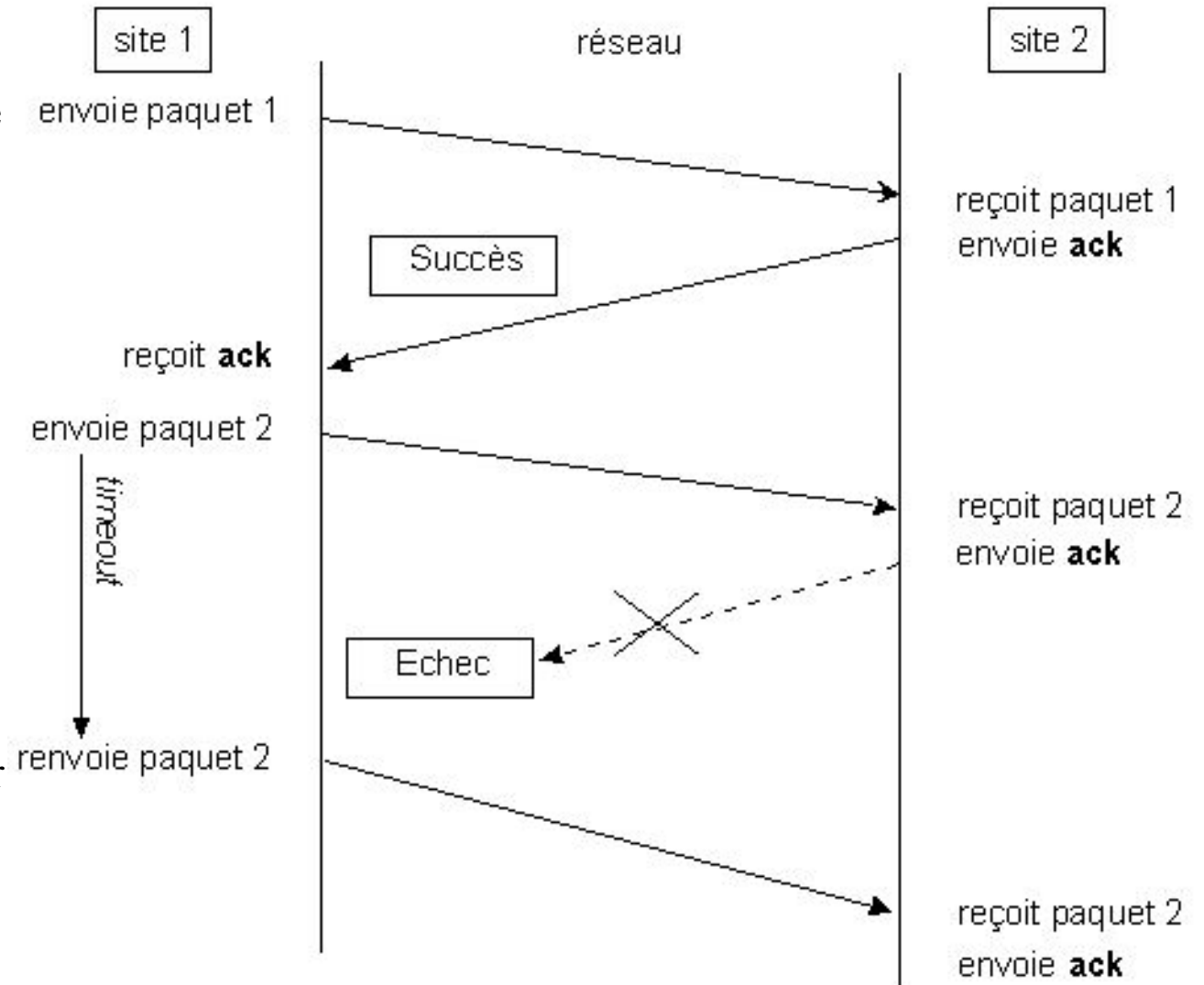
Principe de l'acquittement positif avec retransmission TCP

Après avoir envoyé un paquet, le site émetteur 1 attend l'acquittement que doit lui envoyer en retour le site récepteur 2.

S'il le reçoit dans un délai fixé, il envoie le paquet suivant.

Sinon il réémet le paquet. Plusieurs phénomènes peuvent provoquer un échec de transmission : le paquet peut avoir été perdu, ou l'acquittement. Dans ces deux cas la réémission corrige la perte. Il peut aussi arriver que l'acquittement arrive hors délai.

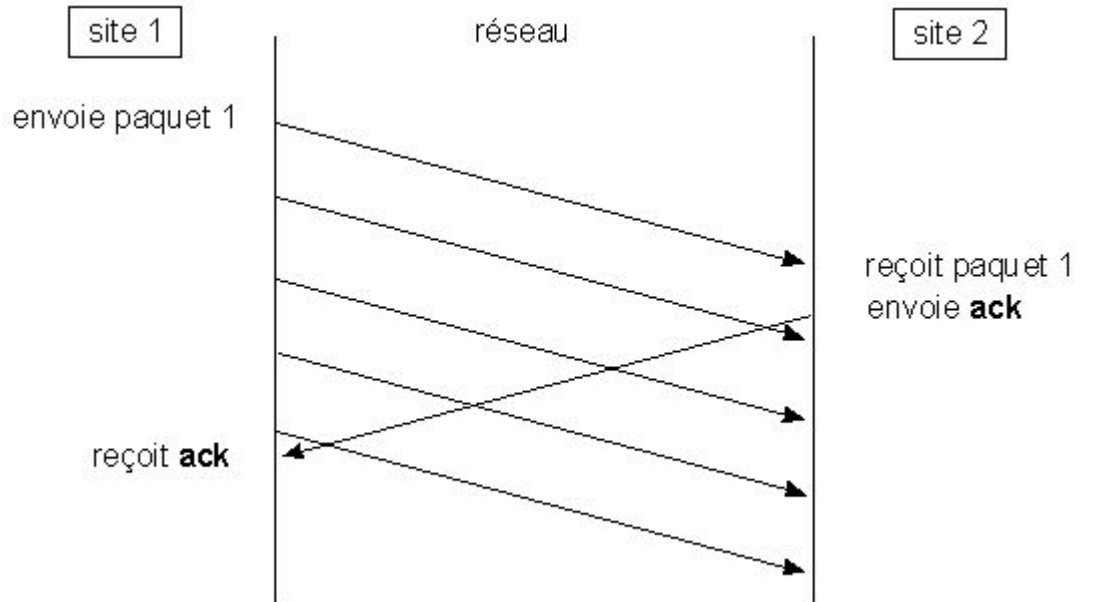
Dans ce cas le site 1 a réémis le paquet A et peut penser que c'est l'acquittement de ce paquet qu'il reçoit. Il risque d'émettre un nouveau paquet et d'introduire ainsi un décalage entre les paquets et leur acquittement. Il faut donc préciser dans chaque acquittement quel est l'identificateur du paquet acquitté.



Fenêtre d'émission

Pour optimiser les transferts on peut envoyer plusieurs paquets à la suite, sans attendre l'acquittement !

Les paquets doivent être numérotés. En fait il n'est pas nécessaire que tous les acquittements parviennent à l'émetteur : l'acquittement d'un paquet acquitte en même temps tous les paquets précédents.

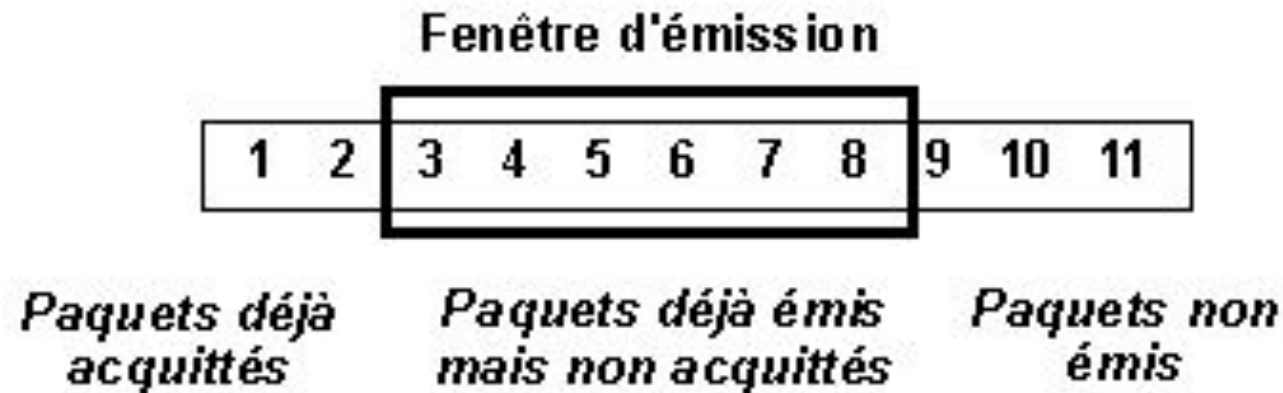


Fenêtre d'émission

En effet le récepteur peut recevoir des paquets dans le désordre, s'ils sont acheminés par des voies différentes. Il les remet dans l'ordre en les stockant provisoirement, mais ne les acquitte que lorsqu'il a reçu tous les paquets précédents.

Par exemple, lorsque le récepteur a reçu les paquets 2, 3 et 4, il lui suffit d'acquitter le 4 lorsqu'il reçoit le paquet 1 pour que l'ensemble des paquets soient acquittés.

Du côté de l'émetteur, lorsqu'il ne reçoit pas dans les délais l'acquittement d'un paquet, il le réémet automatiquement ainsi que tous les paquets suivants. La fenêtre d'émission par exemple :



Buffer TCP

Le principe de la fenêtre d'émission s'applique lorsque les données sont envoyées sous forme de paquets séparés. Le protocole TCP est orienté "flux d'octets".

L'émetteur gère les données octet par octet : chaque octet est numéroté. L'émetteur possède trois pointeurs sur le *buffer* d'émission : un pointeur sur le dernier octet acquitté (début de la fenêtre), un pointeur sur le dernier octet qui a été émis mais dont l'acquittement n'est pas encore revenu, et un pointeur sur le dernier octet de la fenêtre qui peut être envoyé avant qu'un acquittement soit reçu (fin de la fenêtre).

Les octets à transmettre sont en principe envoyés en une seule fois dans un segment, et transmis tels quels au protocole IP qui en fait un datagramme. En fait la largeur de la fenêtre est variable dans le temps.

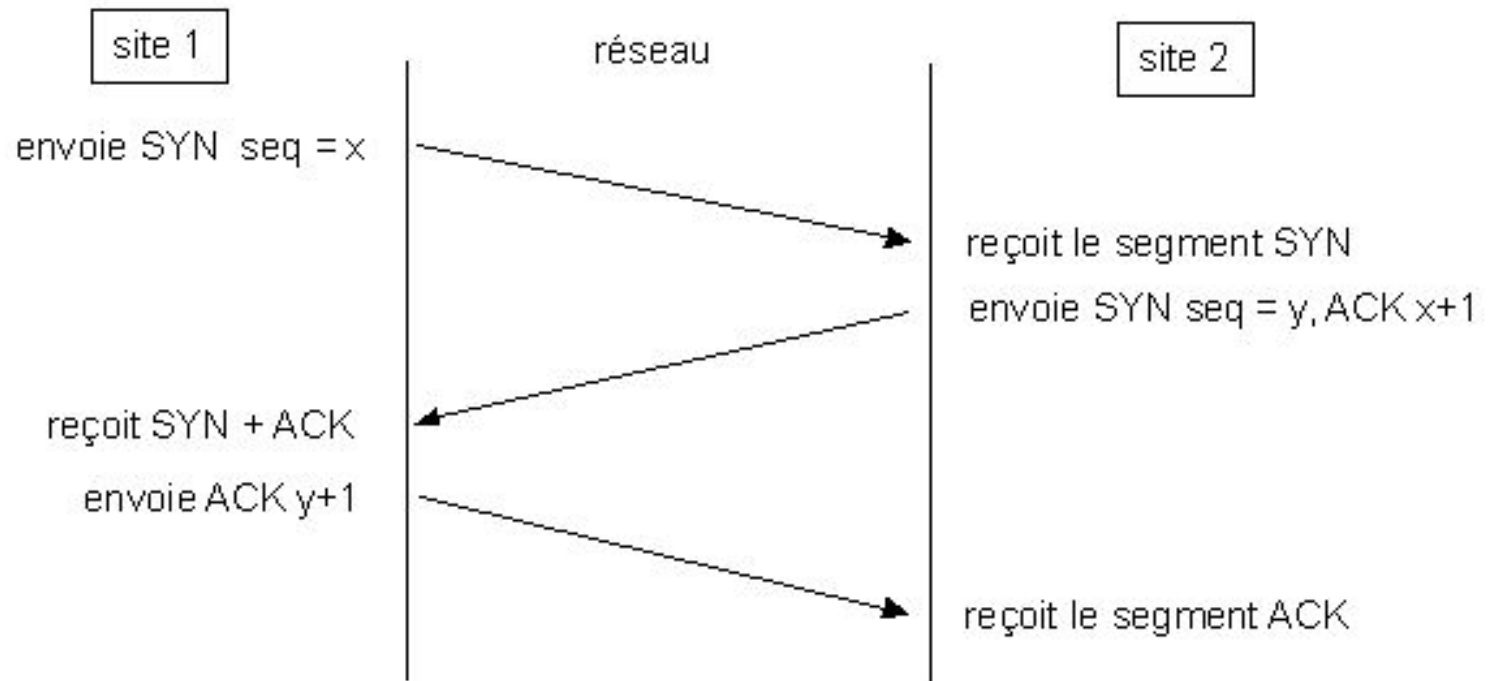
Chaque fois que le site récepteur acquitte une donnée, il y joint une indication sur le nombre d'octets supplémentaires qu'il est prêt à recevoir (c'est la taille restante de son *buffer*). L'émetteur ajuste la taille de la fenêtre d'émission en fonction de cette information. L'acquittement contient le numéro du premier octet manquant dans la reconstitution du flux par le site récepteur. Il est possible que les octets ultérieurs aient été reçus et stockés, mais ils seront acquittés lorsque toute la séquence aura été reconstituée.

Chacun des deux sites gère deux *buffers*, l'un en émission, l'autre en réception.

Etablissement de la connexion TCP

Cette opération est une des caractéristiques du protocole TCP : les deux entités établissent une connexion au moyen d'un mécanisme appelé "**triple poignée de mains**" (*three-way handshake*) :

Le site 1 envoie un segment **SYN** qui indique la demande de synchronisation, accompagné d'un numéro de séquence x choisi de façon aléatoire, qui est le premier numéro de séquence initial. Le site 2 renvoie l'acquittement **ACK** accompagné du numéro $x+1$ de l'octet qu'il attend. Dans le même segment il demande lui-même une synchronisation **SYN** accompagnée du numéro de séquence y de départ qu'il choisit. Le site 1 reçoit ce segment qu'il acquitte par **ACK** en même temps qu'il renvoie le numéro $y+1$ qu'il attend.



Etablissement de la connexion TCP

Faite un diagramme : pour 3 sites (A,B,C) qui se synchronise avec un serveur commun (S)

A envoie à T la demande de SYN. A met +5 temps à répondre à une demande.

B envoie à T+1 la demande de SYN. B met +1 temps à répondre à une demande.

C envoie à T+2 la demande de SYN. C met +1 temps à répondre à une demande.

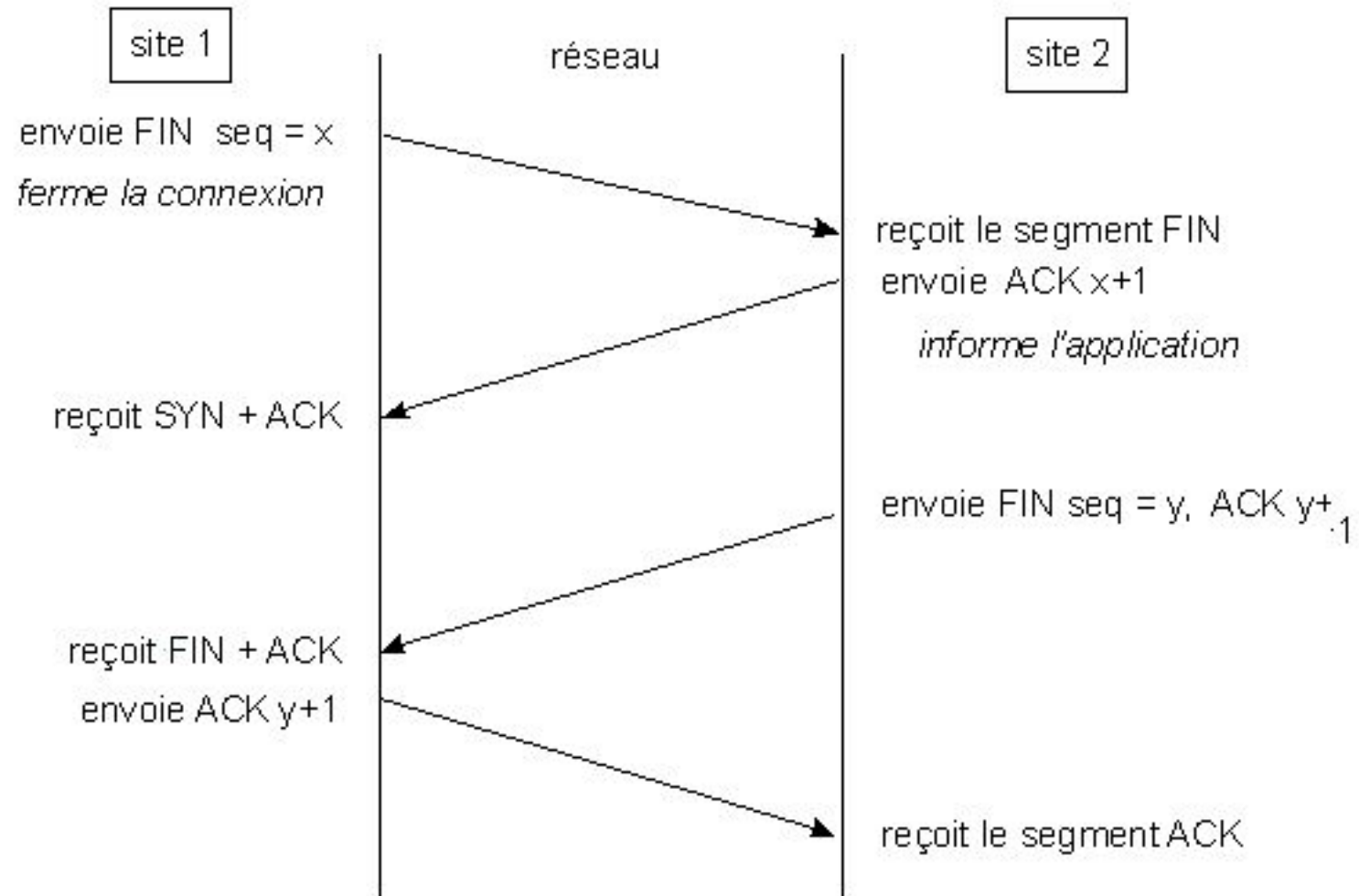
Une trame met +1 temps à passer dans le réseau.

Le serveur S envoie des réponses uniquement à $T + (\text{nombre impair})$, une à la fois et en FIFO.

Donner les temps de synchro de A,B,C avec S

Etablissement de la Déconnexion TCP

Le système est le même que pour la connexion



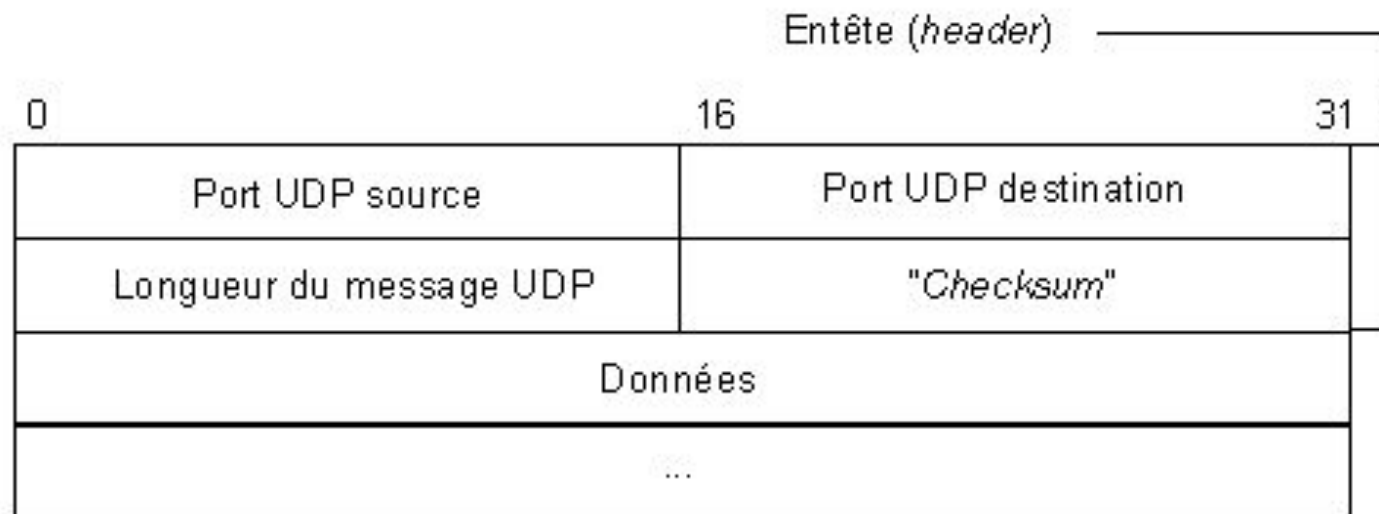
Protocole UDP

UDP est une version simple d'UDP

UDP ne vérifie pas la synchronisation aussi fortement que TCP

UDP envoie sa trame et ne s'en occupe plus.

Donc à votre avis dans quel cas on doit utiliser UDP ou TCP sur les applications ?



Protocole UDP vs TCP

Là où on a besoin de certitude de la donnée on va utiliser

TCP : message asynchrone, mail, Transfert de fichier (On ne tolère pas une seule perte)

UDP : Streaming direct, Voix sur IP (Tolérance de perte de paquet)

ICMP (Internet Control Message Protocol)

Le protocole IP n'offre aucune garantie qu'un datagramme est correctement arrivé et plusieurs problèmes peuvent survenir qui empêchent son fonctionnement correct.

C'est le rôle du protocole ICMP d'offrir un mécanisme de "**compte-rendu d'erreur**" (et non de correction).

Un datagramme IP peut être rejeté si le numéro de port n'existe pas. De nombreux autres problèmes peuvent subvenir lors de l'acheminement d'un datagramme IP : si une machine est déconnectée (provisoirement ou définitivement) le routeur ne peut lui envoyer le datagramme, si la durée de vie d'un paquet est épuisée le routeur ne peut le délivrer, si un routeur est engorgé les datagrammes ne peuvent plus progresser.

En cas de problème, le protocole ICMP ne corrige pas l'erreur mais envoie un message à l'expéditeur qui doit à son tour informer l'application. En effet, la seule information dont dispose le routeur est l'adresse de la source (adresse IP source dans le datagramme en cause), mais si le problème provient par exemple d'un routeur intermédiaire, il n'y a aucun moyen de l'informer.

Un message ICMP en erreur ne génère PAS de message ICMP (on évite le côté infinite loop)

Couche 5 : La couche session

Permet aux utilisateurs de machines distantes d'établir des sessions entre eux.

Si il est essentiel que les deux côtés ne lancent pas la même opération en même temps, c'est cette couche qui gère le jeton.

Gère la synchronisation et place des points de reprises dans le flot de données.

Exemples de protocoles: | SSH | IRC | HTTP

Couche 5 : couche session

Permet aux utilisateurs de machines distantes d'établir des sessions entre eux.

Si il est essentiel que les deux côtés ne lancent pas la même opération en même temps, c'est cette couche qui gère le jeton. Elle permet la synchronisation du dialogue et des hotes.

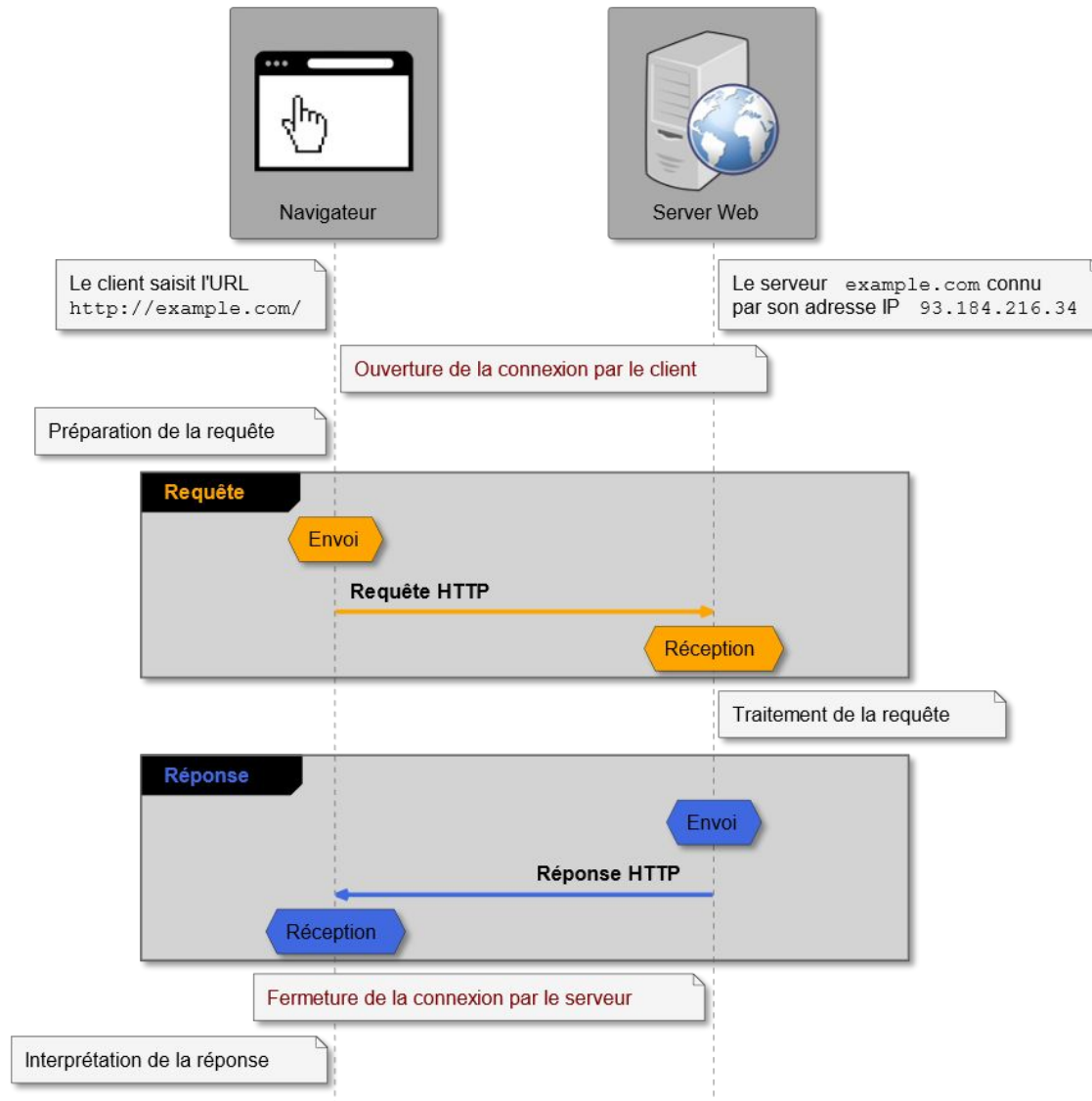
Gère la synchronisation et place des points de reprises dans le flot de données.

Exemples de protocoles: | SSH | IRC | HTTP

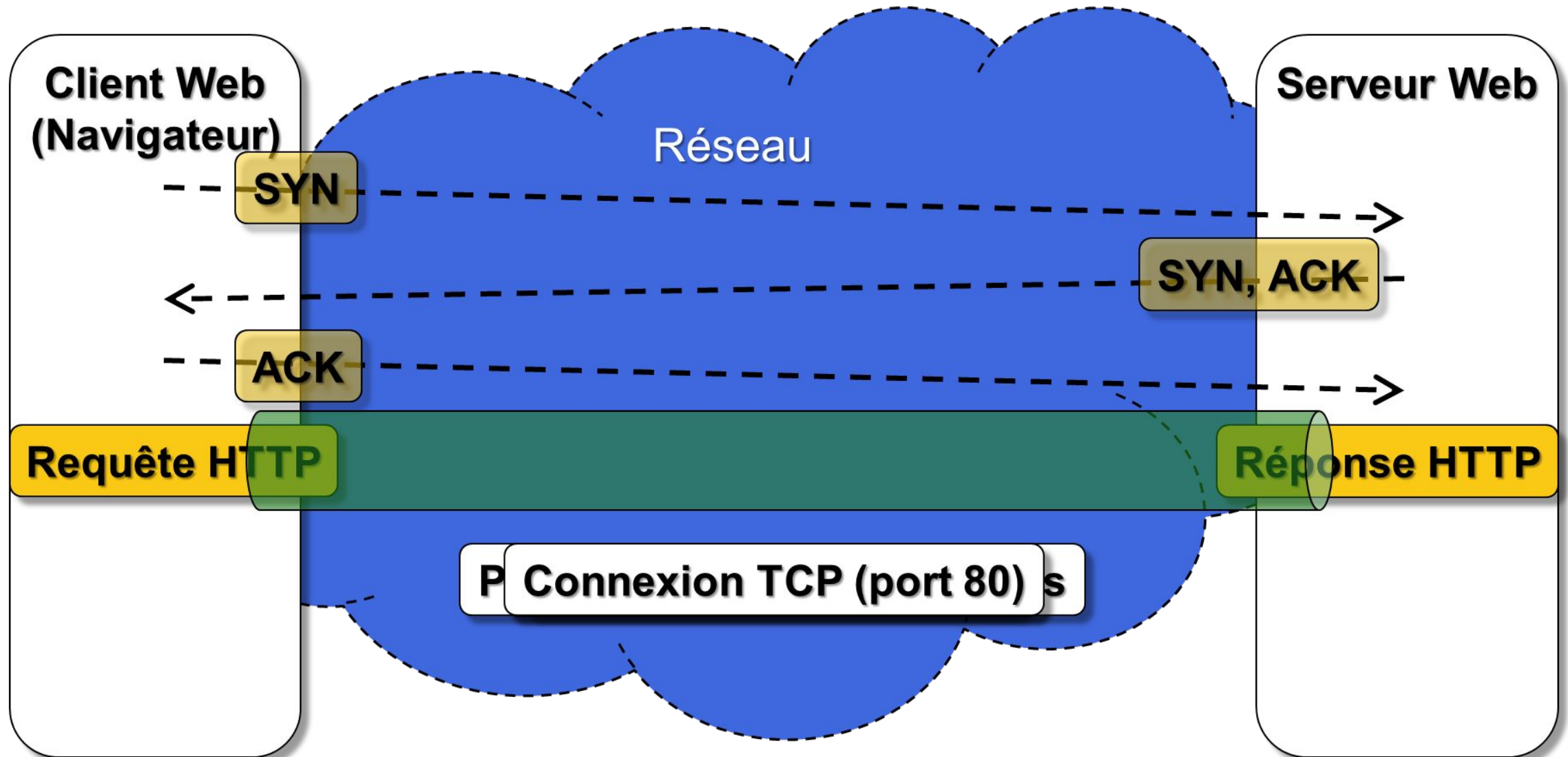
SSH (*Secure Shell*) : permet de crypter les échanges à base de clef privé/publique

IRC : (*Internet Relay Chat*) : Communication textuelle sur Internet

Couche 5 : HTTP (Hypertext Transfer Protocol)



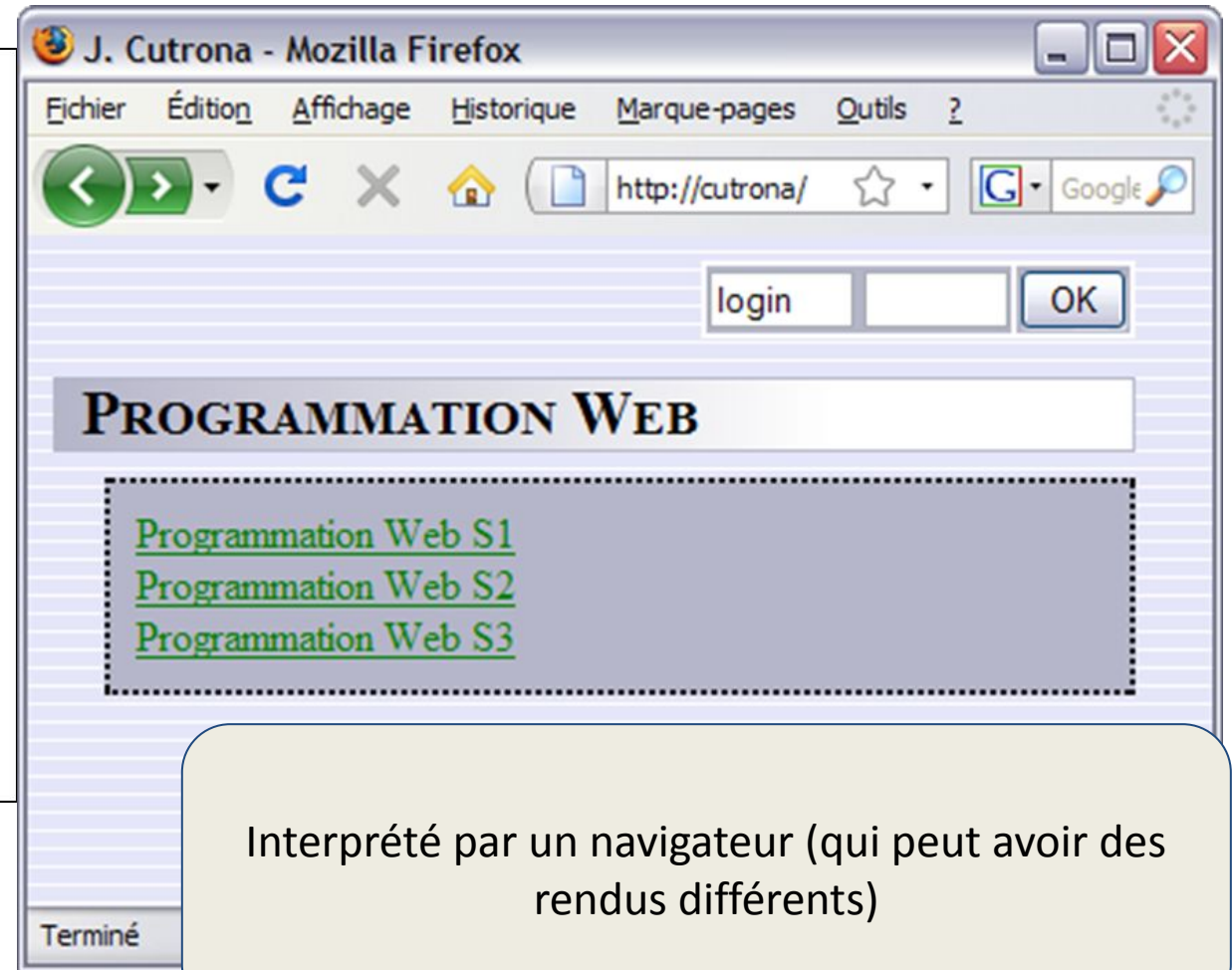
Couche 5 : HTTP (Hypertext Transfer Protocol)



Couche 5 : HTTP (Hypertext Transfer Protocol)

```

<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN">
<html>
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=utf-8">
    <link href="css/style.css" rel="stylesheet" type="text/css">
    <title>J. Cutrona</title>
  </head>
  <body>
    <div id="b_connexion">
      <form action="/index.php" method="post">
        <table>
          <tr><td><input type="text" name="login" size="5" value="login" onClick="if (this.value=='login') this.value=''>
            <input type="password" name="passwd" size="5">
          </td><td><input type="submit" value="OK">
        </tr>
        </table>
      </form>
    </div>
    <h1>Programmation Web</h1>
    <div class="partie">
      <a href="progwebS1">Programmation Web S1</a><br>
      <a href="progwebS2">Programmation Web S2</a><br>
      <a href="progwebS3">Programmation Web S3</a><br>
    </div>
    <span id="lastmodified">
      Dernière modification de cette page le 07/09/2008 à 23h27 </span>
    <div class="valid">
      <a href="http://www.w3c-validator/check?uri=referer">
        </a>
        
      </div>
    </body>
  </html>
  
```



Couche 5 : HTTP (Hypertext Transfer Protocol)

Le **client ouvre** la connexion

Le **serveur ferme** la connexion

1 transaction = 1 ressource transférée (v 1.0) , à partir de la v1.1 le client peut demander à rester connecté

Protocole sans état

Aucune information gardée entre deux transactions

Le serveur "oublie" le client après chaque transaction

Problème pour la gestion d'une session

Couche 5 : HTTP (Hypertext Transfer Protocol)

Le **client demande** des ressources

Le **serveur répond** aux demandes des clients :

Délivre la ressource demandée si possible

Informe de la raison de non remise

Echanges multi-plateforme

ASCII 7bits (encodage si non ASCII 7bits)

Requêtes émises en clair

Réponses émises en clair

Couche 5 : HTTP (Hypertext Transfer Protocol)

Le port standard du protocole HTTP est le port 80 en TCP.

N'importe quel autre port TCP peut être utilisé par HTTP (8080, 8081).

Couche 5 : HTTP (Hypertext Transfer Protocol)

une page est en principe définie par un URI (Unified Ressource Identifier). On dit aussi URL (Unified Ressource Locator). Par exemple : <http://www.debian.org/doc/manuals/reference/ch-preface.fr.html>. Nous savons que dans cet URI, il y a trois parties :

- [http:](#)
qui indique quel protocole nous allons employer ;
- [//www.debian.org](#)
Qui est sensé représenter le serveur web contenant l'information que l'on recherche ;
- [/doc/manuals/reference/ch-preface.fr.html](#)
qui représente le chemin complet de la page visitée dans l'arborescence du serveur concerné.

Pourtant, si nous indiquons comme URI <http://www.debian.org> , il manque toute la partie correspondant au nom de la page. Cependant, la page est interprétée tout de même : parce que le serveur est paramétré pour ajouter [/index.html](#) s'il n'y a pas le nom de la page. Ainsi, <http://www.debian.org> est équivalent à <http://www.debian.org/index.html>.

Couche 5 : HTTP (Hypertext Transfer Protocol)

Principales méthodes en Http :

GET

C'est la méthode la plus courante pour demander une ressource. Une requête GET est sans effet sur la ressource.

HEAD

Cette méthode ne demande que des informations sur la ressource, sans demander la ressource elle-même.

POST

Cette méthode doit être utilisée lorsqu'une requête modifie la ressource.

OPTIONS

Cette méthode permet d'obtenir les options de communication d'une ressource ou du serveur en général.

CONNECT

Cette méthode permet d'utiliser un proxy comme un tunnel de communication.

TRACE

Cette méthode demande au serveur de retourner ce qu'il a reçu -> tester et effectuer un diagnostic sur la connexion.

PUT

Cette méthode permet d'ajouter une ressource sur le serveur.

DELETE

Cette méthode permet de supprimer une ressource du serveur

.

Objectif désormais mise en pratique



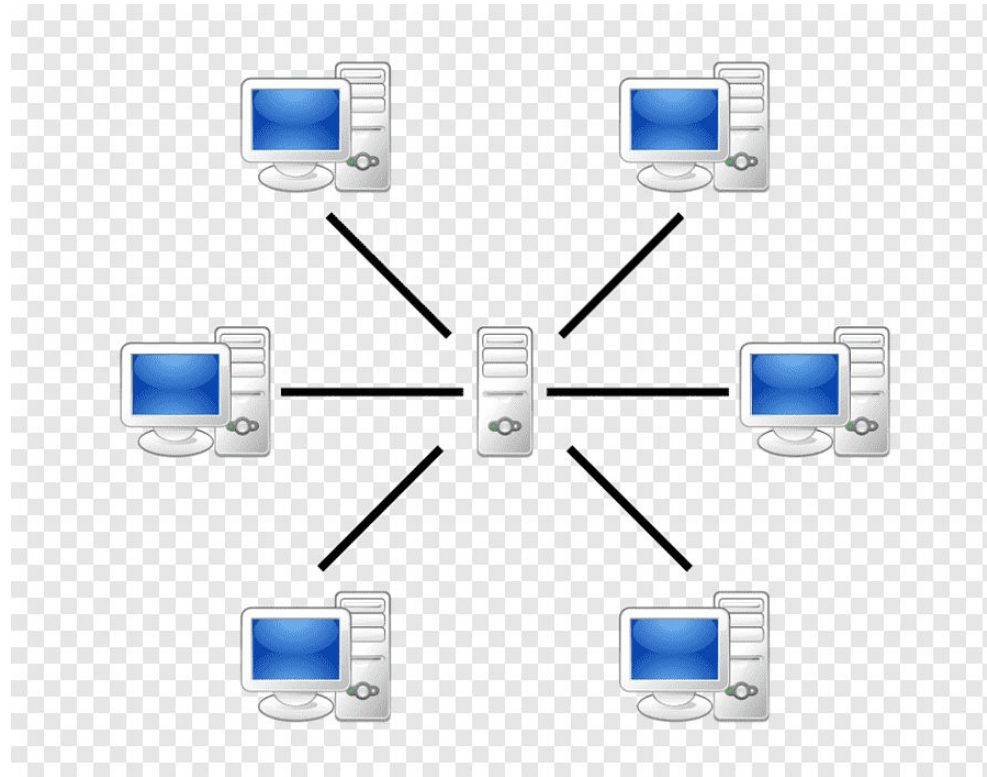
Réflexion pédagogique : ce cours a vocation à vous expliquer les couches dites “basses”, les 3 premières du modèle OSI.

Il existe de (très) nombreuses normes, en faire une liste exhaustive ne me paraît pas pertinent pour vous en enseignement direct.

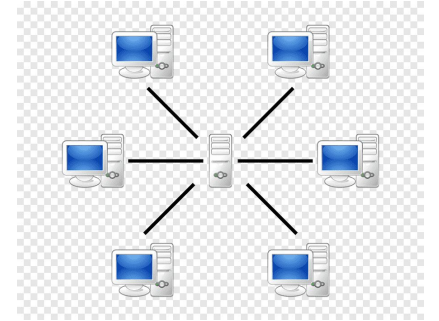
Réaliser un code de trame réseau et d'échange me paraît plus pertinent pour votre apprentissage, être actif de votre scolarité

Objectif désormais mise en pratique

Objectif : réaliser des programmes différents : un client (qui aura x instances (ici 6 dans ce schéma)) et un serveur (toujours 1)

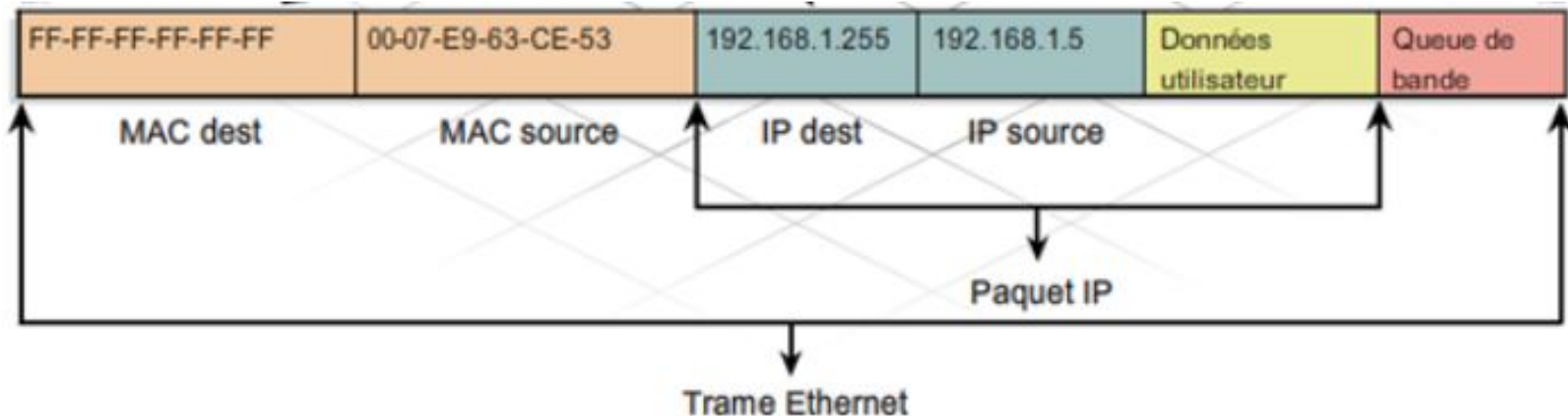


Objectif désormais mise en pratique

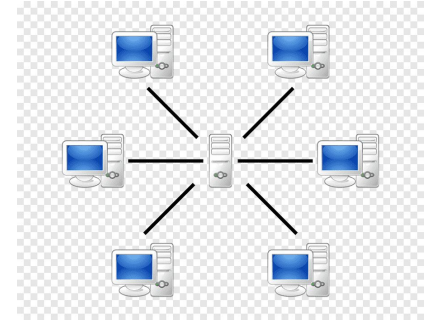


Votre programme simulera une adresse IP (V4) pour chaque client, et le serveur effectue le routage

Chaque trame envoyée pourra être par exemple comme une trame Ethernet, mais vous avez la liberté de concevoir la vôtre.



Objectif désormais mise en pratique



Les données utilisateurs seront fourni uniquement avec du code binaire : 0 , 1.

Les données utilisateurs seront du texte, qu'il faudra émettre par l'utilisateur, et affiché par le programme client cible, le serveur ne le modifie pas !

Le corps du message sera encodé en LZ78,

Objectif désormais mise en pratique

Contrainte(s) :

Nous allons utiliser le langage JAVA

2 à 3 personnes

Objectif désormais mise en pratique

Attention soutenance :

Une soutenance sera prévue pour présenter votre projet : il sera attendu une compréhension complète de l'ensemble de votre code. De nombreux bonus sont possibles si vous avez fini avant.

Ce n'est pas une présentation, ce sera des échanges sur des parties précises de votre code.